



Learn Anywhere



About the Course

- CRASH COURSE FOR SNOWFLAKE EXPERIENCE PROFESSIONALS
- ONLY 3 HOURS TO COVER ALL THE EXAM TOPICS FOR COF-CO2, COF-R02
- 2 FULL LENGTH PRACTICE TEST
- FREE EBOOK (CHEAT SHEET)
- NOT SUITABLE FOR FRESHERS

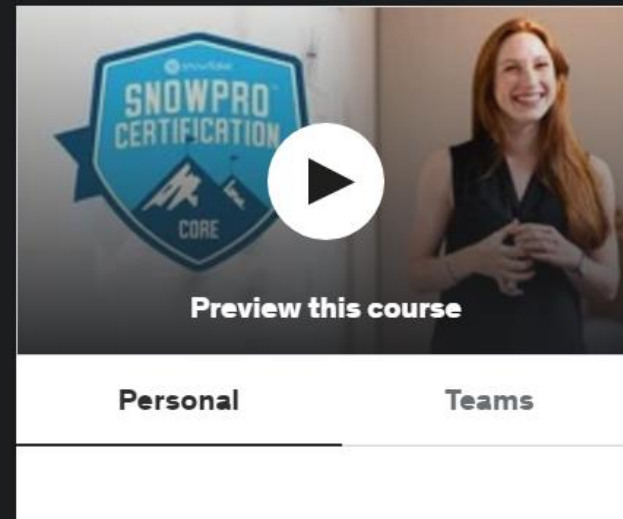
IT & Software > IT Certifications > Snowflake SnowPro Core Certification

SnowPro Core Certification COF-C02 Masterclass (2024)

Get Certified with Snowflake SnowPro Core COF-C02 || All topics covered with Hands-On || Practice Test || Free e-book

4.5 ★★★★★ (205 ratings) 1,695 students

Created by [Learn Anywhere](#), [Learn Anywhere](#), [Learn Anywhere](#)



For Any Queries drop a mail at Learn.Anywhere1@gmail.com



About the Author

- EXPERIENCED IN VARIOUS DATA TECHNOLOGIES
- CERTIFIED IN MULTIPLE TECHNOLOGIES
- 35+ YEAR OF TEAM EXPERIENCE
- MORE THAN 110K STUDENTS ON UDEMY CHANNELS

INSTRUCTOR

Learn Anywhere

Learning for Anyone, Anytime!

Udemy Instructor Partner

Total students Reviews

115,757 **28,783**



Who Should take **this course?**

- SNOWFLAKE DEVELOPER
- SNOWFLAKE DATA ENGINEERS
- SNOWPRO CORE RE/CERTIFICATION SEEKERS

Prerequisite

- HANDS ON EXPERIENCE WITH SNOWFLAKE
- FRESHER CAN ENROLL IN OUR OTHER SNOWFLAKE COURSE



Before Enrolling

- PLEASE CHECK THE COURSE DESCRIPTION
- WATCH PREVIEW VIDEOS
- CHECK IF YOU ARE COMFORTABLE WITH TEACHING STYLE
- 30 DAYS MONEY BACK GUARANTEE



The Certification

Exam Version:	COF-C02
Total Number of Questions:	100
Question Types:	Multiple Select, Multiple Choice, Interactive
Time Limit:	115 minutes
Languages:	English, Japanese, Korean
Passing Score:	750

Exam Version:	COF-R02
Total Number of Questions:	60
Question Types:	Multiple Select, Multiple Choice, Interactive
Time Limit:	85 minutes
Languages:	English and Japanese
Passing Score:	750 + Scaled Scoring from 0 - 1000
Prerequisites:	SnowPro Core Certified

The Certification

Domain	Domain Weightings on Exam
1.0 Snowflake AI Data Cloud Features and Architecture	25%
2.0 Account Access and Security	20%
3.0 Performance Concepts	15%
4.0 Data Loading and Unloading	10%
5.0 Data Transformations	20%
6.0 Data Protection and Data Sharing	10%



What is Snowflake?



What is Snowflake?

- Cloud based data platform
- Not built from old db tech
- Cloud native
- Separates Compute from Storage
- It is SaaS



What is Snowflake?

- Cloud based data platform
- Not built from old db tech
- Cloud native
- Separates Compute from Storage
- It is SaaS



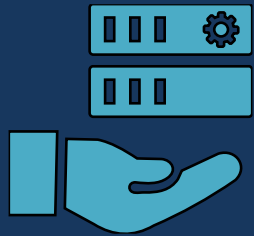
Software as a Service (SaaS)



Software as a Service (SaaS)



Quick setup and with minimal upfront cost.



It is fully managed and eliminates the need for customer-maintained infrastructure.



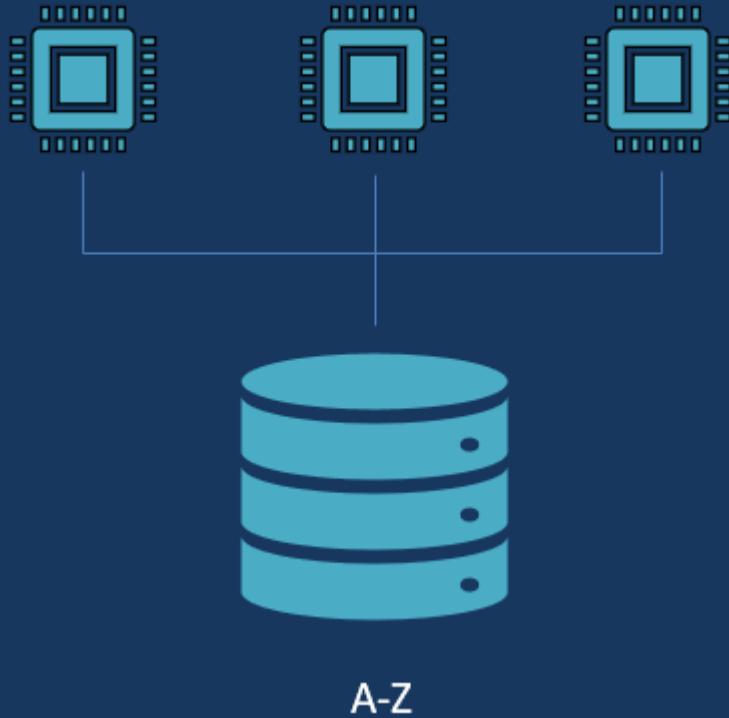
Snowflake uses a subscription-based pricing model.



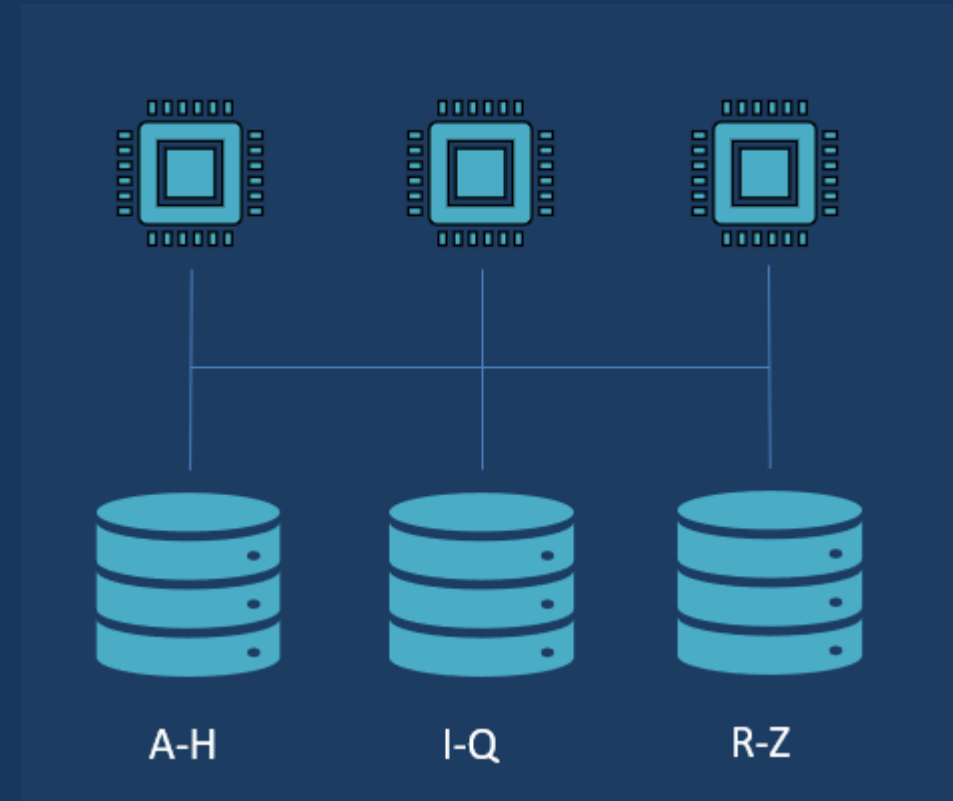
Traditional Architecture



Traditional Architecture



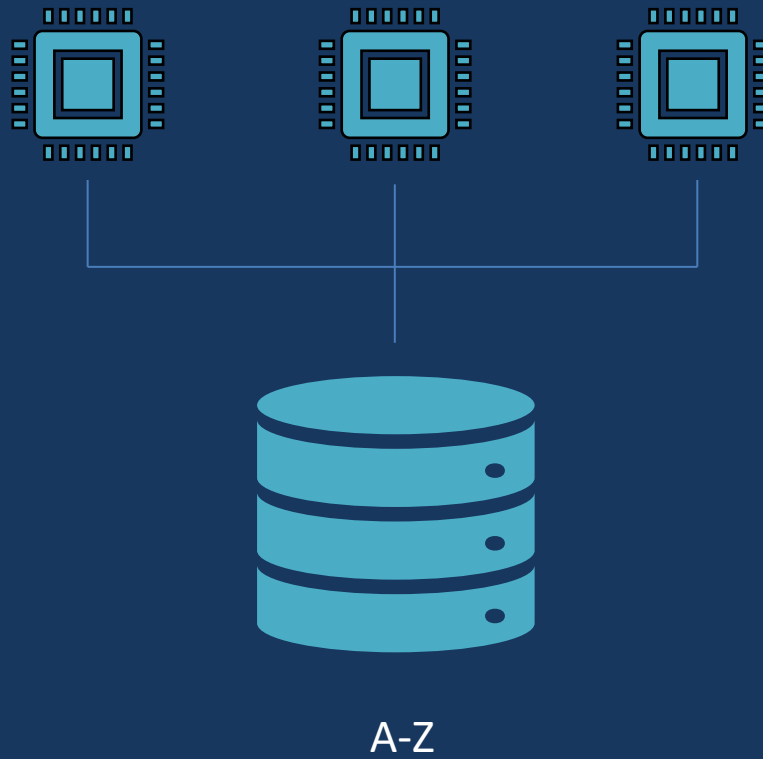
Shared Disk Architecture



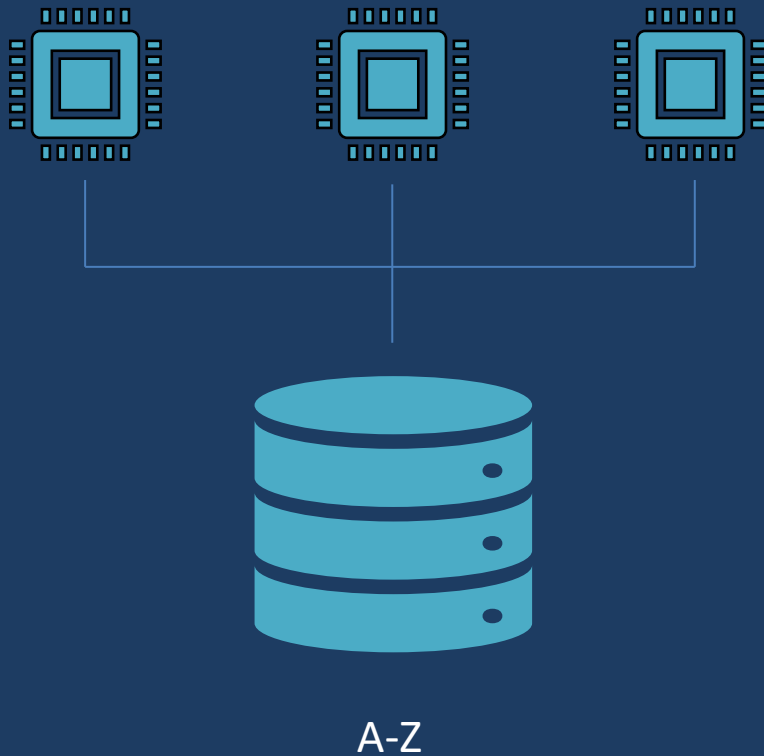
Shared Nothing Architecture



Shared Disk Architecture



Shared Disk Architecture



Pros :

- High data availability and reliability
- Easy data sharing between nodes
- Dynamic Load balancing for mixed workloads

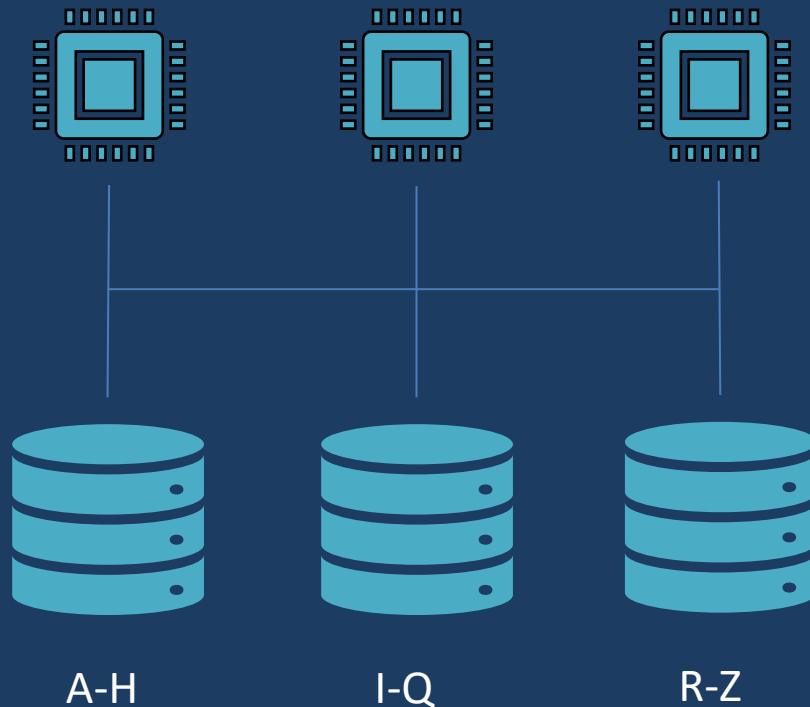
Cons:

- Single point of failure as shared resource.
- Performance issues as nodes/Volume increases.
- May have Network bottlenecks

Oracle Real Application Clusters



Shared Nothing Architecture



Pros :

- Highly scalable architecture
- Better performance due to reduced network traffic
- Flexible HW and SW configurations

Cons:

- Data partitioning can be time-consuming
- Fixed Load balancing
- Can lead to data skew

Apache Cassandra, Teradata

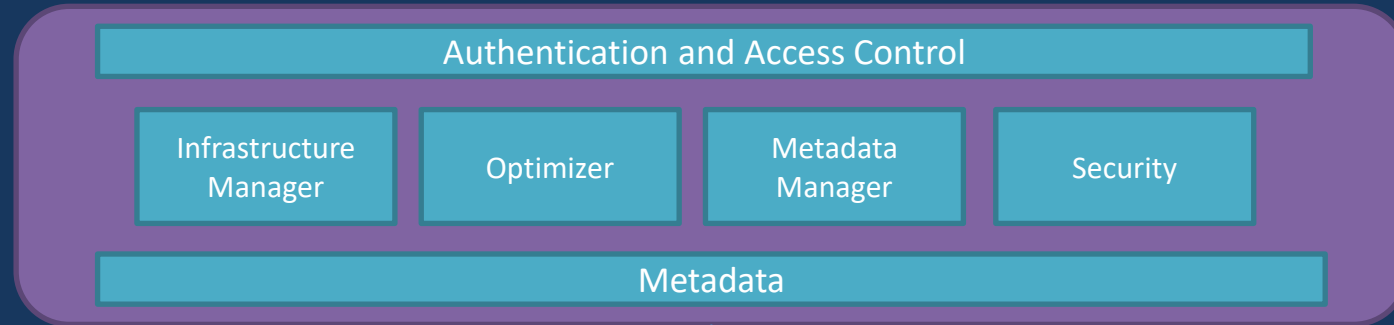


Snowflake Architecture

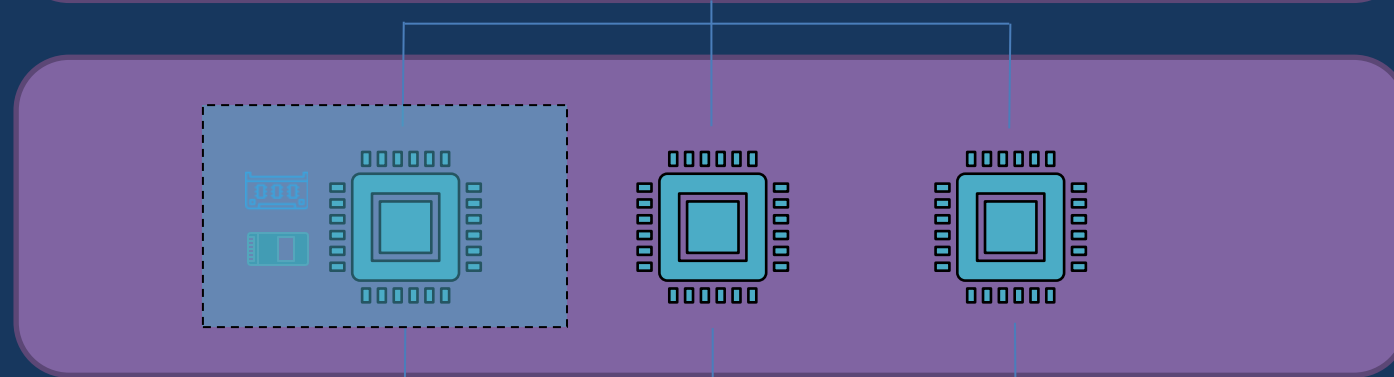


Snowflake Architecture

Cloud Services



Query Processing Layer



Database Storage



Snowflake Architecture

Database Storage

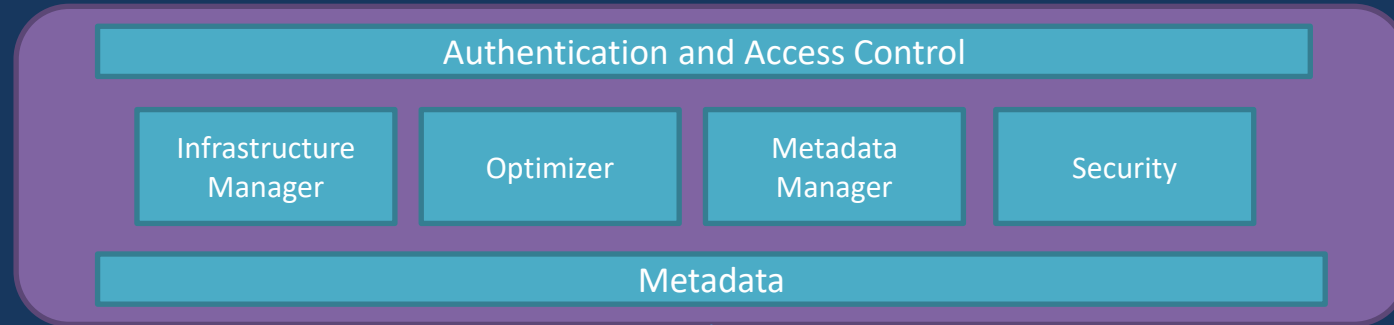


- Elastic Storage (Decoupled from Compute, Scale up and Down easily)
- Snowflake organizes data into optimized, Compressed and columnar format.
- Data is stored in cloud storage.
- Snowflake stores data in centralized DB, accessible to all nodes.
- Data objects are not directly accessible.
- Accessible only through Snowflake SQL queries.

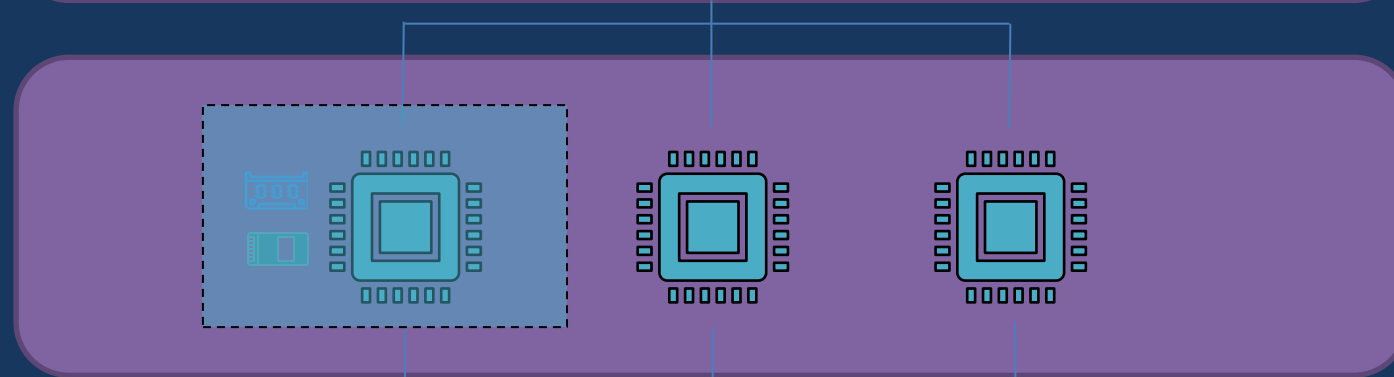


Snowflake Architecture

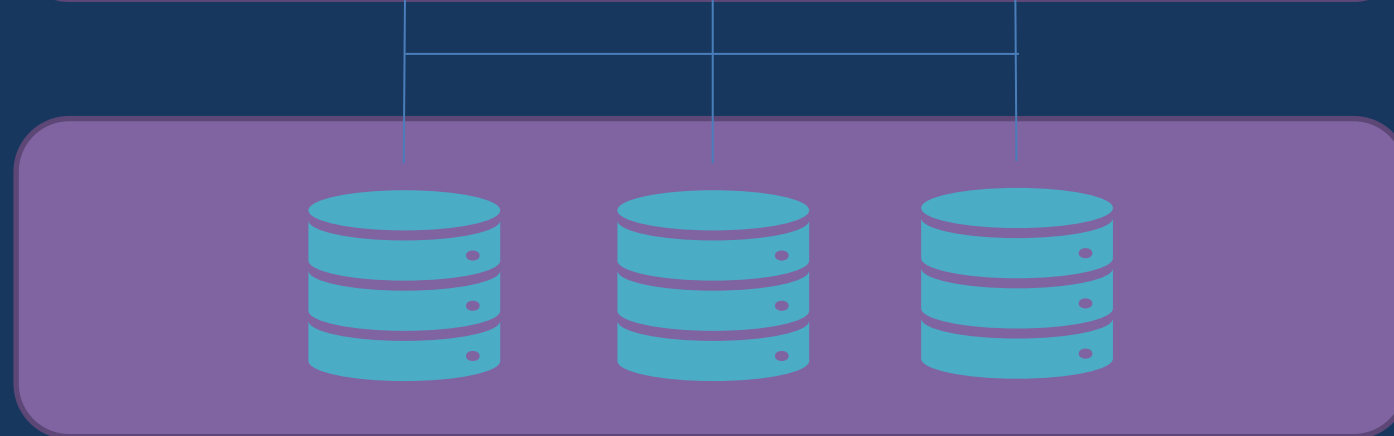
Cloud Services



Query Processing Layer

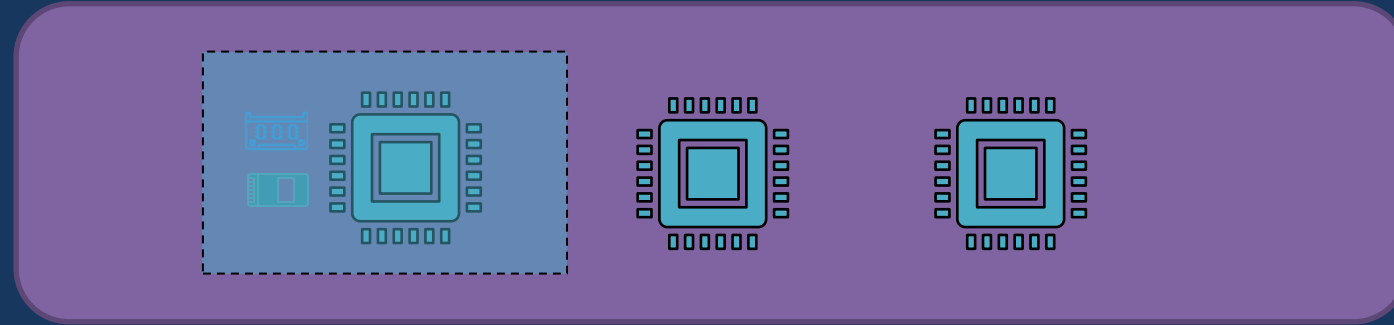


Database Storage



Snowflake Architecture

Query Processing Layer

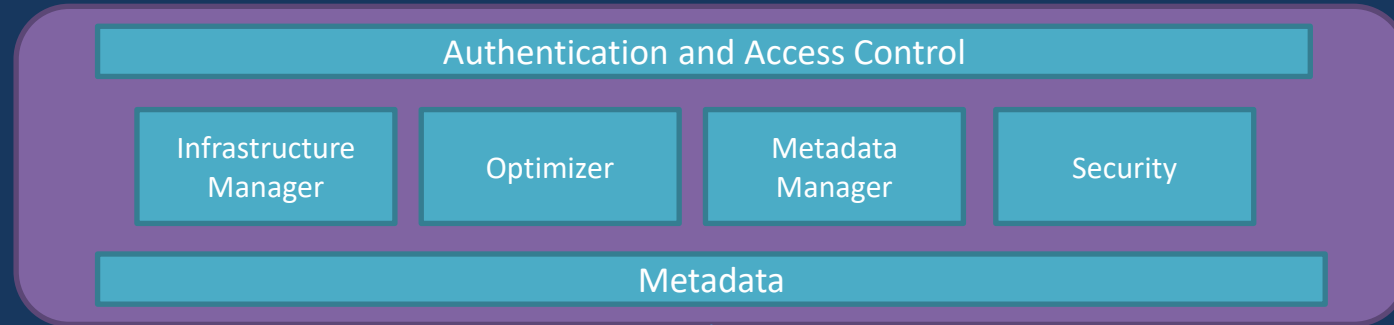


- Elastic Compute (Decoupled from Storage, Scale up and Down easily)
- Queries executed in the processing layer.
- Snowflake uses "virtual warehouses" for processing.
- Each VW is MPP cluster
- Each VW has its own temp memory and cache.
- Each warehouse is independent, with no impact on other warehouses performance.
- Each query result remains in cache, and purged after a point.

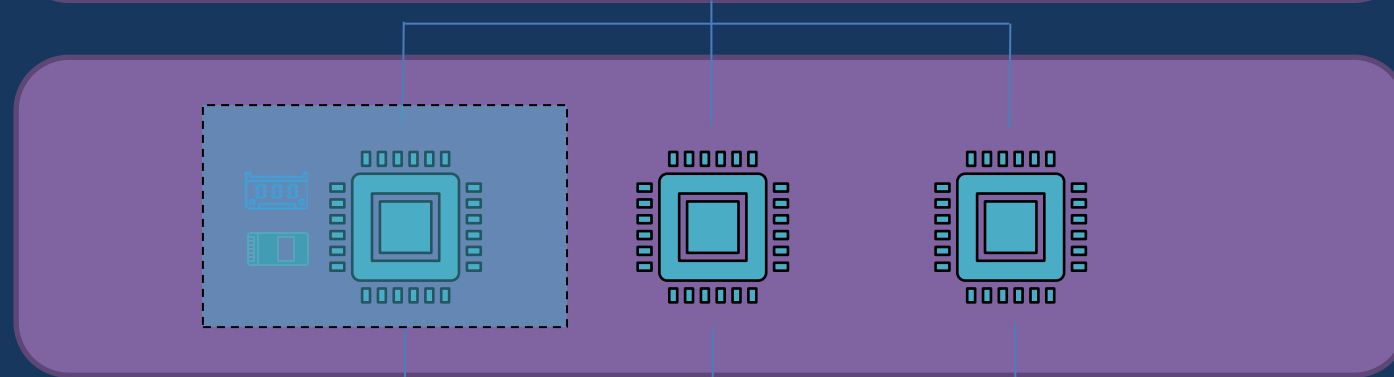


Snowflake Architecture

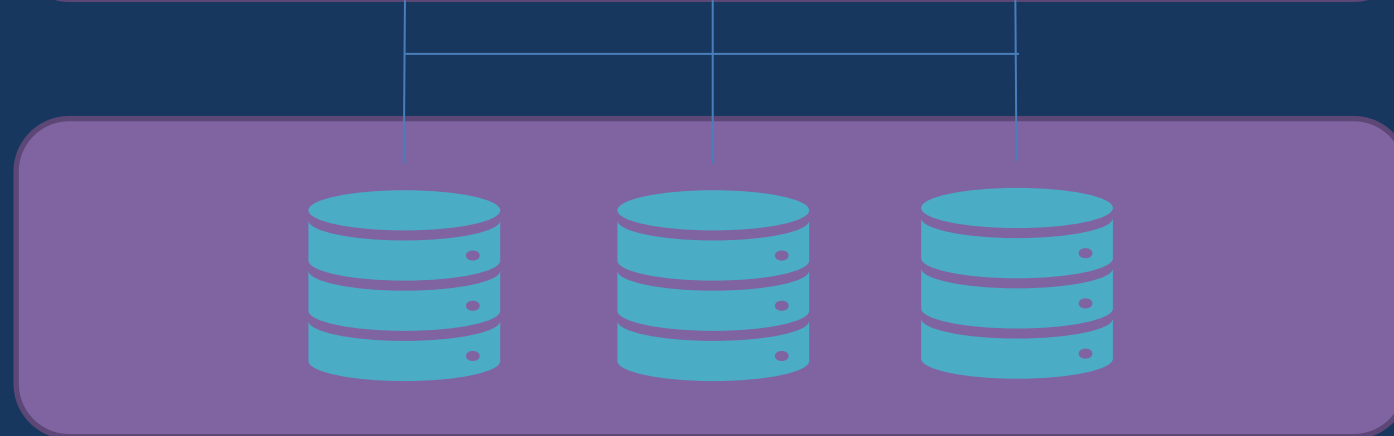
Cloud Services



Query Processing Layer

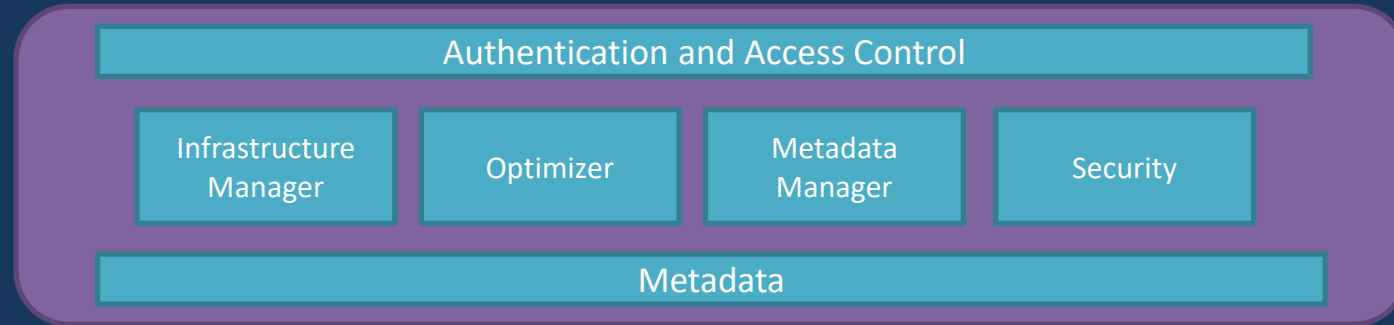


Database Storage



Snowflake Architecture

Cloud Services

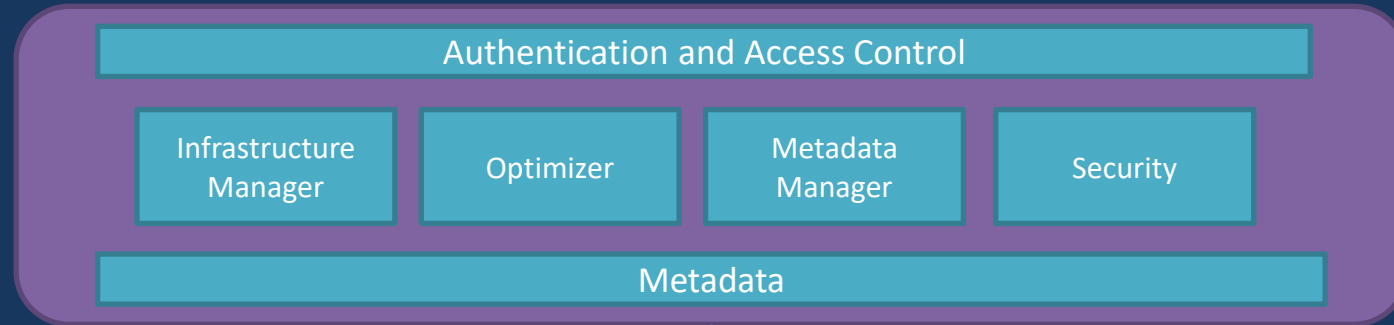


- **Authentication and access control** verifies user identity and manages access to data.
- **Infrastructure manager** provisions and manages compute resources for Snowflake.
- **Snowflake optimizer** determines the most efficient way to execute a query.
- **Metadata Manager** manages the metadata associated with data stored in Snowflake.
- **Snowflake Security** includes encryption, network isolation, and other security features to protect data.

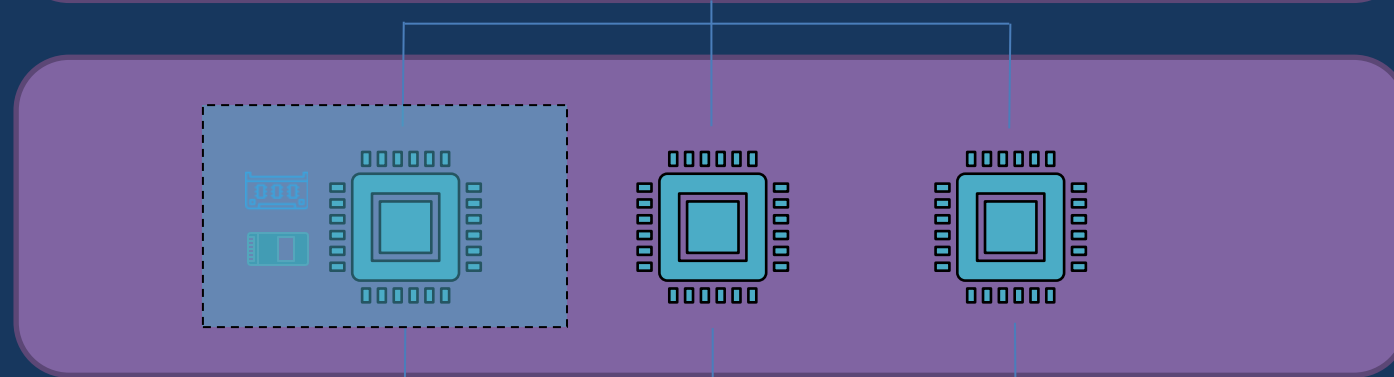


Snowflake Architecture

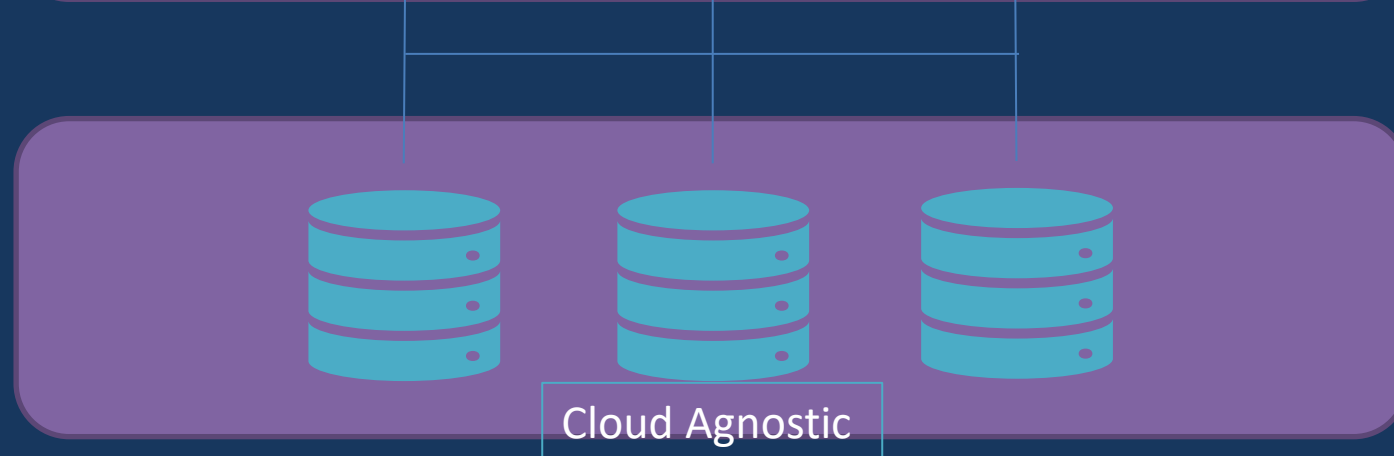
Cloud Services



Query Processing Layer



Database Storage



Snowflake Editions



Snowflake Editions



Standard



Enterprise



Business Critical



VPS



Snowflake Editions



Standard



Enterprise



Business Critical



VPS

Standard	Enterprise	Business Critical	VPS
All Standard Features	All Standard Features	All features of Enterprise	Most features of Business Critical
	Multi Clustered VW	Failover and DR	Highest level of Security
	Materialized View, Search Optimization	Redirecting Client Connection	Separate Environment
	Extended Time Travel	HIPAA, HITRUST CSF Environment	

<https://docs.snowflake.com/en/user-guide/intro-editions>



Snowflake Editions



Standard



Enterprise



Business Critical



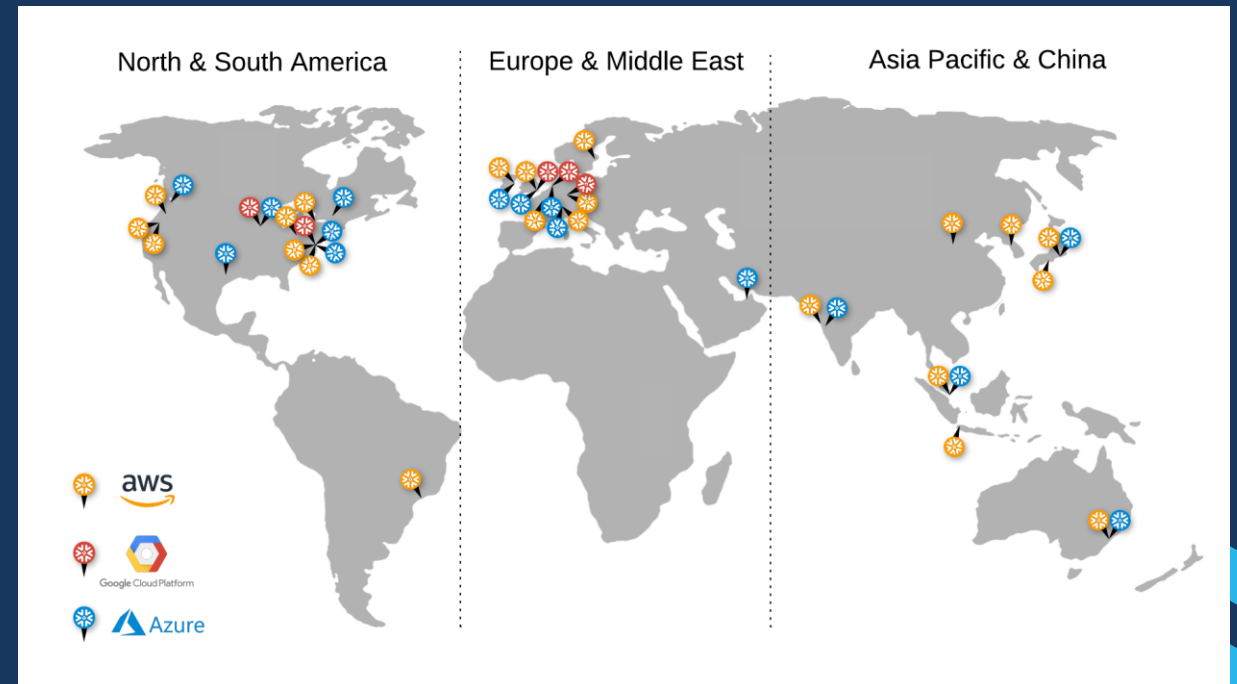
VPS



Google Cloud



Azure







Snowflake Partner

- Data integration
- ML/Data Science
- Security, Governance & Observability
- Business Intelligence (BI)
- CI/CD
- SQL Development & Management
- Native programming Interface





- Data integration,
- ML/Data Science,
- Security, Governance & Observability
- Business Intelligence (BI)
- CI/CD
- SQL Development & Management
- Native programming Interface



Snowflake Partner



+ Create

Home

Search

Projects

Data

Data Products

Marketplace

Apps

Private Sharing

Provider Studio

Partner Connect

AI & ML

Monitoring

Admin

Partner Connect

Search Partner Connect

[All Categories](#) [Business Intelligence](#) [CI/CD](#) [Data Integration](#) [More](#) ▼

Providers [All](#)

Status [All](#)

Business Intelligence

CARTO

CARTO

Get the most from spatial data with interactive maps, analysis workflows, applications, H3 and more.



Domo

High-leverage BI on Snowflake with mobile-first, intelligent apps for business.



Sigma

Cloud-native analytics platform empowering business users to make data-driven decisions.



Sisense

Empowering builders to simplify complex data and transform it into powerful analytic apps.



Snowflake UI





Snowflake UI

+ Create

Home

Search

Projects

Data

Data Products

AI & ML

Monitoring

Admin

\$400 credits left

Trial ends in 29 days

Upgrade

LA

learnr any

ACCOUNTADMIN

Home

PREVIEW

Search this account, Marketplace, and documentation

Quick actions

Upload local files

Quickly convert data into tables

Load from cloud storage

Use a SQL template to load data

Query data

Create a SQL Worksheet

Create User

Provide access to collaborators

All projects

All projects Worksheets Notebooks Streamlit Dashboards

TITLE	TYPE	VIEWED ↓	UPDATED
Load sample data with SQL from S3 bucket Getting Started Tutorials	SQL Worksheet	—	16 hours ago
Load sample data with Python from S3 bucket Getting Started Tutorials	Python Worksheet	—	16 hours ago
[Template] Adding a user and granting roles Getting Started Tutorials	SQL Worksheet	—	16 hours ago
Sample queries on TPC-H data Benchmarking Tutorials	SQL Worksheet	—	16 hours ago
Sample queries on TPC-DS data Benchmarking Tutorials	SQL Worksheet	—	16 hours ago
TPC-DS 10TB Complete Query Test Benchmarking Tutorials	SQL Worksheet	—	16 hours ago
TPC-DS 100TB Complete Query Test Benchmarking Tutorials	SQL Worksheet	—	16 hours ago

Snowsight provides a unified experience for working with your Snowflake data by using SQL or Python.





Snowflake CLI

```
snowsq | snowsq -a -u - 94x29
#COMPUTE_WH@SF_TUTS.PUBLIC>create or replace warehouse sf_tuts_wh with
                             warehouse_size='X-SMALL'
                             auto_suspend = 180
                             auto_resume = true
                             initially_suspended=true;
+-----+
| status |
+-----+
| Warehouse SF_TUTS_WH successfully created. |
+-----+
1 Row(s) produced. Time Elapsed: 3.662s
#SF_TUTS_WH@SF_TUTS.PUBLIC>
```

SnowSQL

- CLI based tool
- Performs DML/DDDL
- Loading and unloading data



Snowflake Drivers & Connectors



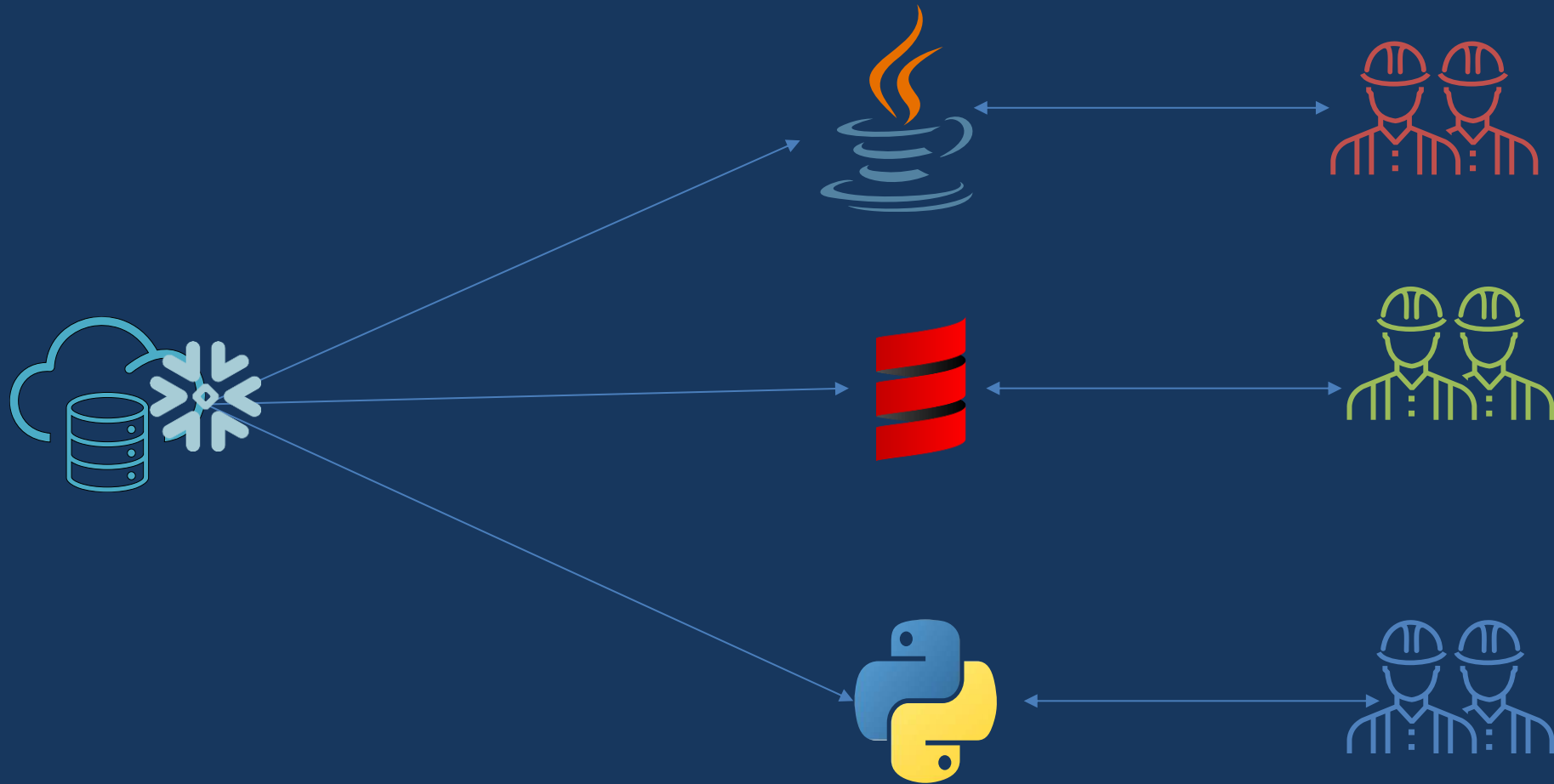
Snowflake Drivers & Connectors





SnowPark

Set of libraries and code execution environments that run Python, Scala, Java programming languages next to your data in Snowflake.







SnowCD : SnowCD (i.e. Snowflake Connectivity Diagnostic Tool) helps users to diagnose and troubleshoot their network connection to Snowflake.

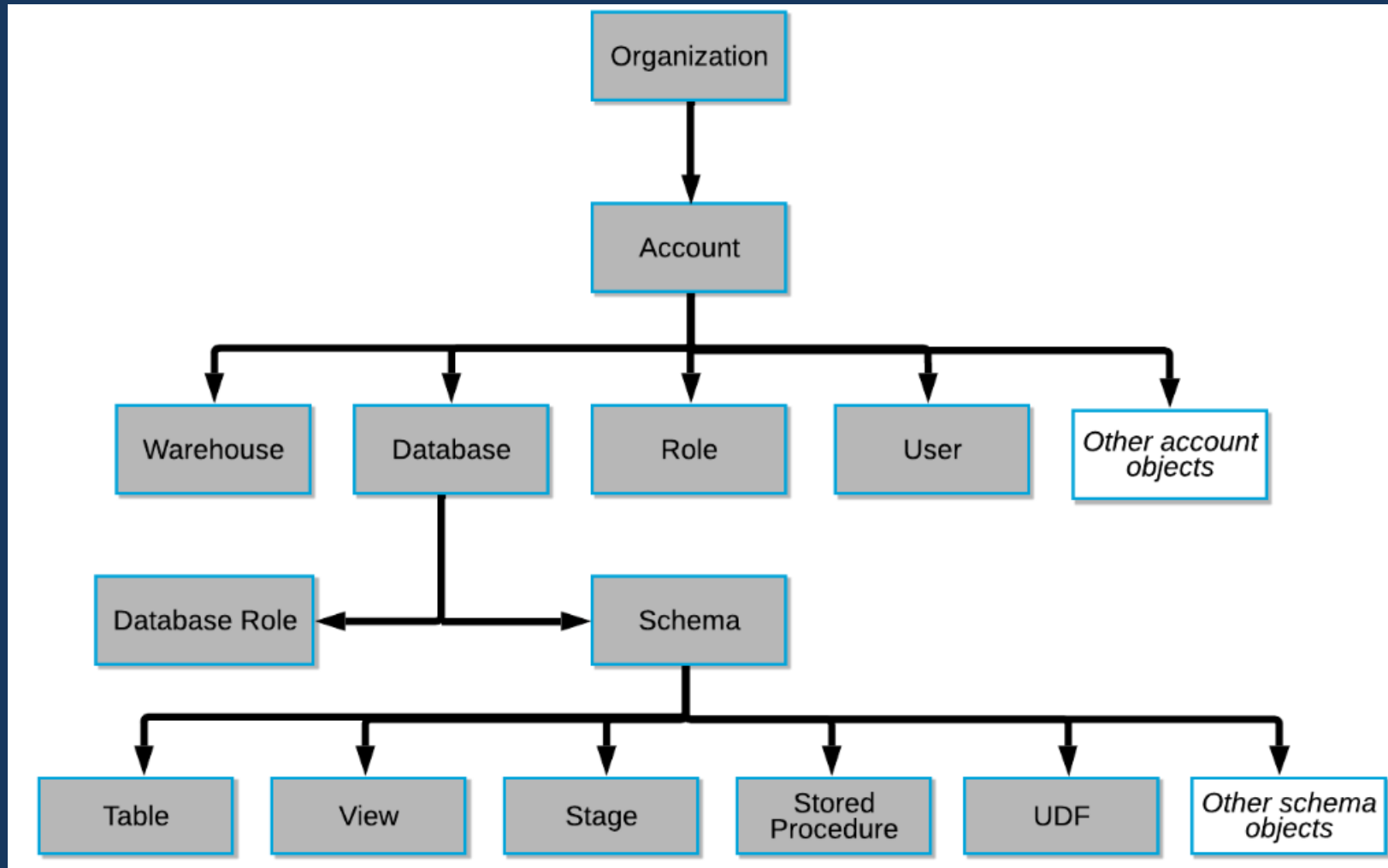
Runs multiple Connection checks to make sure your connection to Snowflake is fine

SnowCD leverages IP Listed by

- `SYSTEM$ALLOWLIST()`
- `SYSTEM$ALLOWLIST_PRIVATELINK()` 



Hierarchy Of Containers



Database



Database : A database is a logical grouping of schemas. Each database belongs to a single Snowflake account.

```
-- Create DATABASE  
  
CREATE DATABASE LA_DB;  
  
-- Display DATABASE METADATA  
  
SHOW DATABASES LIKE 'LA%';
```

Database

CREATE DATABASE

ALTER DATABASE

DESCRIBE DATABASE

DROP DATABASE

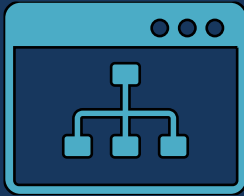
UNDROP DATABASE

USE DATABASE

SHOW DATABASES



Schema



Schema : A schema is a logical grouping of database objects (tables, views, etc.). Each schema belongs to a single database

```
-- Create SCHEMA  
  
CREATE SCHEMA LA_SCHEMA;  
  
-- Display SCHEMA METADATA  
  
SHOW SCHEMAS LIKE 'LA%';
```

Schema

ALTER SCHEMA

DESCRIBE SCHEMA

CREATE SCHEMA

DROP SCHEMA

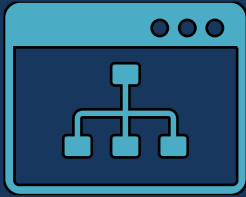
UNDROP SCHEMA

USE SCHEMA

SHOW SCHEMAS



Schema



Schema : A schema is a logical grouping of database objects (tables, views, etc.). Each schema belongs to a single database

Transient Schema don't have fail-safe option, also all tables under such schema are transient as well.

```
CREATE TRANSIENT SCHEMA tschema;
```

In regular schemas, the owner of an object can grant further privileges on their objects to other roles. In **managed access schemas**, the schema owner manages all privilege grants, including future grants, on objects in the schema.

```
CREATE SCHEMA mschema WITH MANAGED ACCESS;
```



Data Types



Numeric

- Number
- Decimal
- Numeric
- INT, FLOAT,
- DOUBLE



String/Binary

- VARCHAR
- CHAR
- STRING, TEXT
- BINARY



Logical

- BOOLEAN



Date & Time

- DATE
- DATETIME
- TIMESTAMP



Geospatial

- GEOGRAPHY
- GEOMETRY



Semi-Structured

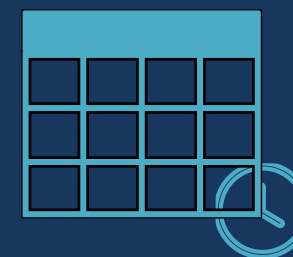
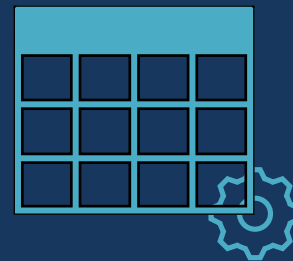
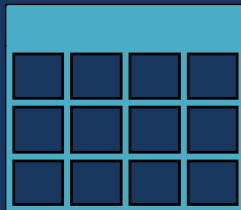
- VARIANT
- OBJECT
- ARRAY

```
CREATE OR REPLACE TABLE my_table (  
  id INT primary key,  
  name VARCHAR(50) NOT NULL,  
  email VARCHAR(100)  
);
```



Tables

Types of Tables



Features	Permanent	Transient	Temporary
Persistence	Until Dropped	Until Dropped	Session
Cloning	To Any type	To Transient or Temporary	To Transient or Temporary
Time Travel (Days, Default is 1)	 0-1 (0-90) 	0-1	0-1
Fail Safe (Days, Fixed)	7	0	0

- External table, none of the above criteria applies.
- There are more tables like Dynamic, Event, Iceberg and Hybrid.



Tables

```
-- CREATE A PERMANENT TABLE (by Default)
CREATE TABLE PERMANENT_TABLE (
    ID NUMBER,
    ...
);

-- CREATE A TRANSIENT TABLE
CREATE TRANSIENT TABLE TRANSIENT_TABLE (
    ID NUMBER,
    ...
);

CREATE TEMPORARY TABLE TEMPORARY_TABLE (
    ID NUMBER,
    ...
);
```



Tables

```
-- Create external Tables

create external table et2(
  col1 date as (parse_json(metadata$external_table_partition):COL1::date),
  col2 varchar as (parse_json(metadata$external_table_partition):COL2::varchar),
  col3 number as (parse_json(metadata$external_table_partition):COL3::number))
partition by (col1,col2,col3)
location=@s2/logs/
partition_type = user_specified
file_format = (type = parquet);
```

--External tables include the following metadata column:

METADATA\$FILENAME: Name of each staged data file included in the external table. Includes the path to the data file in the stage.

METADATA\$FILE_ROW_NUMBER: Row number for each record in the staged data file.

- No Time Travel, No DML
- S3,Azure and GCS, No support for archived data in cloud





External Table

- Use external table (GCP, AWS, AZURE) as if it is local Snowflake table
- Query the table, cant do DML operations
- Snowflake stores file metadata, filename, version identifier
- Queries are slower than native SF tables
- Partitioning table can enhance performance





External Table

Schema on read

All external tables include the following columns:

VALUE	A VARIANT type column that represents a single row in the external file.
METADATA\$FILENAME	A pseudocolumn that identifies the name of each staged data file included in the external table, including its path in the stage.
METADATA\$FILE_ROW_NUMBER	A pseudocolumn that shows the row number for each record in a staged data file.





External Table

General file sizing recommendations

To optimize the number of parallel scanning operations when querying external tables, we recommend the following file or row group sizes per format:

Format	Recommended Size Range	Notes
Parquet files	256 - 512 MB	
Parquet row groups	16 - 256 MB	Note that when Parquet files include multiple row groups, Snowflake can operate on each row group in a different server. For improved query performance, we recommend sizing Parquet files in the recommended range; or, if large file sizes are necessary, including multiple row groups in each file.
All other supported file formats	16 - 256 MB	





External Table

```
-- Create External Table

CREATE OR REPLACE EXTERNAL TABLE <Table_Name>(
  col1 VARCHAR AS (value:c1::VARCHAR),
  col2 VARCHAR AS (value:c2::VARCHAR),
  col3 VARCHAR AS (value:c3::VARCHAR),

)
WITH
LOCATION = <External Stage>
PATTERN = '.*mock.*'
FILE_FORMAT = <File Format>
;
```

```
SELECT * FROM <Table_name>
```

```
--Manual refresh

ALTER EXTERNAL TABLE <table_name> REFRESH;

-- Refresh the metadata for a single day of data files by date

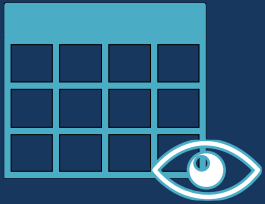
ALTER EXTERNAL TABLE <table_name> REFRESH '2024/08/05/';

--Auto Refresh

ALTER EXTERNAL TABLE <table_name> SET AUTO_REFRESH = { TRUE | FALSE }
```

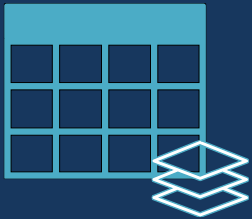


Snowflake Views



Non-Materialized View

Typical View, Virtual, No storage of Data



Materialized View

Behaves like a table, Serverless, Faster, Lots of Limitation*



Secure View

For extra layer of security, can be done for both the views



Snowflake Views

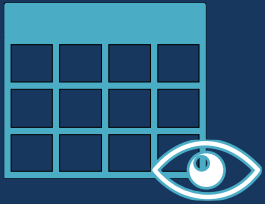
```
CREATE OR REPLACE VIEW CUSTOMER_VIEW AS  
SELECT CUSTOMER_ID, ... FROM CUSTOMER;
```

```
CREATE OR REPLACE MATERIALIZED VIEW CUSTOMER_VIEW AS  
SELECT CUSTOMER_ID, ... FROM CUSTOMER;
```

```
CREATE OR REPLACE SECURE MATERIALIZED VIEW CUSTOMER_VIEW AS  
SELECT CUSTOMER_ID, ... FROM CUSTOMER;
```

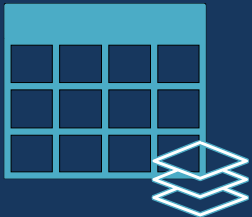


Snowflake Views



Non-Materialized View

```
CREATE OR REPLACE VIEW CUSTOMER_VIEW AS  
SELECT CUSTOMER_ID, .... FROM CUSTOMER;
```



Materialized View

```
CREATE OR REPLACE MATERIALIZED VIEW CUSTOMER_VIEW AS  
SELECT CUSTOMER_ID, .... FROM CUSTOMER;
```

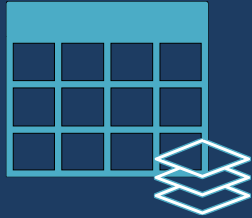


Secure View

```
CREATE OR REPLACE SECURE MATERIALIZED VIEW CUSTOMER_VIEW AS  
SELECT CUSTOMER_ID, .... FROM CUSTOMER;
```



Materialized Views



Table

When to Create Materialized View?

- Query results contain a small number of rows and/or columns
- Query results contain results that require significant processing, including:
 - Analysis of semi-structured data
 - Aggregates that take a long time to calculate
- The query is on an external table
- The base table does not change frequently



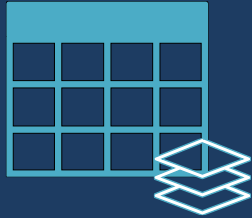
Materialized Views

Advantage:

- Materialized views can improve the performance of repeat queries.
- Materialized views are automatically by Snowflake.
- Data accessed through materialized views is always Up-to-date.

Cost:

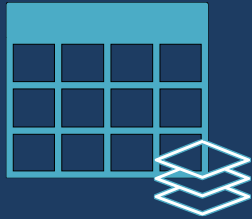
- Storage
- Compute (Managed by SF)



Table



Materialized Views



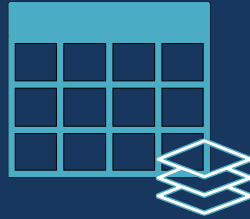
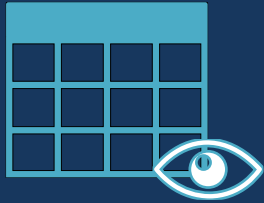
Table

Limitation:

- Only on single table
- Joins are not supported
- Many aggregate functions not allowed
- Full list : <https://docs.snowflake.com/en/user-guide/views-materialized#limitations-on-creating-materialized-views>



Snowflake Views



Definition

Storage

Use Case

Non-Materialized View	Materialized View	Secure View
A virtual table based on the result of a SELECT statement.	A pre-computed and stored table based on the result of a SELECT statement.	Non-Materialized and Materialized view can be set as Secure View
Data is not stored, it is computed on the fly each time the view is queried.	Data is stored in the cache, allowing for faster query execution.	Same as underlying View.
Data transformation, providing a simplified view of the underlying data.	Reporting and analytics, frequently queried data.	When you want to limit access to the data



Snowflake Data Loading



Snowflake Data Loading



Various Data Types



Data Source



Stage

“Snowflake refers to the location of data files in cloud storage as a stage”



Internal



External



Stage



Internal



External



Table Stage	User Stage	Named Stage
Auto Created for each table	Auto Created for each User	Created by user
Can be used by Multi-user for 1 table	Can be used by single user for multiple tables	Used as any other object
Cant be altered/dropped	Cant be altered/dropped	Can be altered/dropped
@%	@~	@
Load using PUT	Load using PUT	Load using PUT

Named Stage (Optional)
Created by user
Refers object on cloud
Specify a URL
Cant point to Archived data



Stage

```
-- Table Stage : Load file via snowsql
put file://C:\data\MOCK.CSV @%customers_table/staged AUTO_COMPRESS = FALSE;

-- User Stage : Load file via snowsql

put file://C:\data\MOCK.CSV @~/customers_user/staged AUTO_COMPRESS = FALSE;

-- Create Named stage

CREATE
OR REPLACE STAGE internal_named_stage file_format = (
    type = 'CSV' FIELD_DELIMITER = ',' SKIP_HEADER = 1
);

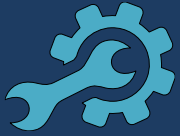
-- Internal Stage : Load via Snowsql

put file://C:\data\MOCK.CSV @internal_named_stage AUTO_COMPRESS = FALSE;

-- Create External Stage

CREATE OR REPLACE STAGE my_s3_stage
STORAGE_INTEGRATION = sample_int
URL = 's3://learn-train-bucket/MOCK.csv'
FILE_FORMAT = mycsvformat;
```





User Defined Functions (UDFs)

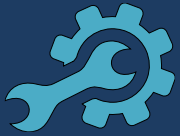
“With user-defined functions (UDFs), you can extend the system to perform operations that are not available through the built-in, system-defined functions provided by Snowflake.”

Scalar UDF



Table UDF (UDTF)





User Defined Functions (UDFs)

UDFs are DB object, created in specific DB and Schema.

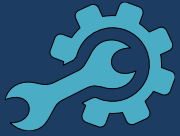
db.schema.<function_name>

demodb.demoschema.udf_sample

SELECT demodb.demoschema.udf_sample()

*Not allowed to create UDF with same name as system defined function





User Defined Functions (UDFs)

```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
LANGUAGE JAVASCRIPT
AS
$$
return x * x;
$$;
```





User Defined Functions (UDFs)

```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
LANGUAGE JAVASCRIPT
AS
$$
return x * x;
$$;
```



```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
LANGUAGE PYTHON
AS
$$
    return x * x
$$;
```



```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
AS
$$
SELECT x * x;
$$;
```



```
CREATE OR REPLACE FUNCTION square(x FLOAT)
RETURNS FLOAT
LANGUAGE JAVA
AS
$$
    class Square {
        public static float square(float x) {
            return x * x;
        }
    }
    return Square.square(x);
$$;
```



Overloading Functions

```
CREATE OR REPLACE FUNCTION add_numbers(x NUMBER)
RETURNS NUMBER
AS
$$
SELECT x + 8;
$$;
```

```
CREATE OR REPLACE FUNCTION add_numbers(x STRING)
RETURNS STRING
AS
$$
SELECT 'x || "8"';
$$;
```

```
SELECT add_numbers(1); -- Returns 9
SELECT add_numbers('1'); -- Returns 18
```





User Defined Table Functions (UDTFs)

```
CREATE OR REPLACE FUNCTION get_orders_by_customer(customer_id INT)
RETURNS TABLE (order_id INT, order_date DATE, total_amount FLOAT)
AS
$$
SELECT order_id, order_date, total_amount
FROM orders
WHERE customer_id = customer_id;
$$;
```

```
SELECT * FROM TABLE(get_orders_by_customer(1234));
```





Stored Procedure

A stored procedure is a named, precompiled collection of SQL statements and control-of-flow language constructs that are stored in a database and can be called from SQL CALL command, including other stored procedures.

```
CREATE OR REPLACE PROCEDURE procedure_name (parameter_list)
RETURNS return_type NOT NULL
LANGUAGE {SQL | JAVASCRIPT | PYTHON | JAVA}
EXECUTE AS CALLER/OWNER
AS
$$
BEGIN
RETURN message;
END;
$$;
```

```
CALL procedure_name(parameter_values);
```





Stored Procedure



Snowpark



Snowflake Scripting



Java



Python



Scala



JavaScript



SQL



UDF vs Stored Procedure

<i>Feature</i>	<i>User-Defined Function (UDF)</i>	<i>Stored Procedure</i>
Return Value	Must return a value	Not Mandatory to return a value
Use case	When part of SQL stmt	For DML, DDL Operations
Languages	Java, JS, Python, Scala, SQL	Java, JS, Python, Scala, Snowflake Scripting
Granularity	Can be called for every row of a query result set	Can only be executed as a standalone command
Inside SELECT	Multiple UDFs can be called	Single SP with a statement, Can have nested SP
DB Access	May not have access	Has access



Sequence

Sequences are used to generate unique numbers across sessions and statements; however, Snowflake doesn't guarantee generating sequence numbers without gaps.

Syntax : `CREATE OR REPLACE SEQUENCE SEQUEN_EX START = 1 INCREMENT = 1;`

```
SELECT SEQUENCE_SAMPLE2.nextval a, SEQUENCE_SAMPLE2.nextval b, SEQUENCE_SAMPLE2.nextval c,  
SEQUENCE_SAMPLE2.nextval d;
```

-Default value for the INCREMENT and START is 1.





Micro-Partitioning





Micro-Partitioning



Traditional DBs have static partitioning to manage performance and scaling , with limitations like data skew and maintenance overheads



Snowflake overcomes limitations with Micro-Partition





Micro-Partitioning

- Snowflake automatically divides table in micro-partitions
- Each micro-partition contain 50-500 MB of uncompressed data (SF compress it further and stores)
- Organized in columnar format
- Very large tables can have multi-million micro-partitions
- Micro-partitions are immutable (Time-Travel, Cloning)





Micro-Partitioning

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)																								
type	<table><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>2</td></tr></table>	2	4	3	2	3	2	<table><tr><td>3</td><td>2</td><td>4</td></tr><tr><td>5</td><td>1</td><td>5</td></tr></table>	3	2	4	5	1	5	<table><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>5</td><td>3</td></tr></table>	2	4	2	1	5	3	<table><tr><td>1</td><td>4</td><td>5</td></tr><tr><td>5</td><td>3</td><td>2</td></tr></table>	1	4	5	5	3	2
2	4	3																										
2	3	2																										
3	2	4																										
5	1	5																										
2	4	2																										
1	5	3																										
1	4	5																										
5	3	2																										
name	<table><tr><td>A</td><td>C</td><td>C</td></tr><tr><td>B</td><td>A</td><td>C</td></tr></table>	A	C	C	B	A	C	<table><tr><td>Z</td><td>B</td><td>C</td></tr><tr><td>X</td><td>A</td><td>A</td></tr></table>	Z	B	C	X	A	A	<table><tr><td>X</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>A</td></tr></table>	X	Z	Y	B	X	A	<table><tr><td>C</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>Z</td></tr></table>	C	Z	Y	B	X	Z
A	C	C																										
B	A	C																										
Z	B	C																										
X	A	A																										
X	Z	Y																										
B	X	A																										
C	Z	Y																										
B	X	Z																										
country	<table><tr><td>UK</td><td>SP</td><td>DE</td></tr><tr><td>DE</td><td>FR</td><td>SP</td></tr></table>	UK	SP	DE	DE	FR	SP	<table><tr><td>DE</td><td>UK</td><td>NL</td></tr><tr><td>FR</td><td>NL</td><td>FR</td></tr></table>	DE	UK	NL	FR	NL	FR	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK
UK	SP	DE																										
DE	FR	SP																										
DE	UK	NL																										
FR	NL	FR																										
FR	NL	SP																										
SP	DE	UK																										
FR	NL	SP																										
SP	DE	UK																										
date	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr></table>	11/2	11/2	11/2	11/2	11/2	11/2	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/3</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/3	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/4</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/4	<table><tr><td>11/3</td><td>11/4</td><td>11/4</td></tr><tr><td>11/5</td><td>11/5</td><td>11/5</td></tr></table>	11/3	11/4	11/4	11/5	11/5	11/5
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/3	11/3	11/3																										
11/2	11/2	11/2																										
11/3	11/3	11/4																										
11/3	11/4	11/4																										
11/5	11/5	11/5																										





Micro-Partitioning

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)																								
type	<table><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>2</td></tr></table>	2	4	3	2	3	2	<table><tr><td>3</td><td>2</td><td>4</td></tr><tr><td>5</td><td>1</td><td>5</td></tr></table>	3	2	4	5	1	5	<table><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>5</td><td>3</td></tr></table>	2	4	2	1	5	3	<table><tr><td>1</td><td>4</td><td>5</td></tr><tr><td>5</td><td>3</td><td>2</td></tr></table>	1	4	5	5	3	2
2	4	3																										
2	3	2																										
3	2	4																										
5	1	5																										
2	4	2																										
1	5	3																										
1	4	5																										
5	3	2																										
name	<table><tr><td>A</td><td>C</td><td>C</td></tr><tr><td>B</td><td>A</td><td>C</td></tr></table>	A	C	C	B	A	C	<table><tr><td>Z</td><td>B</td><td>C</td></tr><tr><td>X</td><td>A</td><td>A</td></tr></table>	Z	B	C	X	A	A	<table><tr><td>X</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>A</td></tr></table>	X	Z	Y	B	X	A	<table><tr><td>C</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>Z</td></tr></table>	C	Z	Y	B	X	Z
A	C	C																										
B	A	C																										
Z	B	C																										
X	A	A																										
X	Z	Y																										
B	X	A																										
C	Z	Y																										
B	X	Z																										
country	<table><tr><td>UK</td><td>SP</td><td>DE</td></tr><tr><td>DE</td><td>FR</td><td>SP</td></tr></table>	UK	SP	DE	DE	FR	SP	<table><tr><td>DE</td><td>UK</td><td>NL</td></tr><tr><td>FR</td><td>NL</td><td>FR</td></tr></table>	DE	UK	NL	FR	NL	FR	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK
UK	SP	DE																										
DE	FR	SP																										
DE	UK	NL																										
FR	NL	FR																										
FR	NL	SP																										
SP	DE	UK																										
FR	NL	SP																										
SP	DE	UK																										
date	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr></table>	11/2	11/2	11/2	11/2	11/2	11/2	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/3</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/3	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/4</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/4	<table><tr><td>11/3</td><td>11/4</td><td>11/4</td></tr><tr><td>11/5</td><td>11/5</td><td>11/5</td></tr></table>	11/3	11/4	11/4	11/5	11/5	11/5
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/3	11/3	11/3																										
11/2	11/2	11/2																										
11/3	11/3	11/4																										
11/3	11/4	11/4																										
11/5	11/5	11/5																										

Snowflake stores MP metadata

- Range of values for each of the column in MP
- Number of distinct values
- Additional info which can be used for optimized and efficient query processing





Micro-Partitioning

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)																								
type	<table><tr><td>2</td><td>4</td><td>3</td></tr><tr><td>2</td><td>3</td><td>2</td></tr></table>	2	4	3	2	3	2	<table><tr><td>3</td><td>2</td><td>4</td></tr><tr><td>5</td><td>1</td><td>5</td></tr></table>	3	2	4	5	1	5	<table><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>5</td><td>3</td></tr></table>	2	4	2	1	5	3	<table><tr><td>1</td><td>4</td><td>5</td></tr><tr><td>5</td><td>3</td><td>2</td></tr></table>	1	4	5	5	3	2
2	4	3																										
2	3	2																										
3	2	4																										
5	1	5																										
2	4	2																										
1	5	3																										
1	4	5																										
5	3	2																										
name	<table><tr><td>A</td><td>C</td><td>C</td></tr><tr><td>B</td><td>A</td><td>C</td></tr></table>	A	C	C	B	A	C	<table><tr><td>Z</td><td>B</td><td>C</td></tr><tr><td>X</td><td>A</td><td>A</td></tr></table>	Z	B	C	X	A	A	<table><tr><td>X</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>A</td></tr></table>	X	Z	Y	B	X	A	<table><tr><td>C</td><td>Z</td><td>Y</td></tr><tr><td>B</td><td>X</td><td>Z</td></tr></table>	C	Z	Y	B	X	Z
A	C	C																										
B	A	C																										
Z	B	C																										
X	A	A																										
X	Z	Y																										
B	X	A																										
C	Z	Y																										
B	X	Z																										
country	<table><tr><td>UK</td><td>SP</td><td>DE</td></tr><tr><td>DE</td><td>FR</td><td>SP</td></tr></table>	UK	SP	DE	DE	FR	SP	<table><tr><td>DE</td><td>UK</td><td>NL</td></tr><tr><td>FR</td><td>NL</td><td>FR</td></tr></table>	DE	UK	NL	FR	NL	FR	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK	<table><tr><td>FR</td><td>NL</td><td>SP</td></tr><tr><td>SP</td><td>DE</td><td>UK</td></tr></table>	FR	NL	SP	SP	DE	UK
UK	SP	DE																										
DE	FR	SP																										
DE	UK	NL																										
FR	NL	FR																										
FR	NL	SP																										
SP	DE	UK																										
FR	NL	SP																										
SP	DE	UK																										
date	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr></table>	11/2	11/2	11/2	11/2	11/2	11/2	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/3</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/3	<table><tr><td>11/2</td><td>11/2</td><td>11/2</td></tr><tr><td>11/3</td><td>11/3</td><td>11/4</td></tr></table>	11/2	11/2	11/2	11/3	11/3	11/4	<table><tr><td>11/3</td><td>11/4</td><td>11/4</td></tr><tr><td>11/5</td><td>11/5</td><td>11/5</td></tr></table>	11/3	11/4	11/4	11/5	11/5	11/5
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/2	11/2	11/2																										
11/3	11/3	11/3																										
11/2	11/2	11/2																										
11/3	11/3	11/4																										
11/3	11/4	11/4																										
11/5	11/5	11/5																										

Query Pruning





Micro-Partitioning

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)
type	2 4 3 2 3 2	3 2 4 5 1 5	2 4 2 1 5 3	1 4 5 5 3 2
name	A C C B A C	Z B C X A A	X Z Y B X A	C Z Y B X Z
country	UK SP DE DE FR SP	DE UK NL FR NL FR	FR NL SP SP DE UK	FR NL SP SP DE UK
date	11/2 11/2 11/2 11/2 11/2 11/2	11/2 11/2 11/2 11/3 11/3 11/3	11/2 11/2 11/2 11/3 11/3 11/4	11/3 11/4 11/4 11/5 11/5 11/5

Query Pruning

- Skipping unnecessary scan of micro-partition is Query Pruning
- Improves the performance





Data Clustering

“Clustering is a technique by which Snowflake co-locate same value data in same micro-partitions, To improve the performance and optimize the cost”





Data Clustering

- It is automatic (Once enabled manually)
- Reduces the scanning of micro-partitions
- Improves the performance
- Storage Cost
- Compute Cost (Serverless)





Data Clustering

How to choose candidate for Clustering:

- Table with large number of Micro-partitions
- Queries need to read small amount of data (WHERE clause)
- Queries Sort the data (Order by)
- Many/most queries do select on, or sort on, the same few column(s)





Data Clustering

Snowflake maintains clustering metadata

- Total Number of Micro-Partitions
- Number of micro-partitions containing values that overlap with each other
- Depth of the overlapping partitions



Data Clustering

			Overlapping MP	Overlap Depth
		A-Z	5	5
		A-Z		
		A-Z		
		A-Z		
		A-Z		
		L-Z	3	3
		L-Z		
		L-Z		
A-E	F-K			



Data Clustering

Overlapping MP

Overlap Depth

Q-Z

L-S

A-D

E-J

K-N

3

2

A-D

E-J

K-N

O-S

T-Z

0

1



Data Clustering

SYSTEM\$CLUSTERING_DEPTH

SYSTEM\$CLUSTERING_INFORMATION (including clustering depth)





Clustering **Key**



Clustering Key

Just like Index/Key in RDBMS System, It is set of columns which are design to co-locate in micro-partitions.



Clustering Key

How to choose Clustering Keys :

- Use column most used in where clause
- Choose Join predicates
- Balance out the cardinality (Lower >> minimal pruning, Higher >> Inefficient grouping)
- For Multi Clustering Key move from low to high cardinality



Clustering Key

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

Original Micro-partitions													
		Micro-partition 1 (rows 1-6)			Micro-partition 2 (rows 7-12)			Micro-partition 3 (rows 13-18)			Micro-partition 4 (rows 19-24)		
2	type	2	4	3	3	2	4	2	4	2	1	4	5
		2	3	2	5	1	5	1	5	3	5	3	2
	name												
	country												
1	date	11/2	11/2	11/2	11/2	11/2	11/2	11/2	11/2	11/2	11/3	11/4	11/4
		11/2	11/2	11/2	11/3	11/3	11/3	11/3	11/3	11/4	11/5	11/5	11/5

```
SELECT name, country FROM t1
WHERE type = 2
AND date = '11/2';
```

Clustering Key

```
ALTER TABLE t1  
CLUSTER BY (date, type);
```

Physical Structure

Original Micro-partitions

		Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)
2	type	2 4 3 2 3 2	3 2 4 5 1 5	2 4 2 1 5 3	1 4 5 5 3 2
	name				
1	country				
	date	11/2 11/2 11/2 11/2 11/2 11/2	11/2 11/2 11/2 11/3 11/3 11/3	11/2 11/2 11/2 11/3 11/3 11/4	11/3 11/4 11/4 11/5 11/5 11/5

New Micro-partitions (After Reclustering)

		Micro-partition 5 (rows 1, 4, 6, 8, 13, 15)	Micro-partition 6 (rows 3, 5, 7, 2, 9, 14)	Micro-partition 7 (rows 10, 12, 17, 11, 16, 19)	Micro-partition 8 (rows 18, 20-24)
type		2 2 2 2 2 2	3 3 3 4 4 4	5 5 5 1 1 1	3 4 5 5 3 2
	name				
country					
	date	11/2 11/2 11/2 11/2 11/2 11/2	11/2 11/2 11/2 11/2 11/2 11/2	11/3 11/3 11/3 11/3 11/3 11/3	11/4 11/4 11/4 11/5 11/5 11/5

```
SELECT name, country FROM t1  
WHERE type = 2  
AND date = '11/2';
```





Data Storage Monitoring

Snowflake provides various methods to view and control data storage for tables, stages, and clones.

Use SnowSight, SHOW TABLES command, or the TABLE_STORAGE_METRICS

Users can also remove staged files or drop cloned objects to reduce storage

--Local Schema

```
SELECT * FROM LA_DB.INFORMATION_SCHEMA.TABLE_STORAGE_METRICS;
```

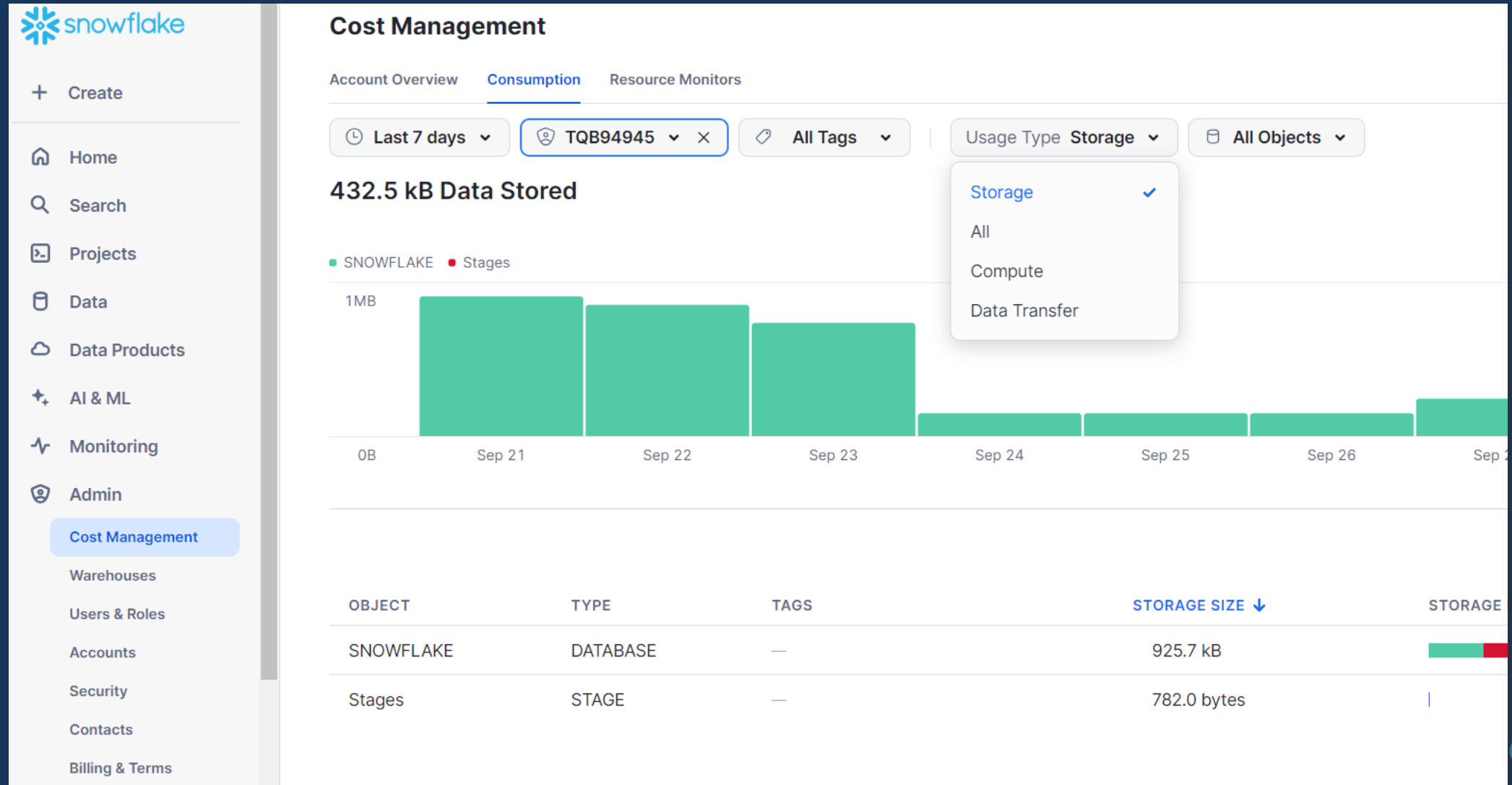
-- Across Account

```
SELECT * FROM SNOWFLAKE_SAMPLE_DATA.INFORMATION_SCHEMA.TABLE_STORAGE_METRICS;
```





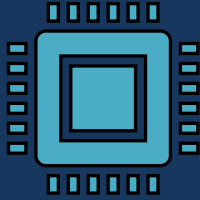
Data Storage Monitoring



Snowflake Billing



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe

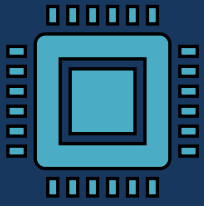


Data Transfer Cost

- Data Transfer

Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>





Compute

Snowflake Billing

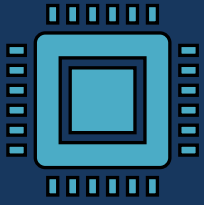
User Managed VW

- A virtual warehouse is one or more clusters of compute resources that enable **executing queries, loading data, and performing other DML operations**.
- Snowflake **credits** are used to pay for the processing time used by each virtual warehouse.

Based on Size of VW and How long they Run, not on complexity of the Query

Type of Virtual Warehouse	X-Small	Small	Medium	Large	X-Large	2X-Large	3X-Large	4X-Large	5X-Large	6X-Large
Standard	1	2	4	8	16	32	64	128	256	512
Snowpark-optimized	n/a	n/a	6	12	24	48	96	192	384	768





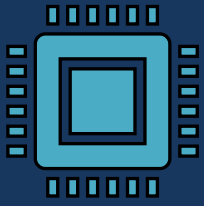
Compute

Snowflake Billing

User Managed VW

- Billed By Seconds
- 60 Sec minimum
- When Scale Up, Charged only for extra compute
- When Scale Down, Charged for both, for brief period





Compute

Snowflake Billing

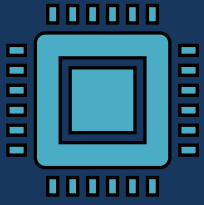
Snowflake VW / Serverless

- Compute resource provided and Managed resources
- Snowflake **credits** are used to pay for the processing time used by each virtual warehouse.

Serverless Features%

Feature	Required Compute	Additional Information
Automatic Clustering	Automated background maintenance of each clustered table, including initial clustering and reclustered as needed.	Automatic Clustering Cost
External Tables	Automated refreshing of the external table metadata with the latest set of associated files in the external stage and path.	External Tables Cost
Materialized Views	Automated background synchronization of each materialized view with changes in the base table for the view.	Materialized Views Cost
Query Acceleration Service [1]	Execution of portions of eligible queries.	Query Acceleration Service Cost
Replication	Automated copying of data between accounts, including initial replication and maintenance as needed.	Database Replication Cost Replication Group Cost
Search Optimization Service	Automated background maintenance of the search access paths used by the search optimization service.	Search Optimization Service Cost
Snowpipe	Automated processing of file loading requests for each pipe object.	Snowpipe Cost
Tasks	Execution of SQL code.	Tasks Cost





Compute

Snowflake Billing

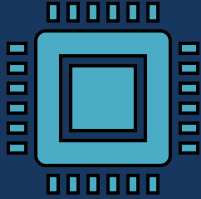
Cloud Services

- Authentication, enforces security, performs query compilation and optimization, handles request query caching etc.
- Snowflake **credits** are used to pay for the processing time used by each virtual warehouse.
- If usage is less than 10% of daily credit consumed, it is free

Date	Compute Credits Used (Warehouses only)	Cloud Services Credits Used	Credit Adjustment for Cloud Services (Lesser of 10% of Compute or Cloud Services)	Credits Billed (Sum of Compute, Cloud Services, and Adjustment)
Nov 1	100	20	-10	110
Nov 2	120	10	-10	120
Nov 3	80	5	-5	80
Nov 4	100	13	-10	103
Total	400	48	-35	413



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe



Data Transfer Cost

- Data Transfer

Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>





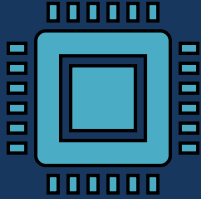
Storage

Snowflake Billing

- Staged File Costs
- Database Costs
- Time Travel and Fail-safe Costs
- Temporary and Transient Tables Costs
- Cloning Tables, Schemas, and Databases Costs



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe



Data Transfer Cost

- Ingress is free
- Egress is charged

Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>





Snowflake Billing

Data Transfer Cost

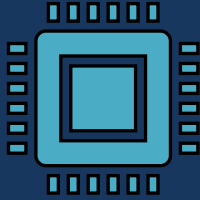
Data transfer is the process of moving data into (ingress) and out of (egress) Snowflake.

- **Unloading** - No charges for data ingress/loading but cost if you unload
- **Replication** - Data transfer in same region and same cloud provider is free
Cost when region or cloud provider is different
- **Writing External Function** - When data gets transferred out for function
- Per-Byte fee for Egress

Snowflake does **not apply data egress charges when a Snowflake client or driver retrieves query results across regions within the same cloud platform or across different cloud platforms.*



Snowflake Billing



Compute

- User Managed VW
- Snowflake Managed VW
- Cloud Services



Storage

- Data Staging
- Database Tables
- Materialized View
- Time Travel and Fail Safe



Data Transfer Cost

- Data Transfer

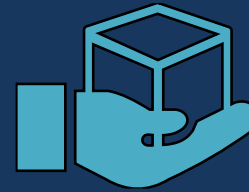
Billing depends on Snowflake Edition, Cloud provider and Region
Check <https://www.snowflake.com/pricing/>



Snowflake Billing



On Demand



Capacity



Snowflake Billing



On Demand



Capacity

Features	On Demand	Capacity
Billing	Pay-as-you-go	Reserve in advance
Rates	Prevailing prices	Discounted prices



A light blue icon of a globe with a network of lines and nodes.

Network Policies



Network Policies

- Allows or restrict access to SF based on IP address
 - Allowed IP List
 - Blocked IP List
- SECURITYADMIN or Higher (eg ACCOUNTADMIN) can create network policies
- Can be set at ACCOUNT or USER level
- Only one Policy per ACCOUNT or per USER
- Supports CIDR Notation
192.168.1.0/24 >> 192.168.1.0 - 192.168.1.255
- Blocked IP gets preference
- Support only Internet Protocol version 4 (i.e. IPv4) addresses, not IPv6



Network Policies



+ Create

Home

Search

Projects

Data

Data Products

AI & ML

Monitoring

Admin

- Cost Management
- Warehouses
- Users & Roles
- Accounts
- Security
- Contacts
- Billing & Terms

Security

Network PoliciesNetwork Rules

Filter by Name

+

Network Policy

Create Network Policy

Creating as ACCOUNTADMIN

Network policy name

test

Add an optional comment...

AllowedBlocked

New rule >

Select network rules that are allowed access to Snowflake

CancelCreate Network Policy





Network Policies

```
-- Create a new policy

CREATE NETWORK POLICY mypolicy1 ALLOWED_IP_LIST=('192.168.1.0/24')
                                BLOCKED_IP_LIST=('192.168.1.99');

-- Display available Policies

SHOW NETWORK POLICIES;

-- Set policy for the network

ALTER ACCOUNT SET NETWORK_POLICY = 'SAMPLE';

-- Unset the policy

ALTER ACCOUNT UNSET NETWORK_POLICY;

-- See details of the policy

DESC NETWORK POLICY mypolicy1;
```





Multi Factor **Authentication** (MFA)





Multi Factor **Authentication** (MFA)



Password



Device



Finger Print



Location





Multi Factor Authentication (MFA)

- Powered By Duo Security Service
- Managed by Snowflake
- Not enabled by default
- Recommended to have MFA for all ACCOUNTADMIN role
- Ideally for Snowsight but fully supported for Snowsql, JDBC/ODBC drivers

Notifications

☒ Email notifications from [Snowflake](#) [Learn more](#) [@Snowflake](#) [Anywhere](#) [i](#)

☐ When queries finish in the background

Multi-factor authentication

Each time you sign in to Snowflake, you'll use your password and a verification code. [Learn more](#)

Enroll





Multi Factor Authentication (MFA)

©Learn Anywhere



Sign in to Snowflake

Username

Password

[Forgot password](#)

Sign in



©Learn Anywhere



Settings

Send Me a Push

Call Me

Enter a Passcode



Federated Authentication (SSO)



Federated Authentication (SSO)

In a federated environment, user authentication is separated from user access through the use of one or more external entities that provide independent authentication of user credentials.



Federated Authentication (SSO)

Service Provider :

In a Snowflake federated environment, Snowflake serves as the SP.

Identity provider (IdP):

Creating and maintaining user credentials and other profile information.

Authenticating users for SSO access to the SP.

Snowflake supports most SAML 2.0-compliant vendors as an IdP

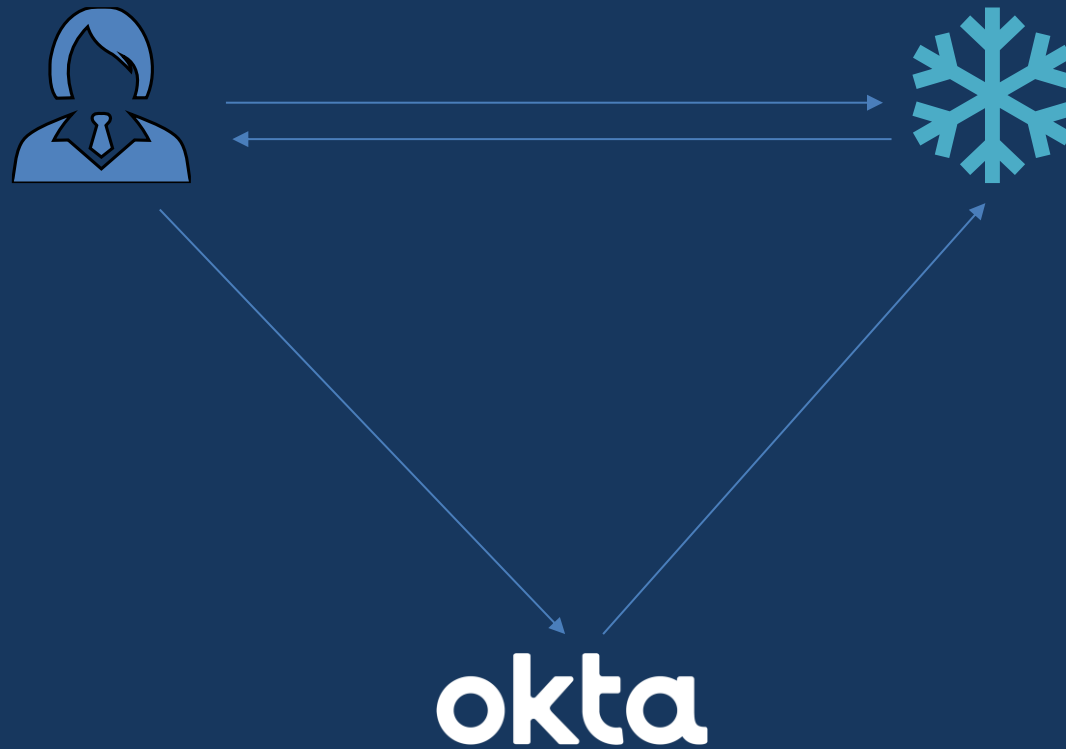


okta



Federated Authentication (SSO)

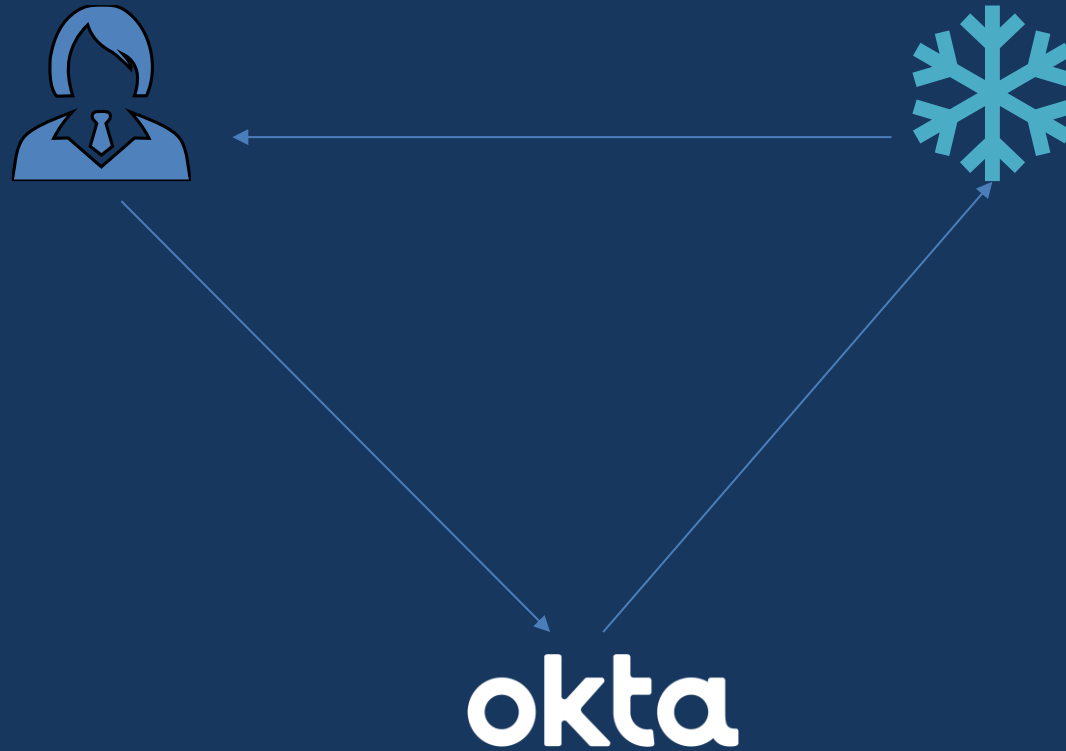
Login Workflow : SF Initiated





Federated Authentication (SSO)

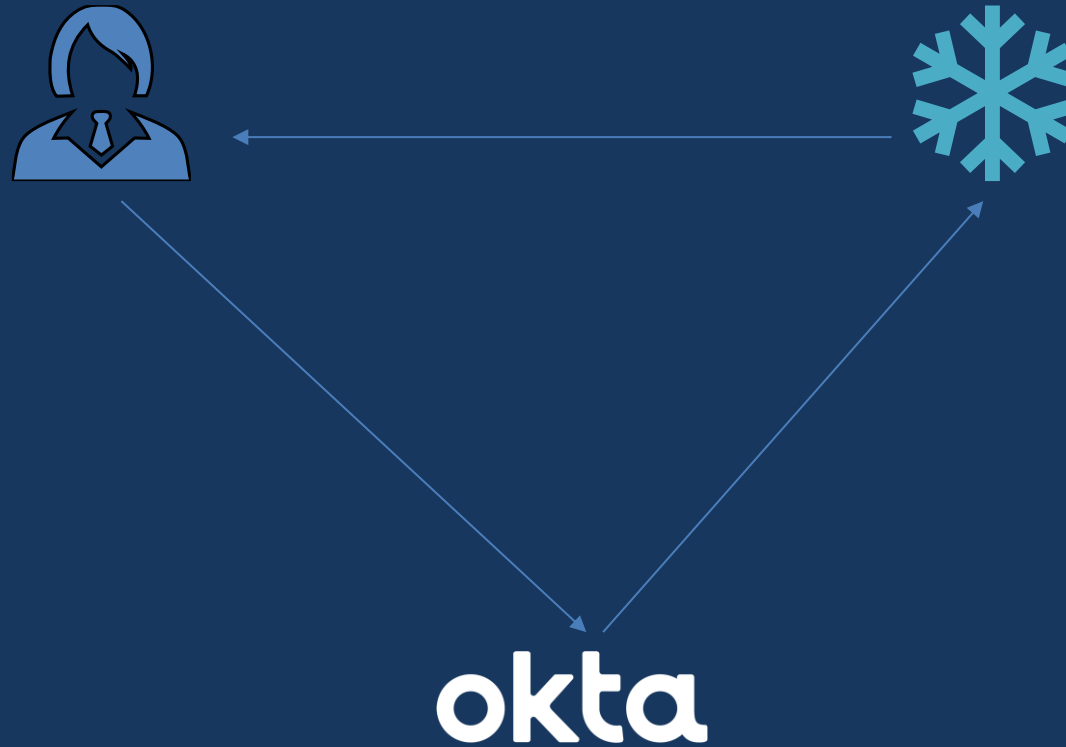
Login Workflow : IdP Initiated





Federated Authentication (SSO)

Logout Workflow : SF Initiated





Key-Pair Authentication





Key-Pair Authentication

Snowflake supports using key pair authentication for enhanced authentication security as an alternative to basic authentication.

This authentication method requires, as a minimum, a 2048-bit RSA key pair.





Key-Pair Authentication

Step 1: Generate the Private Key (OpenSSL)

Step 2: Generate a Public Key (OpenSSL)

Step 3: Store the Private and Public Keys Securely

Step 4: Assign the Public Key to a Snowflake User

Step 5: Verify the User's Public Key Fingerprint

Step 6: Configure the Snowflake Client to Use Key Pair Authentication





Access Control Framework





Access Control Framework

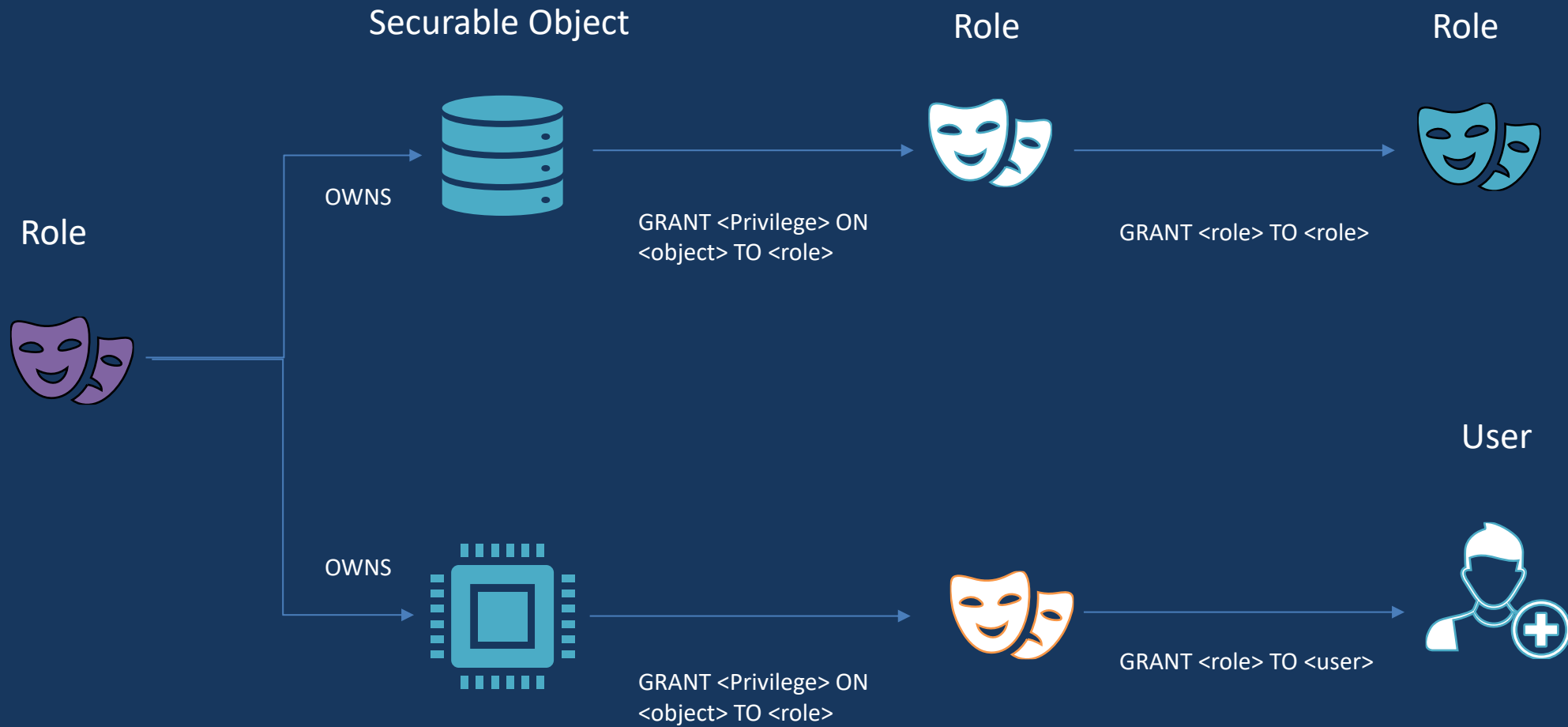
Discretionary Access Control (DAC): Each object has an owner, who can in turn grant access to that object.

Role-based Access Control (RBAC): Access privileges are assigned to roles, which are in turn assigned to users.





Access Control Framework





Access Control Framework

- **Securable object:** An entity to which access can be granted. Unless allowed by a grant, access is denied.
- **Role:** An entity to which privileges can be granted. Roles are in turn assigned to users. Note that roles can also be assigned to other roles, creating a role hierarchy.
- **Privilege:** A defined level of access to an object. Multiple distinct privileges may be used to control the granularity of access granted.
- **User:** A user identity recognized by Snowflake, whether associated with a person or program.





Types of Role

Account roles

To permit SQL actions on any object in your account, grant privileges on the object to an account role.

Database Roles

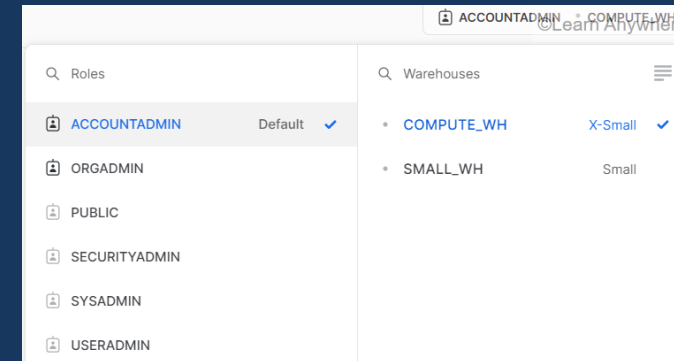
To limit SQL actions to a single database, as well as any object in the database, grant privileges on the object to a database role in the same database.



Types of Active Role

Primary Role

The default assumed Role
USE ROLE <ROLE NAME>



Secondary Role

Can assign any number of secondary role.

```
15  USE SECONDARY ROLES SYSADMIN;
16  SELECT CURRENT_SECONDARY_ROLES();
17
```

©Learn Anywhere

Results Chart

	CURRENT_SECONDARY_ROLES()
1	{"roles":"SYSADMIN","value":"SYSADMIN"}



System Defined Roles





System Defined Roles

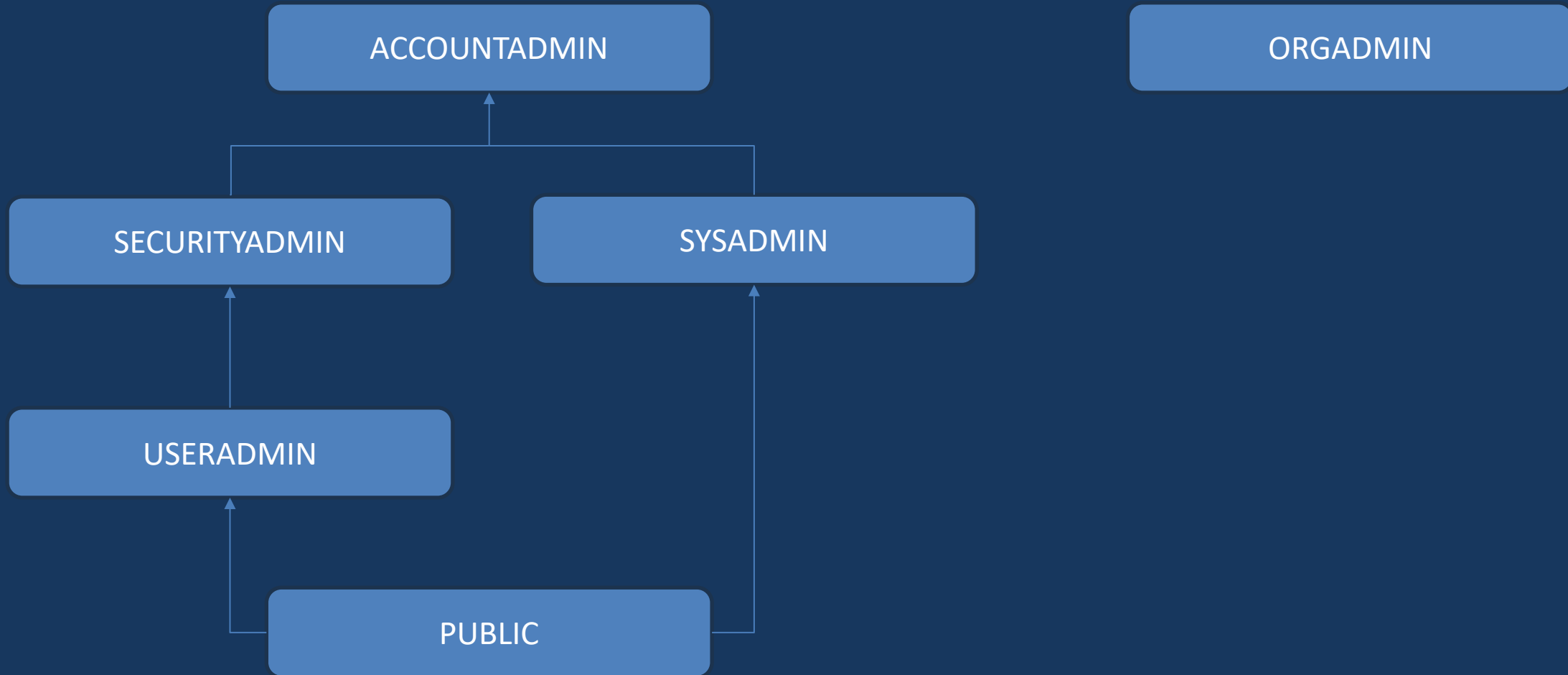
ORGADMIN

- Role that manages ops at organization level
- Can create account in the organization
- Can view all account in the organization and all enabled region
SHOW ORGANIZATION ACCOUNTS, SHOW REGIONS
- Can check usage information across the organization



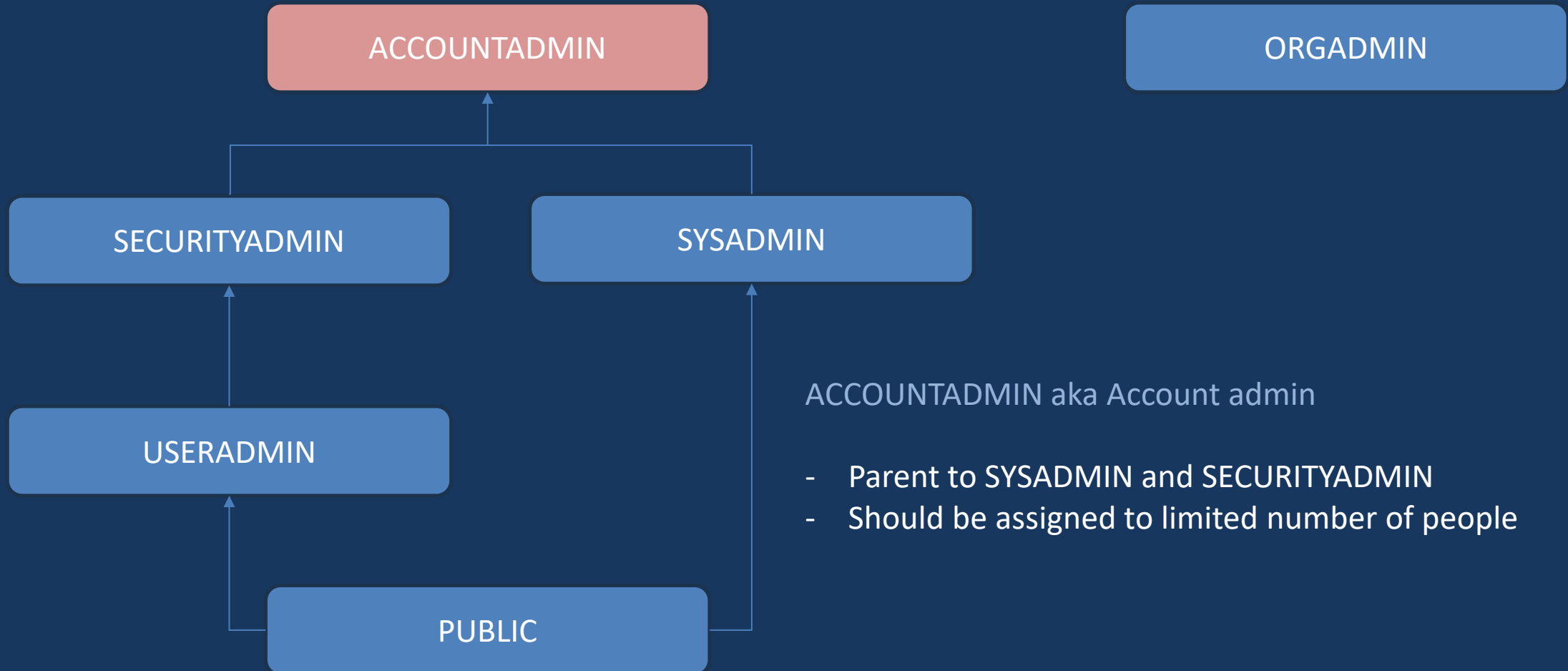


System Defined Roles





System Defined Roles



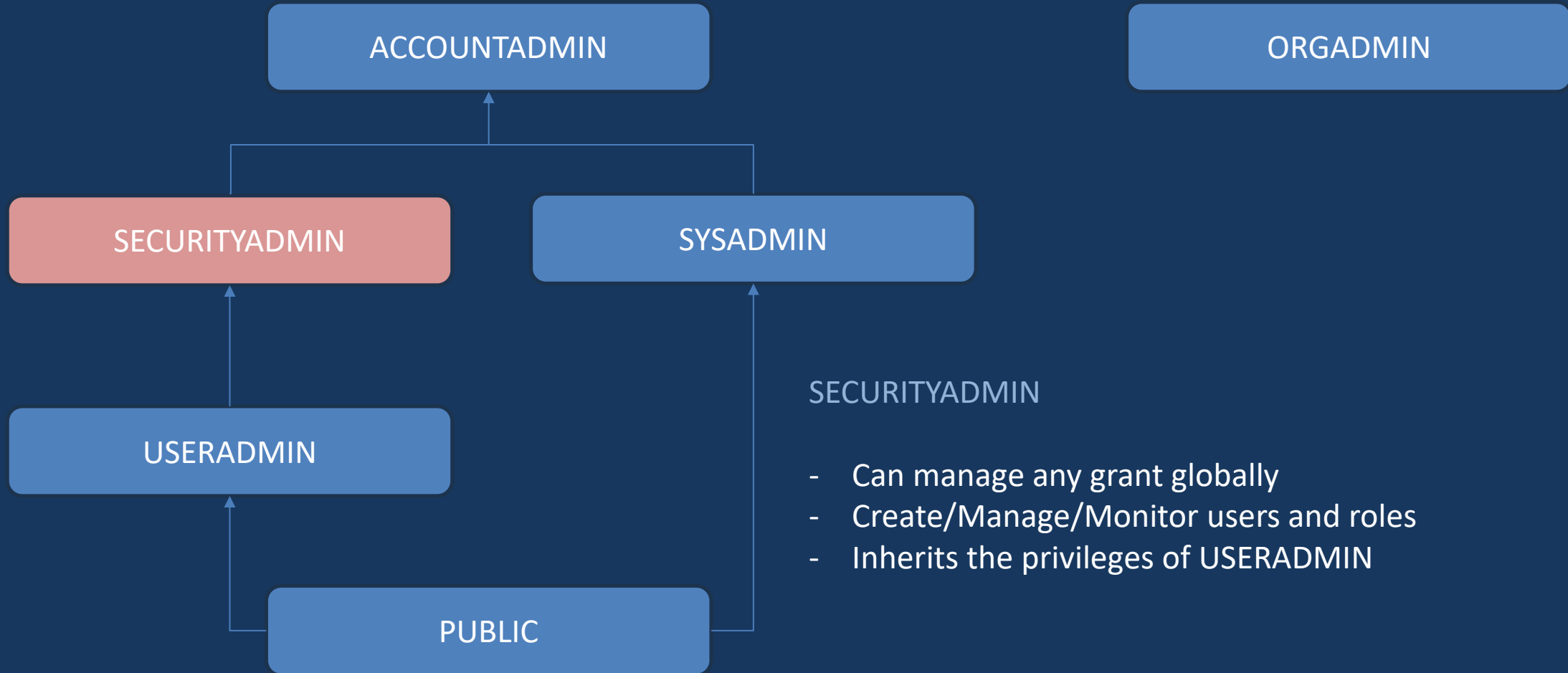
ACCOUNTADMIN aka Account admin

- Parent to SYSADMIN and SECURITYADMIN
- Should be assigned to limited number of people



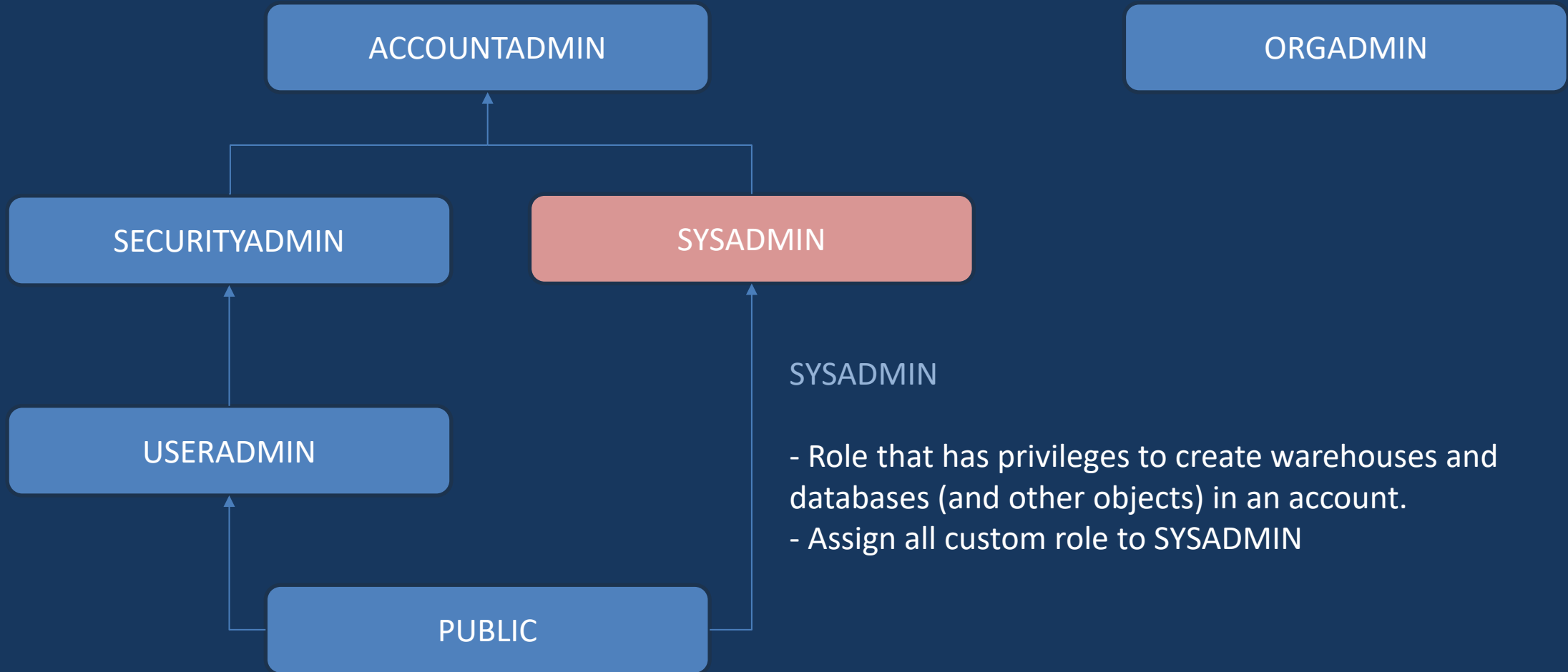


System Defined Roles





System Defined Roles



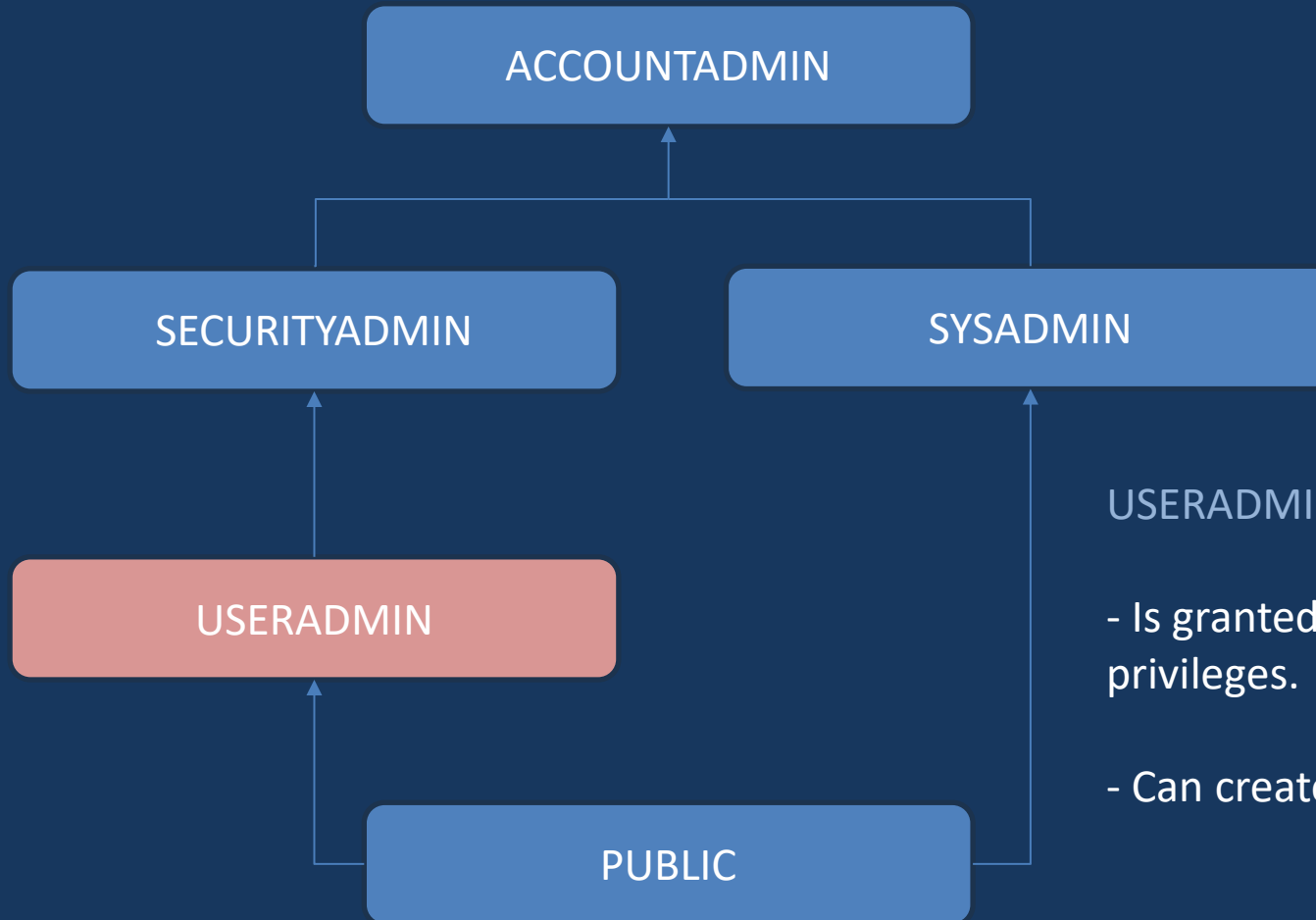
SYSADMIN

- Role that has privileges to create warehouses and databases (and other objects) in an account.
- Assign all custom role to SYSADMIN





System Defined Roles



ORGADMIN

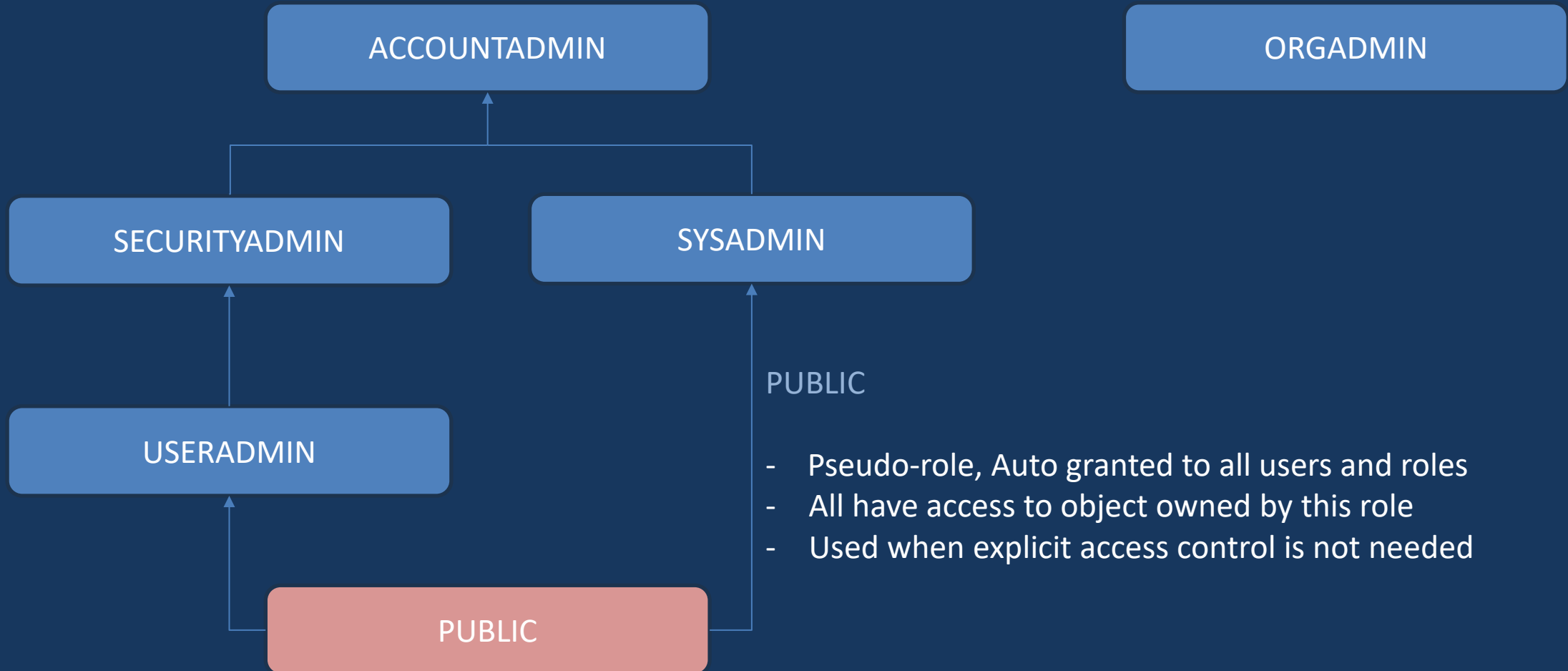
USERADMIN

- Is granted the CREATE USER and CREATE ROLE security privileges.
- Can create users and roles in the account.



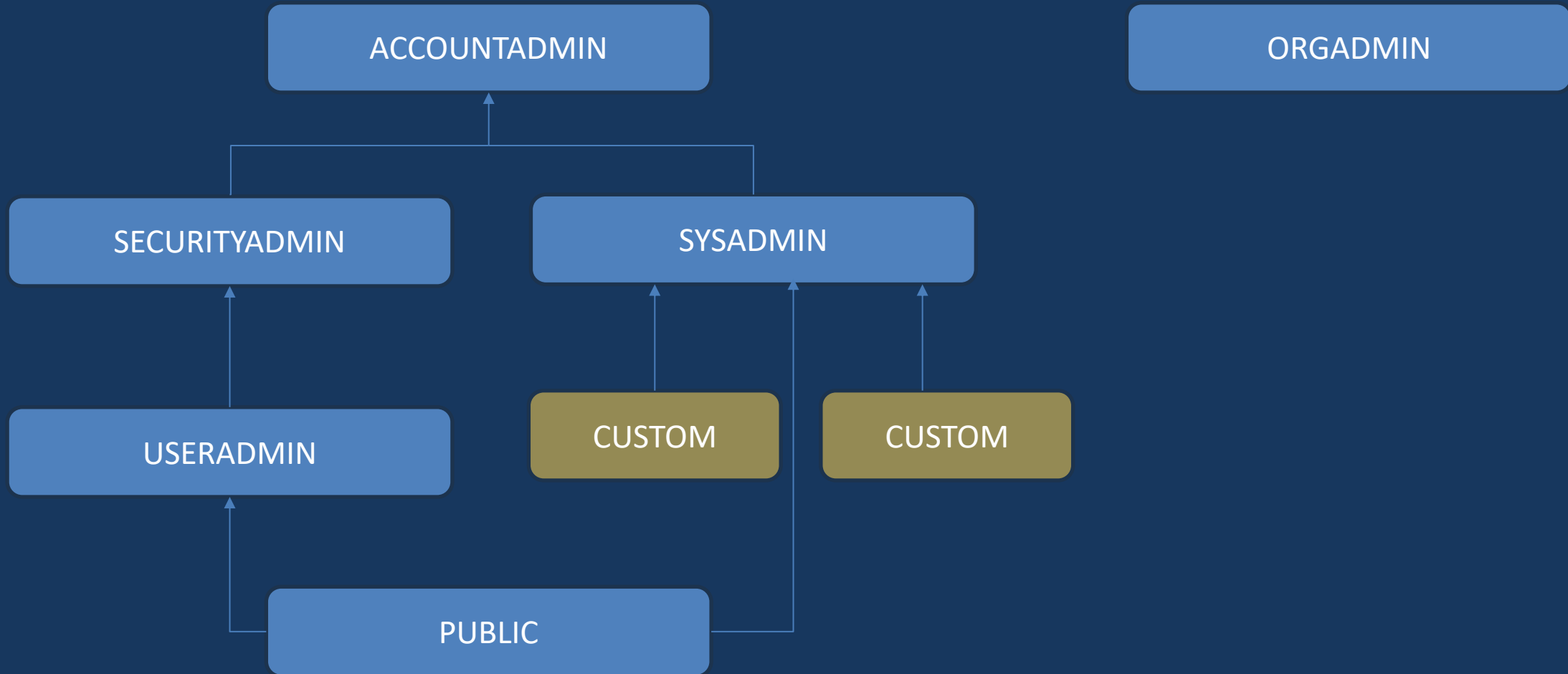


System Defined Roles



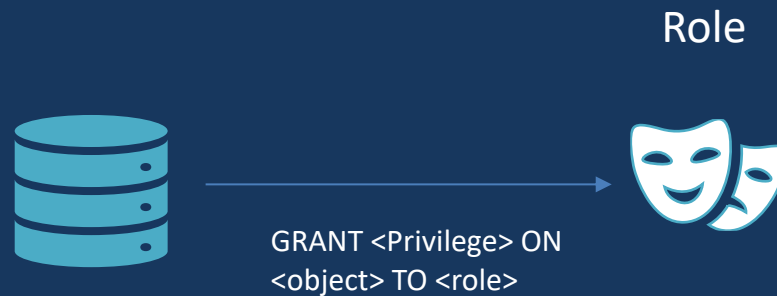


System Defined Roles



Access Control Privilege

Privileges are granted to roles, and roles are granted to users, to specify the operations that the users can perform on objects in the system.





Access Control Privilege

Table Privilege:

SELECT
TRUNCATE
DELETE
OWNERSHIP
...

GRANT SELECT ON TABLE <Table> TO ROLE <Role>;

More details : <https://docs.snowflake.com/en/user-guide/security-access-control-privileges>



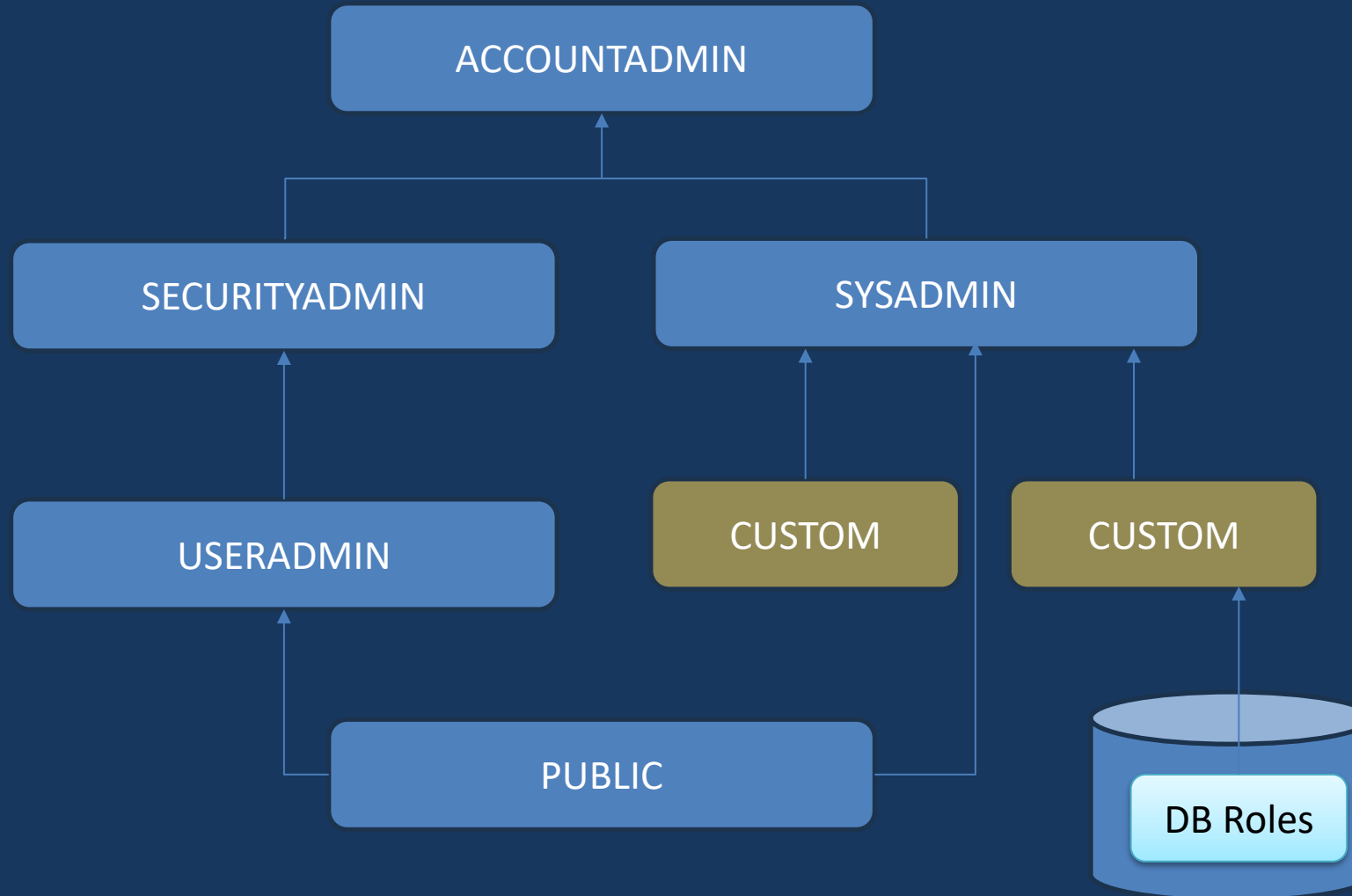


Role Hierarchy & Privilege Inheritance





Role Hierarchy & Privilege Inheritance





Role Hierarchy & Privilege Inheritance

Securable Object

Role



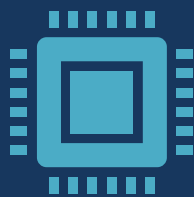
GRANT <Privilege> ON
<object> TO <role>



GRANT <role> TO <role>



GRANT <role> TO <user>



GRANT <Privilege> ON
<object> TO <role>

GRANT <Privilege> ON
<object> TO <role>



GRANT <role> TO <role>

REVOKE SELECT ON ALL TABLES IN SCHEMA <schema> from ROLE <role>;





Organization & Accounts





Organization & Accounts

An *organization* is :

- First-class Snowflake object that links the accounts owned by the org
- Organizations simplify
 - Account management and billing
 - Replication and Failover/Failback
 - Snowflake Secure Data Sharing and administration tasks.
- Use ORGADMIN Role

ORGADMIN has access to Account properties but not to account data

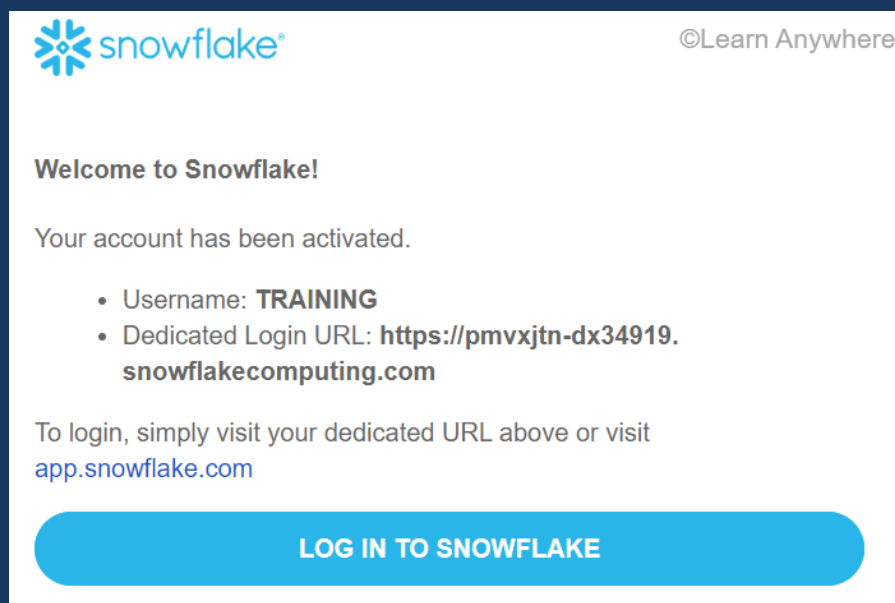




Organization & Accounts

- Account identifier uniquely identify the SF account

`<account_identifier>.snowflakecomputing.com`



```
SELECT CURRENT_ORGANIZATION_NAME();  
SELECT CURRENT_ACCOUNT_NAME();
```





Organization & Accounts

LA learnr any

Switch Role

ACCOUNTADMIN >

Account

aws TQB94945 >

My profile

Support

Appearance >

Client download >

Documentation >

Privacy notice >

Sign Out

VFMONYE

TQB94945 ✓

LEARN - aws US West (Oregon)

→ Sign Into Another Account

TQB94945

Organization VFMONYE

Cloud aws Amazon Web Services

Region US West (Oregon)

Edition Business Critical

Locator SVB20249



Information Schema



Information Schema

The Snowflake Information Schema (aka “Data Dictionary”) consists of a set of system-defined views and table functions that provide extensive metadata information about the objects created in your account.

The Snowflake Information Schema is based on the SQL-92 ANSI Information Schema, but with the addition of views and functions that are specific to Snowflake.



Information Schema

- Read Only Schema
- Available for every DB
- Maintains data about all DB object and some account level information, In form of Views
- Table function for historical and usage data across your account
- Need Privilege



Account Usage



Account Usage

- Views that display object metadata and usage metrics for your account.
- Account_Usage and Readers_Account_Usage
- Very similar to Information Schema with some differences
- Access can be granted by ACCOUNTADMIN role



Account Usage Vs Information Schema

Parameters	Account Usage	Information Schema
Includes dropped object	Yes	No
Latency of data	45 Mins – 3 Hours	Instant
Retention Period	1 Year	7 Days to 6 Months





Access History



Access History

- Needs Enterprise Edition or Higher
- Maintains history of read and write operations
- Used for compliance auditing
- Stores data on source column, object accessed directly and indirectly

*QUERY_ID | QUERY_START_TIME | USER_NAME | DIRECT_OBJECTS_ACCESSED |
BASE_OBJECTS_ACCESSED | OBJECTS_MODIFIED | OBJECT_MODIFIED_BY_DDL |
POLICIES_REFERENCED*





Column **Level** Security





Column Level Security

Dynamic Data Masking (Needs Enterprise Edition or Higher)

- Use masking policies to selectively mask column data

External Tokenization

- Tokenization is the process of removing sensitive data by replacing it with an undecipherable token



Column Level Security

Dynamic Data Masking

- Use masking policies to selectively mask column data

	ID	NAME	EMAIL	CREDIT_CARD_NUMBER	©Learn Anywhere
1	1	John Doe	john.doe@example.com	1234-5678-9012-3456	
2	2	Alex Doe	Alex.doe@example.com	4567-8901-2345-6789	

	...	ID	NAME	EMAIL	CREDIT_CARD_NUMBER	©Learn Anywhere
1		1	John Doe	john.doe@example.com	*****	
2		2	Alex Doe	Alex.doe@example.com	*****	





Column Level Security

```
CREATE OR REPLACE MASKING POLICY cc_mask AS (val string) returns string ->
CASE
  WHEN current_role() IN ('ACCOUNTADMIN') THEN VAL
  ELSE '*****'
END;

-- Assign the policy to CC column in the table

ALTER TABLE CUST MODIFY COLUMN credit_card_number set masking policy cc_mask;
```

```
-- You can check policy assigned to which objects

SELECT * from table(information_schema.policy_references(policy_name=>'cc_mask'));
```

```
-- ALTER command is different the create command

USE ROLE ACCOUNTADMIN;

ALTER MASKING POLICY cc_mask SET BODY ->
CASE
  WHEN current_role() IN ('DUMMY_ROLE2') THEN VAL
  --ELSE NULL
  ELSE sha2(val)
END;
```



*(Needs Enterprise
Edition or Higher)*



Row Level Security





Row Level Security



Sales



Marketing



IT

ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com		Sales	
2	Jane Doe	jane.doe@example.com		Marketing	
3	Peter Smith	peter.smith@example.com		IT	





Row Level Security



ID	NAME	EMAIL	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com	Sales	

ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com		Sales	
2	Jane Doe	jane.doe@example.com		Marketing	
3	Peter Smith	peter.smith@example.com		IT	





Row Level Security



ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
2	Jane Doe	jane.doe@example.com		Marketing	

ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com		Sales	
2	Jane Doe	jane.doe@example.com		Marketing	
3	Peter Smith	peter.smith@example.com		IT	





Row Level Security



ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
2	Jane Doe	jane.doe@example.com		Marketing	

ID	NAME	EMAIL	...	DEPARTMENT	©Learn Anywhere
1	John Doe	john.doe@example.com		Sales	
2	Jane Doe	jane.doe@example.com		Marketing	
3	Peter Smith	peter.smith@example.com		IT	





Row Level Security

```
-- Masking policy

create or replace row access policy dept_policy as (department varchar) returns boolean ->
  'ACCOUNTADMIN' = current_role()
  or exists (
    select 1 from mapping_tab
    where curr_role = current_role()
    and dept = department
  )
;
```

CURR_ROLE	DEPT
DUMMY_SALES	Sales
DUMMY_MARK	Marketing
DUMMY_IT	IT

```
-- Assign the Policy to the table column

ALTER TABLE CUST_ROW ADD ROW ACCESS POLICY dept_policy ON (department);
```





Object Tags

Object tags in Snowflake are schema-level objects that allow you to attach metadata to other Snowflake objects, such as tables, views, and columns. Needs Enterprise Edition or Higher.





Object Tags

Purpose of Tags: Tags help in monitoring sensitive data for compliance, discovery, protection, and resource usage. They can be used in both centralized and decentralized data governance approaches.

Tag Creation and Assignment: Tags are schema-level objects assigned to other Snowflake objects with a key-value pair.

Tag Usage and Monitoring: Tags can be queried to monitor usage, facilitate data governance operations, and ensure compliance.

Tag Limits and Management: There are limits on the number of unique tags per object (50). Tags can be inherited based on the Snowflake securable object hierarchy, and tag assignments can be replicated within databases.





Object Tags

```
-- Create tag with allowed values
```

```
CREATE TAG IF NOT EXISTS COST ALLOWED_VALUES 'MKT', 'ADVT';  
CREATE TAG IF NOT EXISTS HR ALLOWED_VALUES 'ADMIN', 'TA';
```

```
-- Check list of tags
```

```
SELECT SYSTEM$GET_TAG_ALLOWED_VALUES ('COST');
```

```
-- Tag objects while creating or use ALTER
```

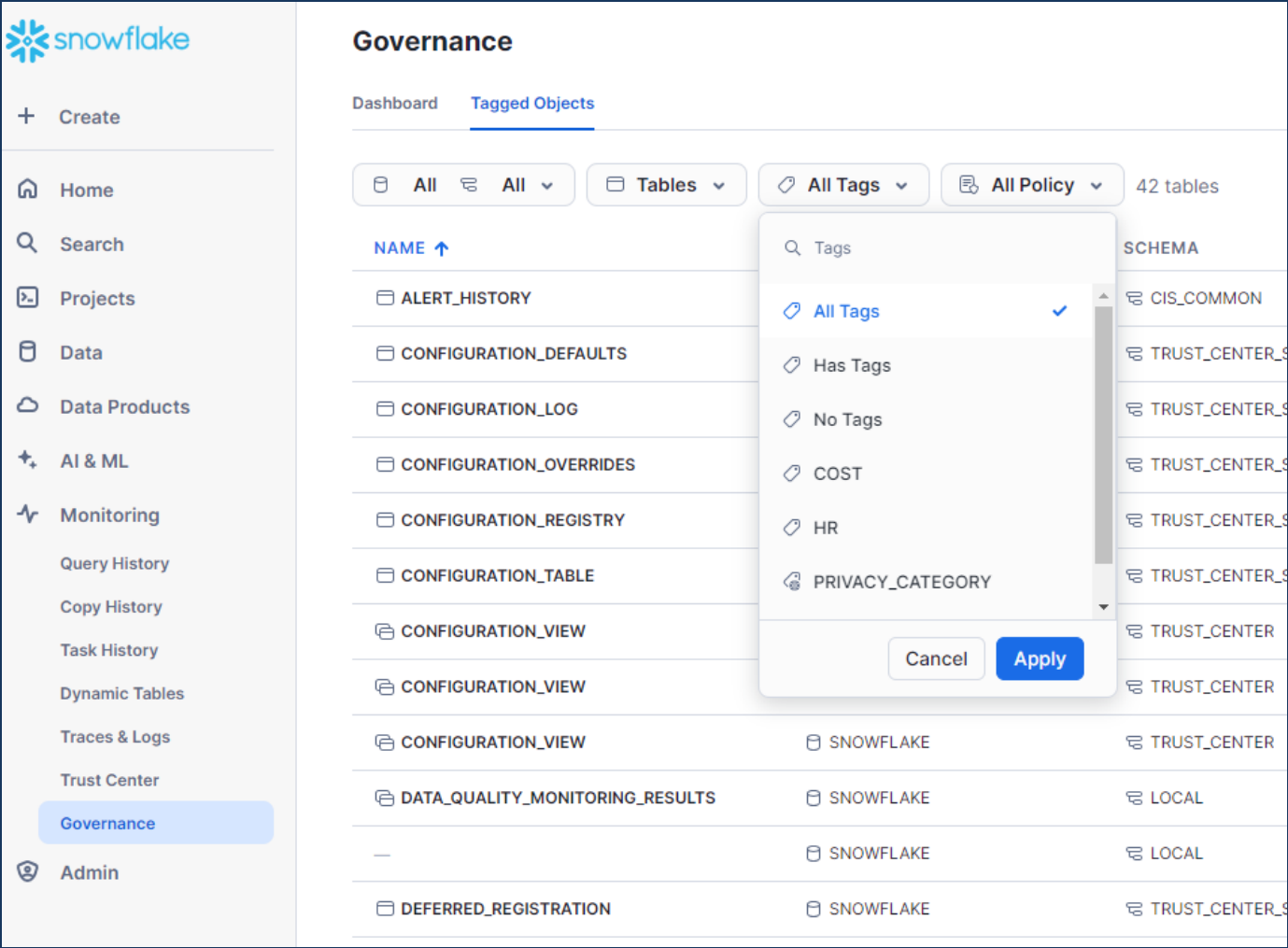
```
ALTER WAREHOUSE COMPUTE_WH SET TAG COST = 'MKT', HR = 'TA';
```

```
-- Query TAG with Lineage, all object with lineage
```

```
SELECT * FROM TABLE(SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES_WITH_LINEAGE('COST'));
```



Object Tags



Governance

Dashboard **Tagged Objects**

All All Tables All Tags All Policy 42 tables

NAME ↑

- ALERT_HISTORY
- CONFIGURATION_DEFAULTS
- CONFIGURATION_LOG
- CONFIGURATION_OVERRIDES
- CONFIGURATION_REGISTRY
- CONFIGURATION_TABLE
- CONFIGURATION_VIEW
- CONFIGURATION_VIEW
- CONFIGURATION_VIEW
- DATA_QUALITY_MONITORING_RESULTS
-
- DEFERRED_REGISTRATION

SCHEMA

- CIS_COMMON
- TRUST_CENTER_S
- TRUST_CENTER_S
- TRUST_CENTER_S
- TRUST_CENTER_S
- TRUST_CENTER_S
- TRUST_CENTER
- TRUST_CENTER
- TRUST_CENTER
- LOCAL
- LOCAL
- TRUST_CENTER_S

Tags

- All Tags ✓
- Has Tags
- No Tags
- COST
- HR
- PRIVACY_CATEGORY

Cancel Apply

Latency up to 2 hours





Object Tags

The screenshot displays the Snowflake web interface. On the left is a navigation sidebar with options like Home, Search, Projects, Data, Databases (selected), Add Data, Data Products, AI & ML, Monitoring, and Admin. The main area shows a tree view of databases and schemas, with 'LA_DB / LA_SCHEMA / LA_TABLE' selected. A modal titled 'Edit Table' is open, showing the table name 'LA_TABLE' and a dropdown for 'Tags'. The dropdown menu is open, listing tags: COST, HR, PRIVACY_CATEGORY, and SEMANTIC_CATEGORY. A 'Save' button is visible at the bottom right of the modal. The background interface includes a search bar, a 'Load Data' button, and a 'Group by Role' dropdown.

Edit Table
LA_TABLE as ACCOUNTADMIN

Name

Comment (optional)

Tags

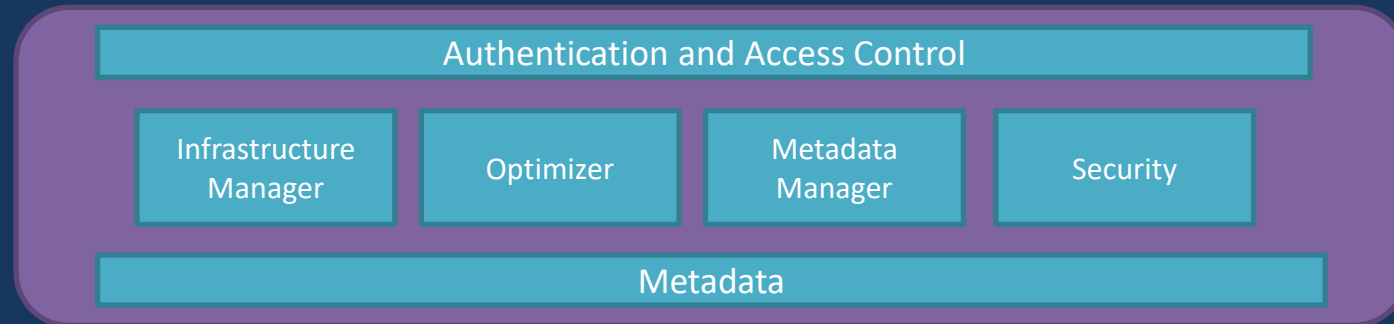
- Tags
- COST
- HR
- PRIVACY_CATEGORY
- SEMANTIC_CATEGORY

Select a tag to see values

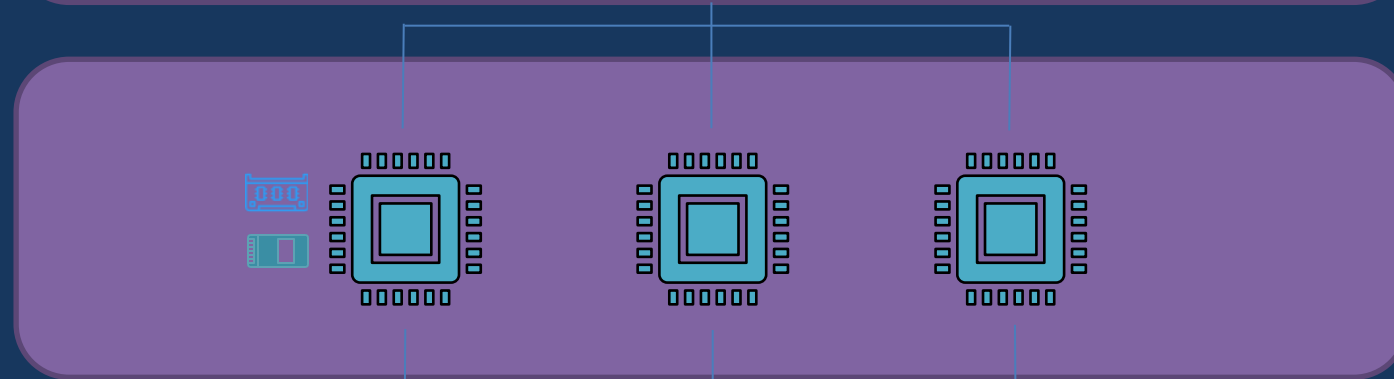


Snowflake Architecture

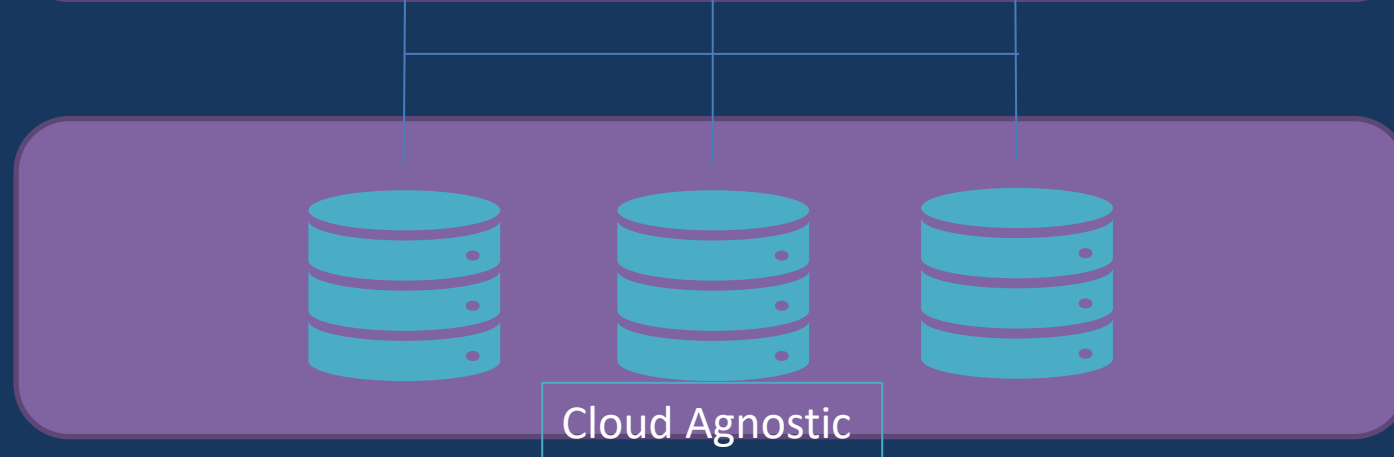
Cloud Services



Query Processing Layer



Database Storage



Query Result Cache

Metadata Cache

Warehouse Cache





Caching

Query Result Cache (USE_CACHED_RESULT)

- In Cloud services layer
- Expires in 24 hours
- Work for exact same query
- Doesn't work with functions, which evaluates at runtime eg CURRENT_TIMESTAMP() and UUID_STRING(), CURRENT_DATE() is exception
- Doesn't work with external functions
- Doesn't work when , table change, config change, Micro-partitions change

```
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
```





Caching

Query Result Cache (USE_CACHED_RESULT)

- Query result cache > 100KB uses security token
- Security token expires in 6 hours (except for spark connector)
- Doesn't work for Hybrid tables
- 24 hours extension happens for 31 days, post which result is purged
- Repeating, Query should be exact, word to word, can't change case also





Caching

Metadata Cache

- In Cloud services layer
- Stores Metadata Information
- Warehouse not needed
- Row Count, MIN/MAX values, DISTINCT count etc.

```
SELECT DISTINCT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.CUSTOMER_ADDRESS;
```





Caching

Warehouse Cache (Local Disk Cache)

- At Query Processing Layer
- Local to the WH
- Purged when WH is suspended or shut
- Need to find whether it is worth to auto-suspend a WH

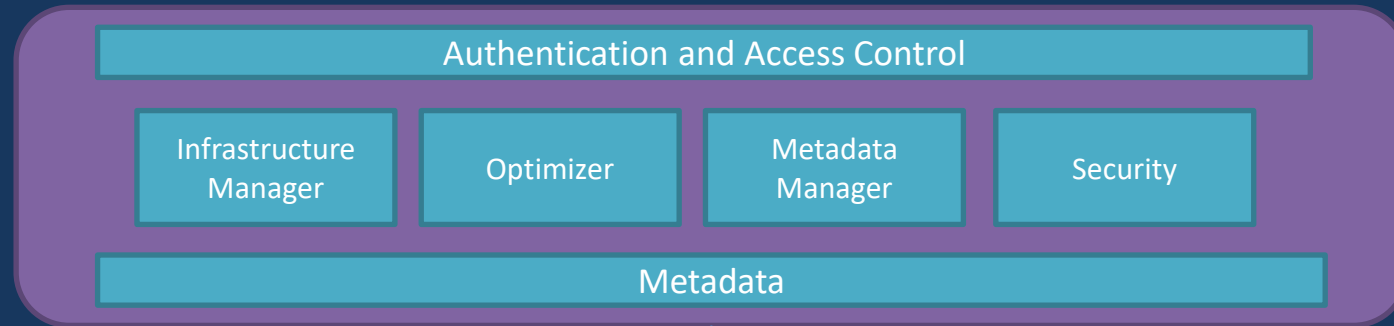
```
ALTER WAREHOUSE COMPUTE_WH SUSPEND ;
```

```
ALTER WAREHOUSE COMPUTE_WH RESUME;
```

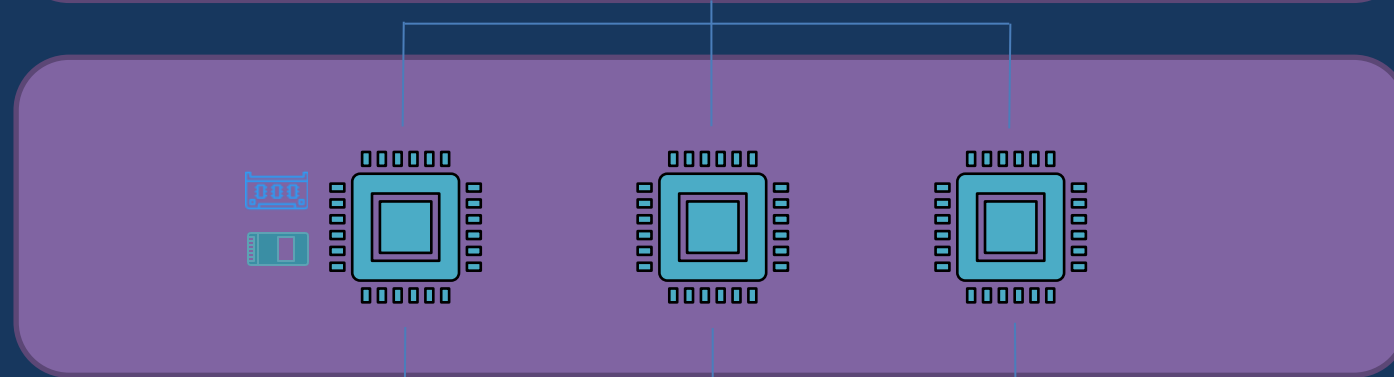


Snowflake Architecture

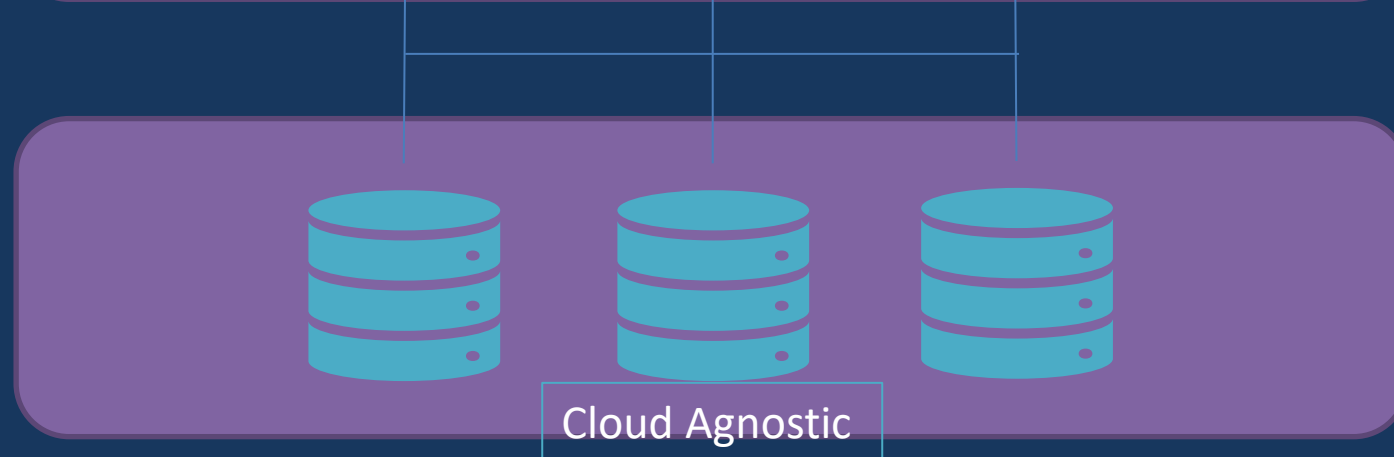
Cloud Services



Query Processing Layer



Database Storage



Query Result Cache

Metadata Cache

Warehouse Cache





Micro-Partitioning (R)

Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure

	Micro-partition 1 (rows 1-6)	Micro-partition 2 (rows 7-12)	Micro-partition 3 (rows 13-18)	Micro-partition 4 (rows 19-24)
type	2 4 3 2 3 2	3 2 4 5 1 5	2 4 2 1 5 3	1 4 5 5 3 2
name	A C C B A C	Z B C X A A	X Z Y B X A	C Z Y B X Z
country	UK SP DE DE FR SP	DE UK NL FR NL FR	FR NL SP SP DE UK	FR NL SP SP DE UK
date	11/2 11/2 11/2 11/2 11/2 11/2	11/2 11/2 11/2 11/3 11/3 11/3	11/2 11/2 11/2 11/3 11/3 11/4	11/3 11/4 11/4 11/5 11/5 11/5

Query Pruning

- Skipping unnecessary scan of micro-partition is Query Pruning
- Improves the performance





Query Profile

```
6
7 | SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.CUSTOMER limit 100;
8
9
10 -- Dummy Query
```

Results Chart

	C_CUSTOMER_SK	C_CUSTOMER_ID	C_CURRENT_CDEMO_SK	C_CURRENT_HDEMO_SK	C_CURRENT_ADDR_SK	C_FIRST_SHIPTO_DATE_SK	C_FIRST_SALES_DATE_SK
1	28437730	AAAAAAAACOMBLBA	854203	2961	28176519	2451242	2451212
2	28437731	AAAAAAAADOMBLBA	1107909	3831	28930953	2449681	2449651
3	28437732	AAAAAAAAEOMBLBA	1465719	6324	16092176	2449602	2449572
4	28437733	AAAAAAAAFOMBLBA	386019	3750	39012328	2450153	2450123
5	28437734	AAAAAAAAGOMBLBA	1091543	6927	36141064	2449815	2449785
6	28437735	AAAAAAAHHOMBLBA	1748087	5737	2761787	2450264	2450234
7	28437736	AAAAAAAHHOMBLBA	1867458	2164	40560670	2450601	2450661

Query Details

- Query duration
- Rows
- Query ID: 01b7a299-0003-da9e-0...
- End time: Oct 12, 12:19 PM
- Scan: 15.7MB (100%)
- Role: ACCOUNTADMIN
- Warehouse: COMPUTE_WH

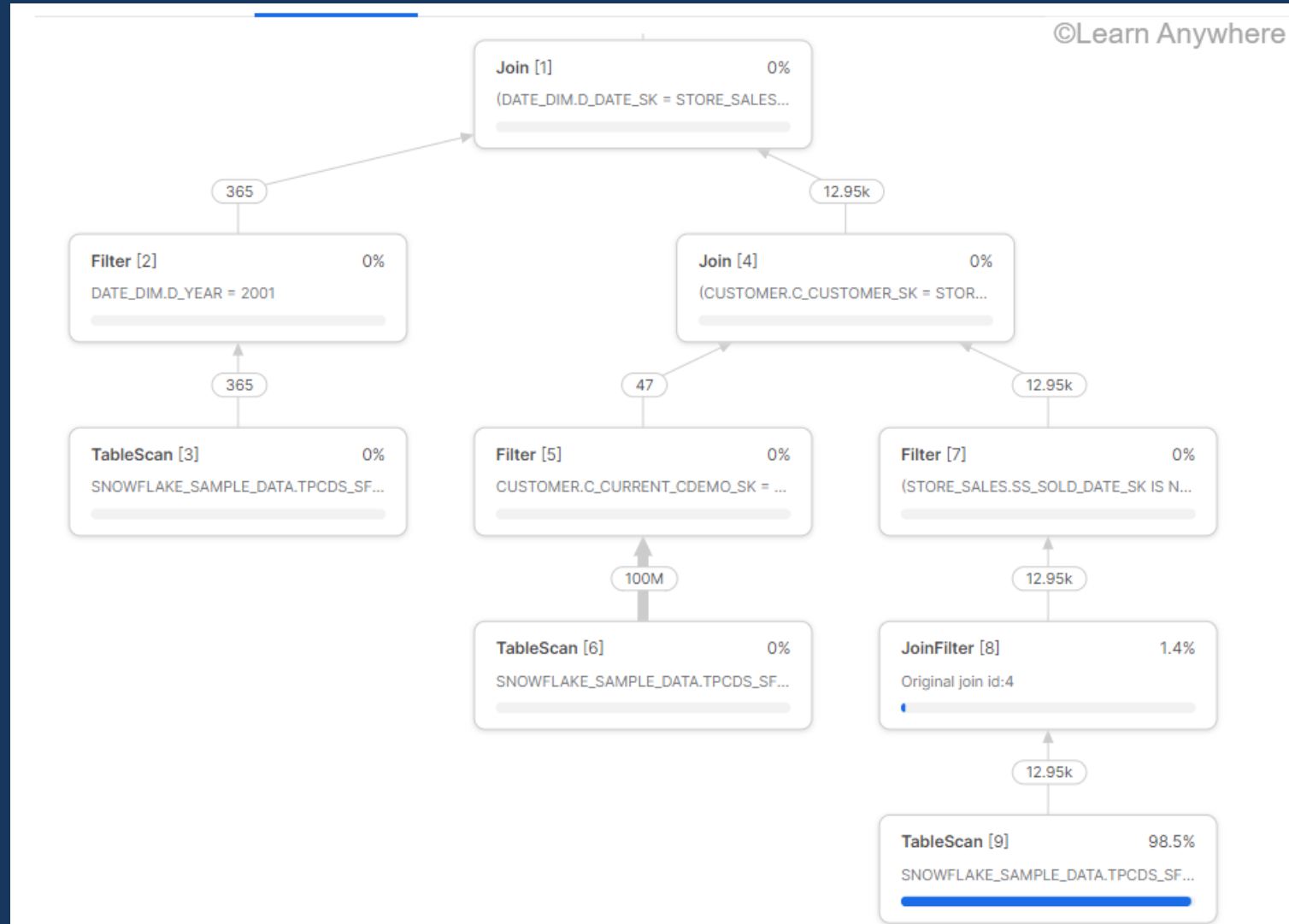
Copy Query ID

View Query Profile





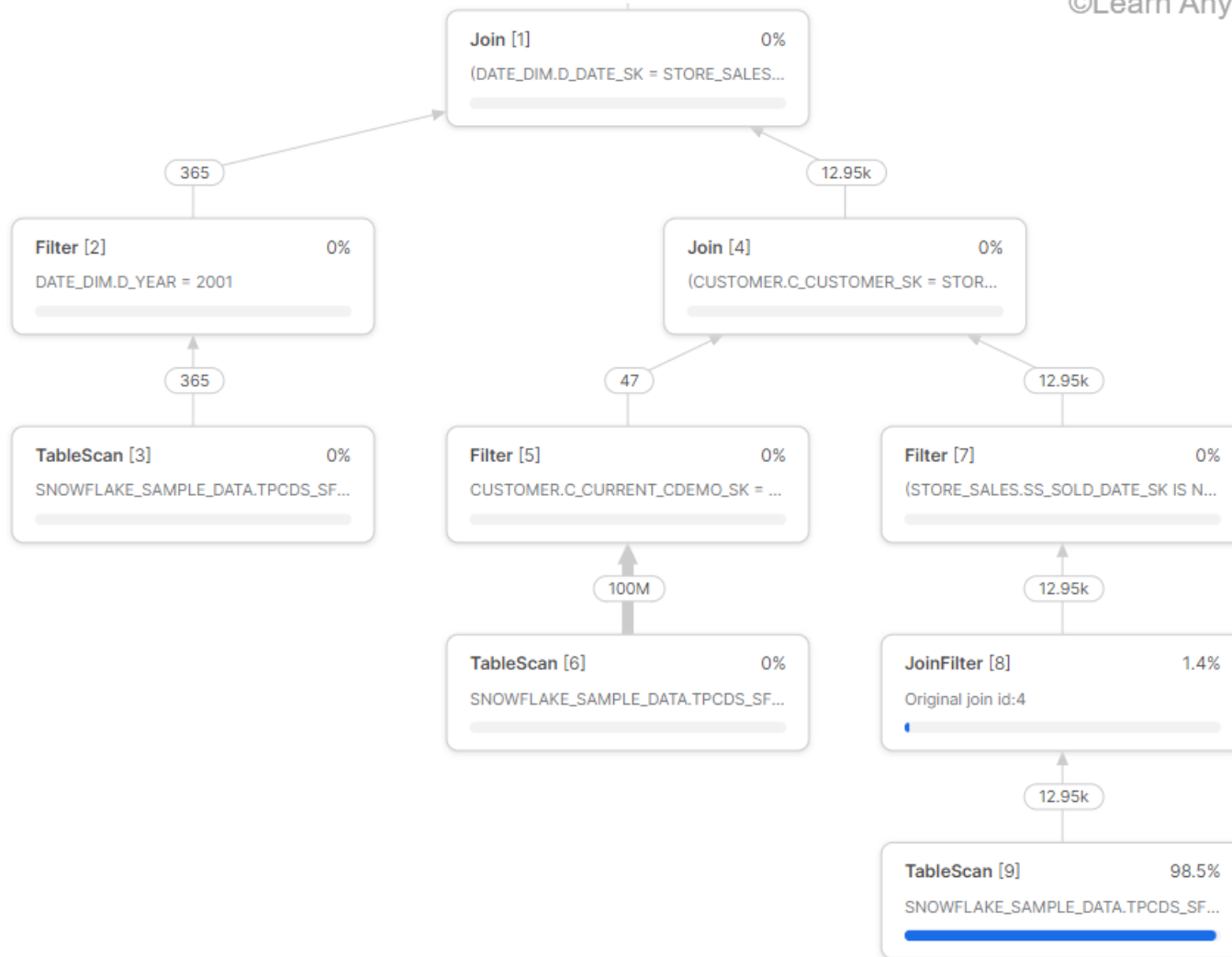
Query Profile





Query Profile

©Learn Anywhere



©Learn Anywhere

Most Expensive Nodes (2 of ...)

TableScan [9]	98.5%
JoinFilter [8]	1.4%

Profile Overview (Finished)

Total Execution Time (15m 26s) 100.0%

Processing	3.1%
Local Disk I/O	2.0%
Remote Disk I/O	94.8%
Initialization	0.1%

Statistics

Scan progress	100.00%
Bytes scanned	310.79GB
Percentage scanned from cache	0.00%
Bytes written to result	0.11MB
Partitions scanned	139738
Partitions total	721729



Query Profile : Data Spilling

©Learn Anywhere

Statistics

Scan progress	25.75%
Bytes scanned	541.63GB
Percentage scanned from cache	0.00%
Partitions scanned	196697
Partitions total	784050
Bytes spilled to local storage	400.87GB
Bytes spilled to remote storage	132.40GB

©Learn Anywhere

Statistics

Scan progress	27.87%
Bytes scanned	599.96GB
Percentage scanned from cache	0.00%
Bytes sent over the network	226.96GB
Partitions scanned	212942
Partitions total	784050
Bytes spilled to local storage	447.91GB





Query Profile

EXPLAIN PLANS

EXPLAIN [USING { TABULAR | JSON | TEXT }] <statement>

```
21
22 | EXPLAIN USING TEXT SELECT CC_CALL_CENTER_SK from SNOWFLAKE_SAMPLE_DATA.TPCDS_SF10TCL.CALL_CENTER;
23
24 -- Query History Function
25
26 SELECT * FROM TABLE(information_schema.query_history())
```

Results Chart

content
GlobalStats: partitionsTotal=1 partitionsAssigned=1 bytesAssigned=18432 Operations: 1:0 →Result CALL_CENTER.CC_CALL_CENTER_SK 1:1 →TableScan SNOWFLAKE_SAMPL

content

GlobalStats:
partitionsTotal=1
partitionsAssigned=1
bytesAssigned=18432

Operations:
1:0 →Result
CALL_CENTER.CC_CALL_CENTER_SK

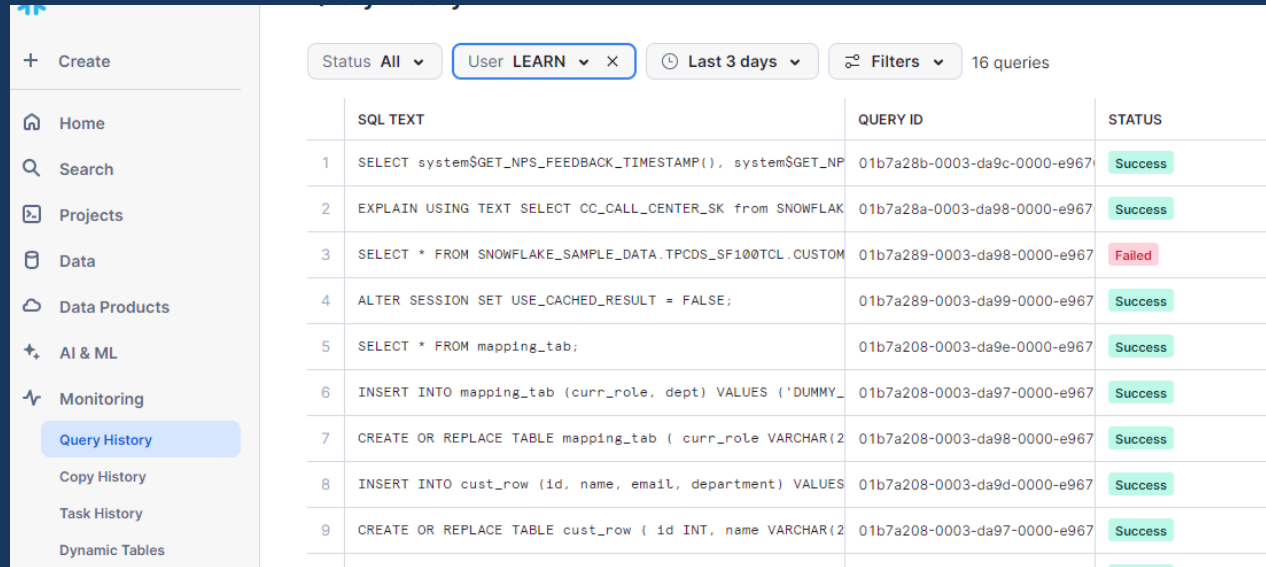
1:1 →TableScan
SNOWFLAKE_SAMPLE_DATA.TPCDS_SF10TCL.CALL_CENTER
CC_CALL_CENTER_SK
{partitionsTotal=1,
partitionsAssigned=1,
bytesAssigned=18432}



Query Profile

Query History(Option under Monitoring)

Provides 14 days history



Status: All User: LEARN Last 3 days Filters 16 queries			
	SQL TEXT	QUERY ID	STATUS
1	SELECT system\$GET_NPS_FEEDBACK_TIMESTAMP(), system\$GET_NP	01b7a28b-0003-da9c-0000-e967	Success
2	EXPLAIN USING TEXT SELECT CC_CALL_CENTER_SK from SNOWFLAK	01b7a28a-0003-da98-0000-e967	Success
3	SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.CUSTOM	01b7a289-0003-da98-0000-e967	Failed
4	ALTER SESSION SET USE_CACHED_RESULT = FALSE;	01b7a289-0003-da99-0000-e967	Success
5	SELECT * FROM mapping_tab;	01b7a208-0003-da9e-0000-e967	Success
6	INSERT INTO mapping_tab (curr_role, dept) VALUES ('DUMMY_	01b7a208-0003-da97-0000-e967	Success
7	CREATE OR REPLACE TABLE mapping_tab (curr_role VARCHAR(2	01b7a208-0003-da98-0000-e967	Success
8	INSERT INTO cust_row (id, name, email, department) VALUES	01b7a208-0003-da9d-0000-e967	Success
9	CREATE OR REPLACE TABLE cust_row (id INT, name VARCHAR(2	01b7a208-0003-da97-0000-e967	Success





Query Profile

QUERY_HISTORY (Under Information Schema)

Last 7 days activity

QUERY_HISTORY View (Under SF Account Usage)

Up to 365 days activity

```
-- Query History Function

SELECT * FROM TABLE(information_schema.query_history())
ORDER BY START_TIME;

-- Query History View 45 Min to 3 hours latency, also available for readers account usage

SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY;
```



Virtual Warehouse



Virtual Warehouse

snowflake

+ Create

Home

Search

Projects

Data

Data Products

AI & ML

Monitoring

Admin

Cost Management

Warehouses

Users & Roles

Accounts

Security

Contacts

Billing & Terms

Warehouses

+ Warehouse

Type All Size All Status All 2 Warehouses Columns

NAME	SI...	STATUS ↑	CLUSTERS	RUNNING ?	QUEUED ?	OWNER	RESUMED	
COMPUTE_WH	XS	Started		1	0	ACCOUNTADMIN	just now	...
SYSTEM\$STREAMLIT_NOTEBOOK_WH	XS	Suspended		0	0	—	3 weeks ago	...



Virtual Warehouse

New Warehouse

Creating as ACCOUNTADMIN

Type: Standard Size: X-Small

Advanced Options

The current advanced settings are auto-resume: on, auto-suspend: 5mins, multi-cluster: off, query acceleration: off

CancelCreate Warehouse

Type: Standard

Size: X-Small

Standard

Standard warehouses can be used for most workloads including Snowpark

Snowpark-optimized

Snowpark-optimized warehouses offer a larger memory and cache. They are often used for memory intensive operations

Size: X-Small

X-Small 1 credit/hour

Small 2 credits/hour

Medium 4 credits/hour

Large 8 credits/hour

X-Large 16 credits/hour

2X-Large 32 credits/hour

3X-Large 64 credits/hour

4X-Large 128 credits/hour

5X-Large 256 credits/hour

6X-Large 512 credits/hour

Virtual Warehouse

Warehouse Size	Credits / Hour	Credits / Second	Notes
X-Small	1	0.0003	Default size for warehouses created in Snowsight and using CREATE WAREHOUSE .
Small	2	0.0006	
Medium	4	0.0011	
Large	8	0.0022	
X-Large	16	0.0044	Default size for warehouses created using the Classic Console.
2X-Large	32	0.0089	
3X-Large	64	0.0178	
4X-Large	128	0.0356	
5X-Large	256	0.0711	Generally available in Amazon Web Services (AWS) and Microsoft Azure regions, and in preview in US Government regions.
6X-Large	512	0.1422	Generally available in Amazon Web Services (AWS) and Microsoft Azure regions, and in preview in US Government regions.



Virtual Warehouse

1 ▾

✓ 1
2
3
4
5
6
7
8
9
10

Advanced Options ▴

- ☒ **Auto resume**
Automatically resumes the warehouse when any statement that requires a warehouse is submitted.
- ☒ **Auto suspend**
Automatically suspends the warehouse if it is inactive for the specified period of time.

Suspend After

5 min(s) of inactivity

- ☒ **Multi-cluster Warehouse**
Scale compute resources as query concurrency needs change. [Learn more](#)

Min Clusters Max Clusters Scaling Policy

1 ▾ 2 ▾ Standard ▾

- ☒ **Query Acceleration**
Accelerate outlier queries with additional flexible compute resources. [Learn more](#)

Scale Factor ⓘ x8

Cancel

Create Warehouse

Standard ✓

Prevents/minimizes queuing of workloads

Economy

Conserves credits by running fully loaded clusters



Virtual Warehouse

```
CREATE [ OR REPLACE ] WAREHOUSE [ IF NOT EXISTS ] <name>
  [ [ WITH ] objectProperties ]
  [ [ WITH ] TAG ( <tag_name> = '<tag_value>' [ , <tag_name> = '<tag_value>' , ... ] ) ]
  [ objectParams ]

objectProperties ::=
  WAREHOUSE_TYPE = { STANDARD | 'SNOWPARK-OPTIMIZED' }
  WAREHOUSE_SIZE = { XSMALL | SMALL | MEDIUM | LARGE | XLARGE | XXLARGE | XXXLARGE | X4LARGE | X5LARGE | X6LARGE }
  RESOURCE_CONSTRAINT = { MEMORY_1X | MEMORY_1X_x86 | MEMORY_16X | MEMORY_16X_x86 | MEMORY_64X | MEMORY_64X_x86 }
  MAX_CLUSTER_COUNT = <num>
  MIN_CLUSTER_COUNT = <num>
  SCALING_POLICY = { STANDARD | ECONOMY }
  AUTO_SUSPEND = { <num> | NULL }
  AUTO_RESUME = { TRUE | FALSE }
  INITIALLY_SUSPENDED = { TRUE | FALSE }
  RESOURCE_MONITOR = <monitor_name>
  COMMENT = '<string_literal>'
  ENABLE_QUERY_ACCELERATION = { TRUE | FALSE }
  QUERY_ACCELERATION_MAX_SCALE_FACTOR = <num>

objectParams ::=
  MAX_CONCURRENCY_LEVEL = <num>
  STATEMENT_QUEUED_TIMEOUT_IN_SECONDS = <num>
  STATEMENT_TIMEOUT_IN_SECONDS = <num>

DROP WAREHOUSE IF EXISTS xs_warehouse;

USE WAREHOUSE compute_wh;

SHOW WAREHOUSES LIKE '%com%'
```

```
CREATE WAREHOUSE IF NOT EXISTS xs_warehouse
  WAREHOUSE_SIZE = XSMALL
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED=TRUE;
```

Scaling Out VS Scaling Up



Scale Out

*(Needs Enterprise
Edition or Higher)*



Multi-cluster Warehouse

Scale compute resources as query concurrency needs change. [Learn more](#)

Min Clusters

1



Max Clusters

10

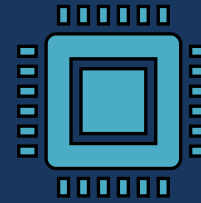
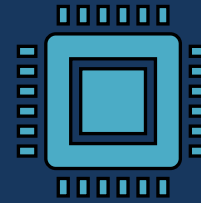
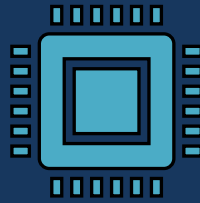
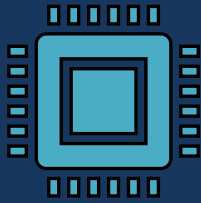


Scaling Policy

Standard



Scale Out



☒ **Multi-cluster Warehouse**
Scale compute resources as query concurrency needs change. [Learn more](#)

Min Clusters Max Clusters Scaling Policy

1



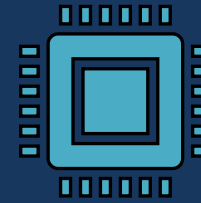
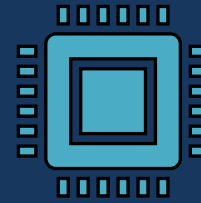
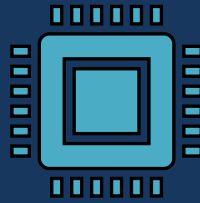
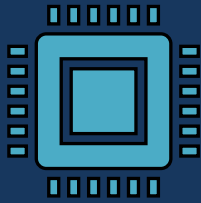
10



Standard



Scale Out



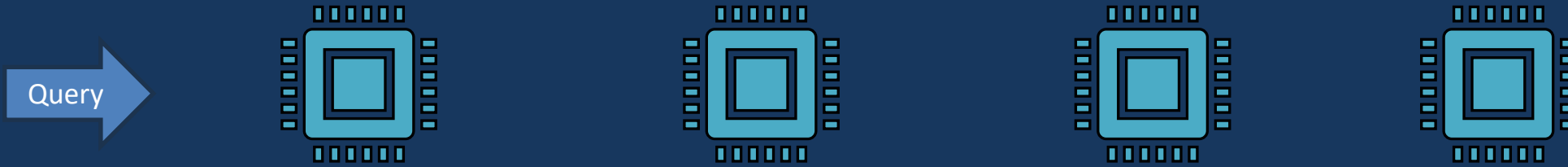
- Can be Automated
- Used when you have concurrent Queries
- Soft limit of Max 10 cluster

☒ **Multi-cluster Warehouse**
Scale compute resources as query concurrency needs change. [Learn more](#)

Min Clusters	Max Clusters	Scaling Policy
1	10	Standard



Standard Scaling Policy



Cluster Starts

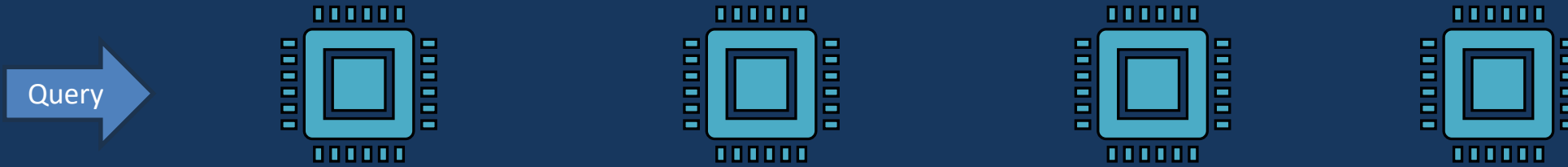
- Starts cluster when query is queued
- Each new cluster waits for 20 second after the prior one started

Cluster Shuts

- Cluster shuts, when load can be redistributed from least-loaded cluster to others
- After 2-3 consecutive checks and 1 minute interval



Economy Scaling Policy



Cluster Starts

- Starts cluster when enough load to run for 6 minute

Cluster Shuts

- Cluster shuts, when load can be redistributed from least-loaded cluster to others
- After 5-6 consecutive checks and 1 minute interval

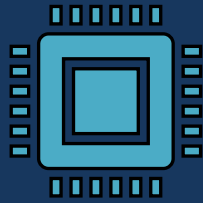


Scaling Policy

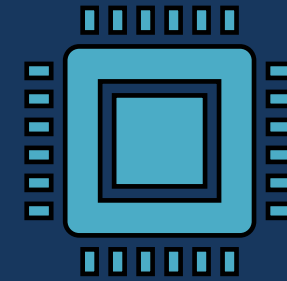
Policy	Description	Warehouse Starts...	Warehouse Shuts Down...
Standard (default)	Prevents/minimizes queuing by favoring starting additional clusters over conserving credits.	The first cluster starts immediately when either a query is queued or the system detects that there's one more query than the currently-running clusters can execute. Each successive cluster waits to start 20 seconds after the prior one has started. For example, if your warehouse is configured with 10 max clusters, it can take a full 200+ seconds to start all 10 clusters.	After 2 to 3 consecutive successful checks (performed at 1 minute intervals), which determine whether the load on the least-loaded cluster could be redistributed to the other clusters without spinning up the cluster again.
Economy	Conserves credits by favoring keeping running clusters fully-loaded rather than starting additional clusters, which may result in queries being queued and taking longer to complete.	Only if the system estimates there's enough query load to keep the cluster busy for at least 6 minutes.	After 5 to 6 consecutive successful checks (performed at 1 minute intervals), which determine whether the load on the least-loaded cluster could be redistributed to the other clusters without spinning up the cluster again.



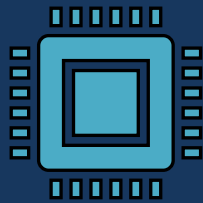
Scale Up/Down



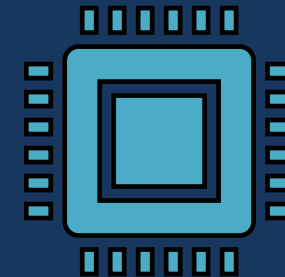
Small VW



BIG VW



Small VW



BIG VW



Resource Monitoring



Resource Monitoring



+ Create

Home

Search

Projects

Data

Data Products

AI & ML

Monitoring

Admin

Cost Management

Warehouses

Users & Roles

Accounts

Security

Contacts

Billing & Terms

Cost Management

Account Overview

Consumption

Resource Monitors

+ Resource Monitor

0 Resource Monitors

Search

Level All

Warehouses All

Frequency All



No Resource Monitors

There are no resource monitors associated with this role. Switch roles to view resource monitors available to that role.



Resource Monitoring

Customize Resource Monitor Schedule

Immediately Custom Start Date Custom End Date Range

Start Monitoring Immediately
End Monitoring Never
Resets Monthly

< October 2024 November 2024 >

M	T	W	T	F	S	S	M	T	W	T	F	S	S
	1	2	3	4	5	6		1	2	3			
7	8	9	10	11	12	13	4	5	6	7	8	9	10
14	15	16	17	18	19	20	11	12	13	14	15	16	17
21	22	23	24	25	26	27	18	19	20	21	22	23	24
28	29	30	31				25	26	27	28	29	30	

Resets Monthly

Monthly
Daily
Weekly
Yearly
Never

New Resource Monitor

Creating as ACCOUNTADMIN

Name sample

Credit Quota 100

Monitor Type Warehouse

Warehouse COMPUTE_WH

Schedule Start Monitoring Immediately
End Monitoring Never
Resets Monthly Customize

Actions Specify an action to perform when the quota is reached. Add

% Suspend immediately and notify when this % of credit is used. ?

% Suspend and notify when this % of credit is used. ?

% Notify when this % of credit is used. ?

Cancel Create Resource Monitor

Monitor Type Warehouse

Warehouse Account
Warehouse

Start Monitoring Immediately



Resource Monitoring

Profile

Profile photo ?

LA Upload

Username

LEARN

First Name

leanr

Last Name

any

Email

learn.anywhere1@gmail.com

Password

.....

Multi-factor authentication

Enroll

Each time you sign in to Snowflake, you'll use your password and a verification code. [Learn more](#)

Default role & warehouse

ACCOUNTADMIN • COMPUTE_WH (X-Small)

Language

English

Notifications

☒ Email notifications from resource monitors ⓘ

☐ When queries finish in the background

Cancel

Save



Resource Monitoring

- Monitoring credit usage by user managed WH and cloud services (Not Serverless)
- Need ACCOUNTADMIN Role to create, others can be granted view/modify access
- Can set at Account level or WH level
- Can set Credit Quota (Eg, 100 Credit)
- You can define frequency at which monitoring resets
(Daily, Weekly, Monthly, Yearly, Never)
- Action on reaching threshold
 - *Suspend Immediately*
 - *Suspend*
 - *Notify*

```
CREATE [ OR REPLACE ] RESOURCE MONITOR [ IF NOT EXISTS ] <name> WITH
    [ CREDIT_QUOTA = <number> ]
    [ FREQUENCY = { MONTHLY | DAILY | WEEKLY | YEARLY | NEVER } ]
    [ START_TIMESTAMP = { <timestamp> | IMMEDIATELY } ]
    [ END_TIMESTAMP = <timestamp> ]
    [ NOTIFY_USERS = ( <user_name> [ , <user_name> , ... ] ) ]
    [ TRIGGERS triggerDefinition [ triggerDefinition ... ] ]

triggerDefinition ::=
    ON <threshold> PERCENT DO { SUSPEND | SUSPEND_IMMEDIATE | NOTIFY }
```



*(Needs Enterprise
Edition or Higher)*

Query Acceleration Service



Query Acceleration Service

- QAS can accelerate part of WH workload
- Reduces the impact of outlier query
- Works for
 - Ad hoc Analysis
 - Unpredictable data volume per query
 - Queries with large scan and selective filter





Query Acceleration Service

```
-- Check various QAS parameters
```

```
SELECT parse_json(system$estimate_query_acceleration('01ae2979-3200-d4c9-0005-3db60005f1be'));
```

```
{
  "estimatedQueryTimes": {
    "1": 473,
    "2": 319,
    "28": 44,
    "4": 196,
    "8": 114
  },
  "originalQueryTime": 927.589,
  "queryUUID": "01ae2979-3200-d4c9-0005-3db60005f1be",
  "status": "eligible",
  "upperLimitScaleFactor": 28
}
```

```
-- Scaling factor cost you in multiple of scaling factor, example scale factor of 5 can cost 5X credit of normal use
```

```
ALTER WAREHOUSE COMPUTE_WH SET
  ENABLE_QUERY_ACCELERATION = TRUE
  QUERY_ACCELERATION_MAX_SCALE_FACTOR = 28;
```

*(Needs Enterprise
Edition or Higher)*

Search Optimization Service



Search Optimization Service

The search optimization service can significantly improve the performance of certain types of lookup and analytical queries that use an extensive set of predicates for filtering.



Search Optimization Service

What Kind of Queries can be improved?

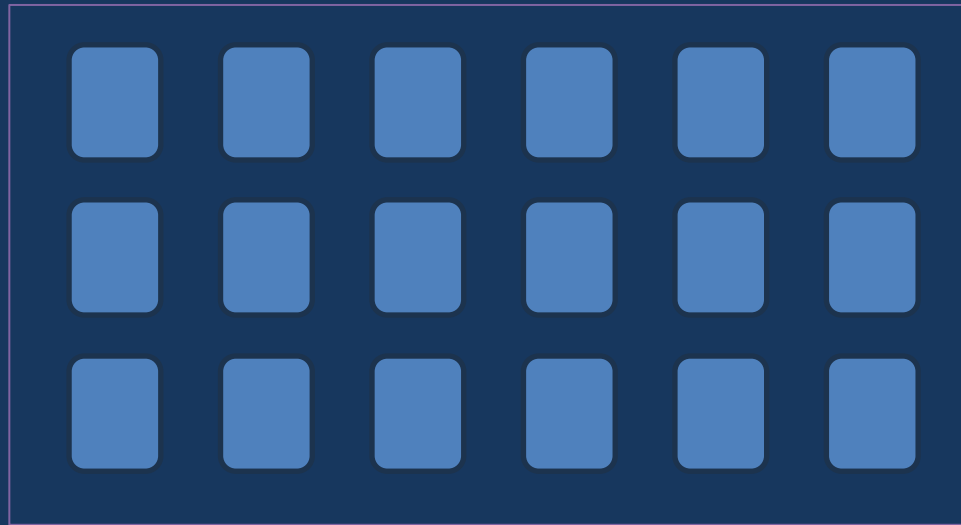
- Selective point look up queries, returns small distinct rows
- Substring or regular expression searches (LIKE, ILIKE etc.)
- Queries on fields in VARIANT, OBJECT, ARRAY columns
(Equality, IN Predicates etc.)
- Queries using selected geo func

```
SELECT * FROM SAMPLE_TABLE WHERE CUSTOMER = '1234'
```



Search Optimization Service

*SELECT * FROM SAMPLE_TABLE WHERE CUSTOMER = '1234'*



Search Optimization Service

*SELECT * FROM SAMPLE_TABLE WHERE CUSTOMER = '1234'*



SOS creates and maintains a persistent data structure called [search access path](#).



Search Optimization Service

- *Search Access Path* gets updated with data needed for look up
- New updates in table gets automatically updated
- Check progress `search_optimization_progress` in SHOW TABLES
- Cost for storage and compute



Search Optimization Service

- *Search Access Path* gets updated with data needed for look up
- New updates in table gets automatically updated
- Check progress `search_optimization_progress` in SHOW TABLES
- Cost for storage and compute



Search Optimization Service

Before SOS

- Query should have a column with 100-200k distinct values
- Selective filters brings very few records
- Query runs at least for few seconds



Search Optimization Service

SOS Vs Clustering

- SOS applies for all columns
- SOS uses additional data structure



Search Optimization Service

```
-- Set the search optimization  
  
ALTER TABLE SOS_CUSTOMER add search optimization;
```

search_optimization	search_optimization_progress	search_optimization_bytes
OFF	null	null

search_optimization	search_optimization_progress	search_optimization_bytes
ON	100	2291359744

```
-- Check the credit used details  
  
SELECT * FROM TABLE(information_schema.search_optimization_history());
```

Results  Chart

START_TIME	END_TIME	CREDITS_USED	TABLE_NAME
2024-10-12 06:59:19.000 -0700	2024-10-12 07:59:19.000 -0700	0.035606618	LA_DB.LA_SCHEMA.SOS_CUSTOMER



Use of Specific **SELECT** Commands





Use of Specific **SELECT** Commands

- Selecting only needed column in the query
- Use proper filtering
- Use windows function
- Use CTE (Common Table Expressions)
- Create Materialized View
- Use proper joining conditions
- Use clustering Key effectively
- Leverage Snowflake Cache



Snowflake Loading



Various Data Types



Data Source



Snowflake Loading



Limit of 250 MB



Snowflake Loading

Bulk Loading



Copy Command

Large Data

User Managed WH

64 days Metadata

Auth done by client

Continuous Loading



Snowpipe + Copy

Small Chunks

Serverless

14 days Metadata

Requires key pair authentication
with JSON Web Token

Supported Type :

CSV, TSV, JSON,
PARQUET, AVRO,
ORC,XML



Bulk Loading



Bulk Loading

- Load using Copy Command
- Can also transform the data
 - Column re-ordering*
 - Column omission*
 - Casts*
 - Truncating text strings that exceed the target column length*
- Auto Duplicate detection
- Stored in the metadata of the target table for 64 days. Available upon completion of the COPY statement as the statement output.



Bulk Loading



Bulk Loading

--Truncating text strings that exceed the target column length, reorder, cast and drop columns and FORCEfully Reload

```
COPY INTO new_table2 (customer, full_name, region, email, gender, "ORDER", extra_col)
FROM (select t.$1, CONCAT(t.$2, ' ', t.$3), t.$4, t.$5, t.$6, cast(t.$7 AS INTEGER), to_binary(t.$4, 'utf-8') from @internal_named_stage/data_0_0_0.csv t)
FILE_FORMAT = (TYPE = 'CSV' SKIP_HEADER = 1)
TRUNCATECOLUMNS = TRUE; --Allow Truncation
FORCE = TRUE; -- Reload the file
```



Bulk Loading



Bulk Loading

Copy INTO TABLE Options

```
ON_ERROR = { CONTINUE | SKIP_FILE (DEFAULT IN SNOWPIPE) | SKIP_FILE_<num> | 'SKIP_FILE_<num>%' | ABORT_STATEMENT (DEFAULT IN BULK LOADING) }
SIZE_LIMIT = <num>
RETURN_FAILED_ONLY = TRUE | FALSE (DEFAULT)
MATCH_BY_COLUMN_NAME = CASE_SENSITIVE | CASE_INSENSITIVE | NONE (DEFAULT) ---
ENFORCE_LENGTH = TRUE (DEFAULT) | FALSE

TRUNCATECOLUMNS = TRUE | FALSE
FORCE = TRUE | FALSE
PURGE = TRUE | FALSE
```

```
-- ABORT STATEMENT IS BY DEFAULT
```

```
COPY INTO customers_named
FROM @internal_named_stage
FILE_FORMAT = my_csv_ff
ON_ERROR = ABORT_STATEMENT;
```

```
-- Use of CONTINUE to ignore error records
```

```
COPY INTO customers_named
FROM @internal_named_stage
FILE_FORMAT = my_csv_ff
ON_ERROR = CONTINUE;
```



Bulk Loading



Bulk Loading

```
-- SKIP the file if erroneous record
```

```
COPY INTO customers_named  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
ON_ERROR = SKIP_FILE;
```

```
-- Ignore first 2 error
```

```
COPY INTO customers_named  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
ON_ERROR = SKIP_FILE_2;
```

```
-- Ignore first 2% error
```

```
COPY INTO customers_named  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
ON_ERROR = 'SKIP_FILE_2%';
```



Bulk Loading



Bulk Loading

```
-- Size Limit Option, First file will always go through
```

```
COPY INTO customers_named  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
SIZE_LIMIT = 640;
```

```
COPY INTO customers_named  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
ON_ERROR = CONTINUE  
RETURN_FAILED_ONLY = TRUE;
```

```
COPY INTO customers_named  
FROM @internal_named_stage/MOCK.CSV  
FILE_FORMAT = my_csv_ff2  
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE;
```



Bulk Loading



Bulk Loading

```
-- ENFORCE_LENGTH = TRUE (DEFAULT) | FALSE  
-- Reverse of TRUNCATECOLUMN
```

```
COPY INTO customers_named2  
FROM @internal_named_stage/MOCK.CSV  
FILE_FORMAT = my_csv_ff  
ENFORCE_LENGTH = TRUE;
```

```
--Loads the file, ignores previous load
```

```
COPY INTO customers_named2  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
FORCE = TRUE;
```

```
--Auto delete the data in stage
```

```
COPY INTO customers_named2  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
PURGE = TRUE;
```



Snowflake Loading

Bulk Loading



Copy Command

Large Data

User Managed WH

64 days Metadata

Auth done by client

Continuous Loading



Snowpipe + Copy

Small Chunks

Serverless

14 days Metadata

Requires key pair authentication
with JSON Web Token

Supported Type :

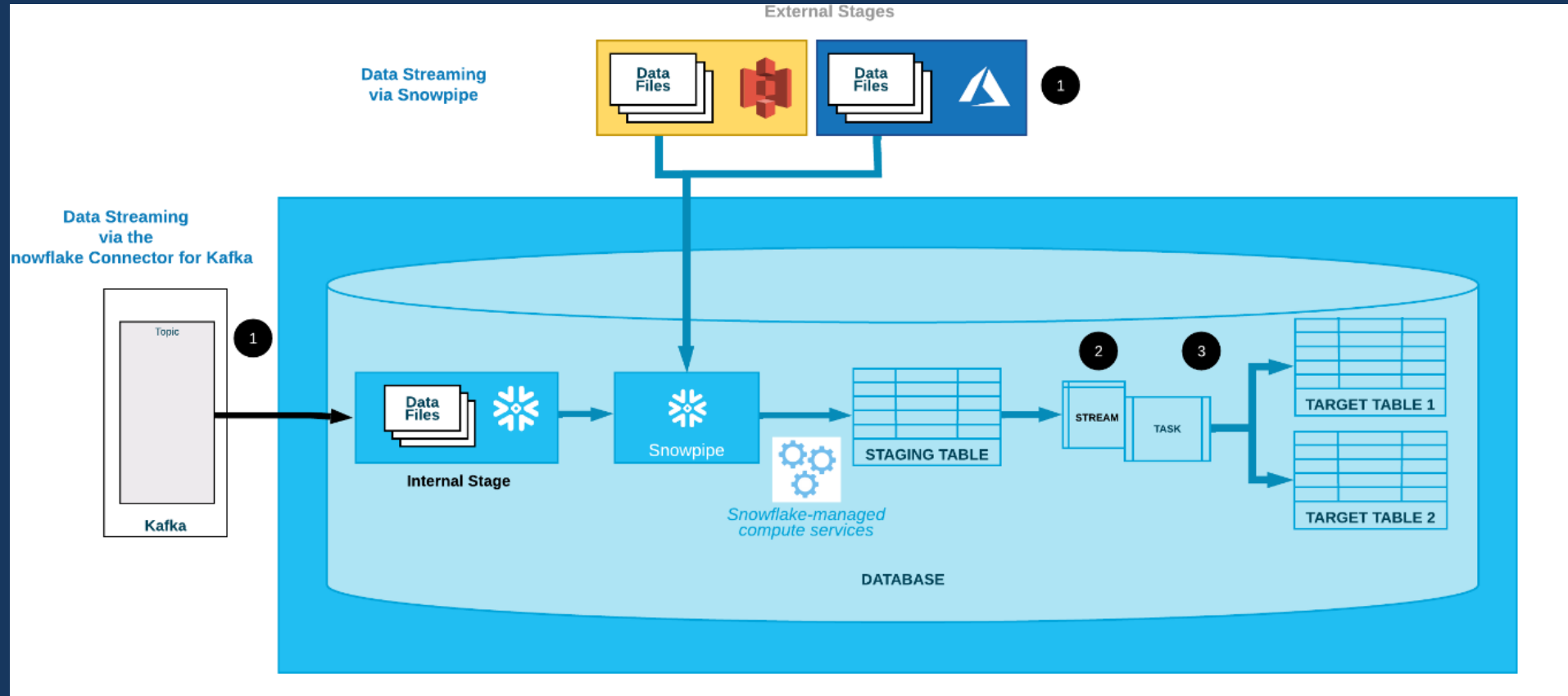
CSV, TSV, JSON,
PARQUET, AVRO,
ORC,XML



Snowpipe



Continuous Loading



Source : Snowflake





Continuous Loading

File Considerations while loading (Also applicable for bulk loading)

- The number of load operations that run in parallel cannot exceed the number of data files to be loaded
- Split Large files in smaller ones and combine small ones in large one
- Ideal size 100-250 mb (or larger) in size compressed
- Loading very large file eg. 100 GB or larger, not recommended
- For large Files , carefully use ON_ERROR option
- If load goes beyond 24 hours then job can be cancelled with no data load
- Consider accumulating file in source at 1 minute frequency



SnowPipe

```
-- Template for PIPE
CREATE [ OR REPLACE ] PIPE [ IF NOT EXISTS ] <name>
  [ AUTO_INGEST = [ TRUE | FALSE ] ]
  [ ERROR_INTEGRATION = <integration_name> ]
  [ AWS_SNS_TOPIC = '<string>' ]
  [ INTEGRATION = '<string>' ]
  [ COMMENT = '<string_literal>' ]
AS <copy_statement>;
```

```
-- Create a Pipe to Copy

CREATE OR REPLACE PIPE MYPIPE
AS
COPY INTO NEW_PIPE_TABLE FROM @INTERNAL_NAMED_STAGE;
```

```
-- Show all PIPES

SHOW PIPES;

--Describe MYPIPE

DESC PIPE MYPIPE;

-- Check data in the table

SELECT COUNT(*) from NEW_PIPE_TABLE;

-- Check Pipe Status
SELECT SYSTEM$PIPE_STATUS( 'MYPIPE' );

-- Refres to get it activate
ALTER PIPE MYPIPE REFRESH;
```



File Format

```
-- Copy CommandALL
COPY INTO customers_named
FROM @internal_named_stage
FILE_FORMAT = (TYPE = 'CSV' SKIP_HEADER = 1) PURGE = TRUE;
```

```
-- File Format definition

CREATE OR REPLACE FILE FORMAT my_csv_ff
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  NULL_IF = ('NULL', 'null')
  EMPTY_FIELD_AS_NULL = true
  COMPRESSION = NONE;
```

```
--Use file format

COPY INTO customers_named
FROM @internal_named_stage
FILE_FORMAT = my_csv_ff FORCE = TRUE;
```

TYPE = CSV | JSON | AVRO | ORC | PARQUET | XML | CUSTOM



File Format

```
COPY INTO mytable
FROM @internal_named_stage
FILE_FORMAT = my_csv_ff;

CREATE STAGE my_int_stage
FILE_FORMAT = my_csv_ff;

CREATE TABLE mytable
USING TEMPLATE (
  SELECT ARRAY_AGG(OBJECT_CONSTRUCT(*))
  WITHIN GROUP (ORDER BY ORDER_ID)
  FROM TABLE(
    INFER_SCHEMA(
      LOCATION=>'@mystage',
      FILE_FORMAT=>'my_csv_ff'
    )
  )
);
```



PUT Command

```
PUT file://<path_to_file>/<filename> internalStage
[ PARALLEL = <integer> ] -- 1-99, Default 4
[ AUTO_COMPRESS = TRUE | FALSE ] --Default is GZIP for auto compress
[ SOURCE_COMPRESSION = AUTO_DETECT | GZIP | BZ2 | BROTLI | ZSTD | DEFLATE | RAW_DEFLATE | NONE ]
[ OVERWRITE = TRUE | FALSE ]
```

-- Load into Named Stage

```
put file://C:\data\MOCK.CSV @internal_named_stage AUTO_COMPRESS = FALSE;
put file://C:\data\MOCK1.CSV @internal_named_stage AUTO_COMPRESS = FALSE;OVERWRITE = TRUE;
```

-- Load into Table Stage

```
put file://C:\data\MOCK.CSV @%customers_table/staged AUTO_COMPRESS = FALSE;
put file://C:\data\MOCK1.CSV @%customers_table/staged AUTO_COMPRESS = FALSE;
```

-- Load into User Stage

```
put file://C:\data\MOCK.CSV @~/customers_user/staged AUTO_COMPRESS = FALSE;
put file://C:\data\MOCK1.CSV @~/customers_user/staged AUTO_COMPRESS = FALSE;
```



Validation Mode and **Validate**

- Validation Mode, to test the data before loading
- VALIDATION_MODE = RETURN_n_ROWS | RETURN_ERRORS | RETURN_ALL_ERRORS

```
-- Return all error by simulation
```

```
COPY INTO customers_named3  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
VALIDATION_MODE = 'RETURN_ERRORS';
```

```
-- Just check first N rows for error
```

```
COPY INTO customers_named3  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
VALIDATION_MODE = 'RETURN_10_ROWS';
```

```
-- Return all error by simulation, also include previous partialy loaded file
```

```
COPY INTO customers_named3  
FROM @internal_named_stage  
FILE_FORMAT = my_csv_ff  
VALIDATION_MODE = 'RETURN_ALL_ERRORS';
```



Validation Mode and **Validate**

- VALIDATE : Validates the files loaded in a past execution of the COPY INTO <table> command

```
-- VALIDATE Option to check error in last COPY COmmand

SELECT * FROM TABLE(VALIDATE(customers_named, JOB_ID => '_last')); --01ade083-3200-d1bb-0005-3db60002918a

-- Save error in separate table

CREATE OR REPLACE TABLE copy_errors AS SELECT * FROM TABLE(VALIDATE(customers_named, JOB_ID => '01ade083-3200-d1bb-0005-3db60002918a'));

SELECT * FROM copy_errors;
```



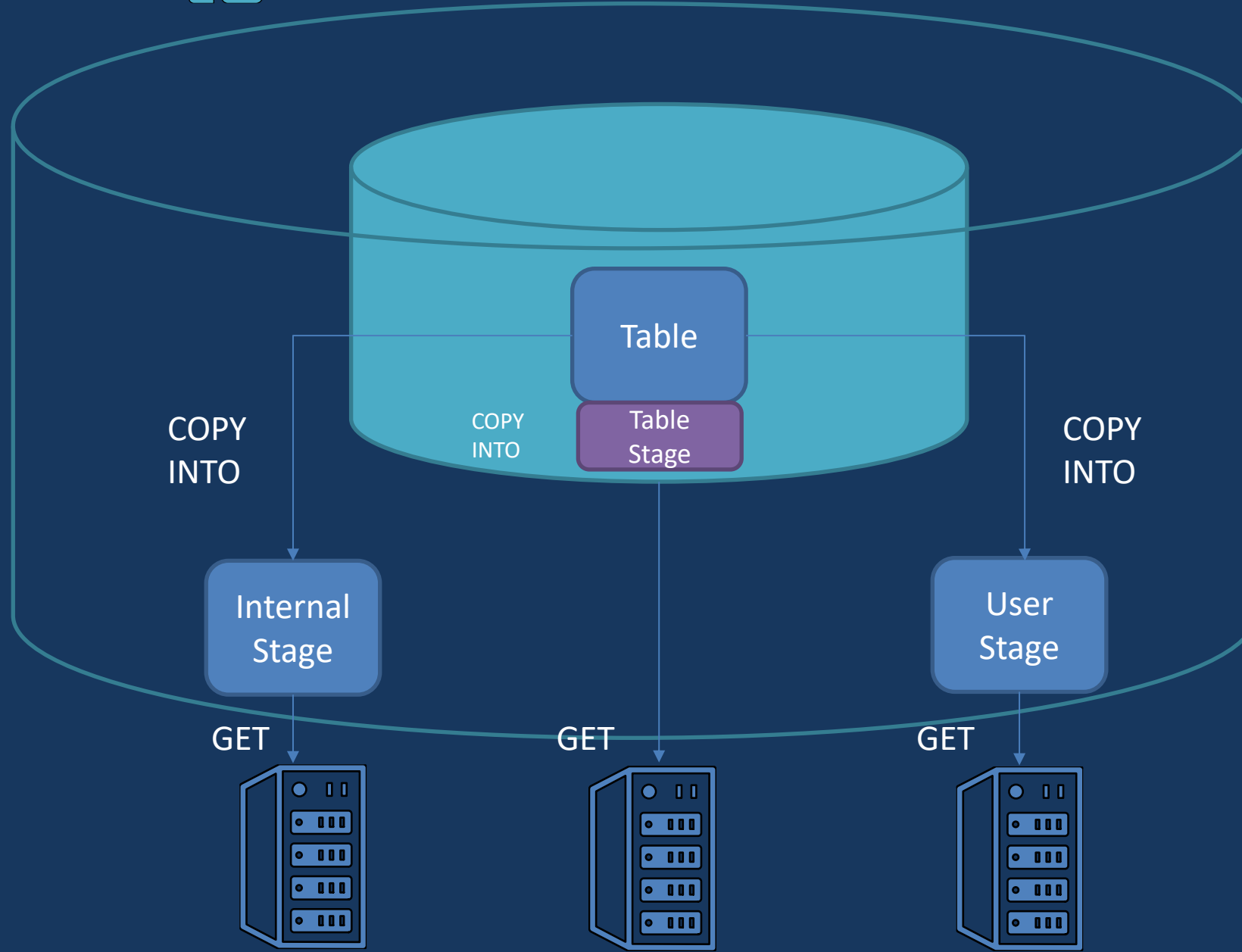


Unloading Data



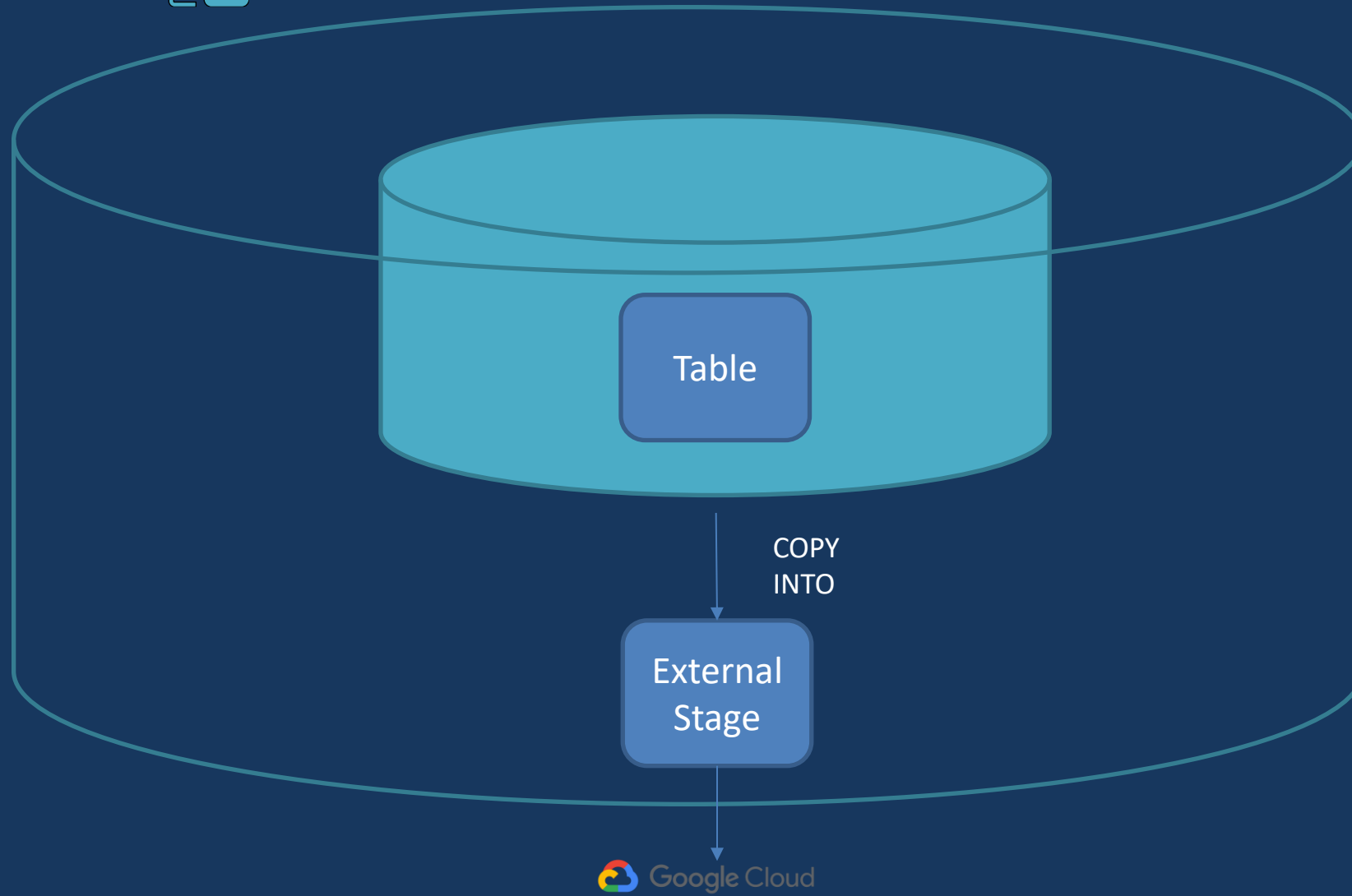


Unloading Data





Unloading Data



Google Cloud





Unloading Data

- Bulk unloading with **COPY INTO** <Location>
- For getting single file from multiple files use **SINGLE = TRUE**, default is **FALSE**
- Can partition the data by **PARTITION BY**

File Format

- CSV,TSV
- JSON
- PARQUET

File Encoding

- UTF-8

File Compression

- Gzip
- Bzip2
- Brotli
- Zstandard

Encryption of o/p File

- Internal (128/256-bit key)
- External (Cloud-User supplied Key)





Unloading Data

```
COPY INTO @internal_named_stage from customers_named FILE_FORMAT = (type = 'CSV' COMPRESSION='NONE')  
OVERWRITE = TRUE  
HEADER= TRUE  
SINGLE= TRUE  
MAX_FILE_SIZE=16777216; --Default 16MB (can be increased), 5 GB for cloud
```

```
COPY INTO @internal_named_stage1 from customers_named  
PARTITION BY region  
FILE_FORMAT = (type = 'CSV' COMPRESSION='NONE') ;
```

```
--Download the file to desktop (Only works in SnowSQL)
```

```
GET @internal_named_stage file:///C:\data\unload;
```

```
--List files in user stage  
list @~;  
LS @~;
```



Empty Strings and NULL Value

- Empty strings Zero length and no characters , denoted as ""
- NULL values means absence of data, demoted as null



Compression Method

```
COPY INTO @my_stage/compressed_file.csv.gz  
FROM (SELECT * FROM my_table)  
FILE_FORMAT = (TYPE = 'CSV' COMPRESSION = 'GZIP');  
  
GET @my_stage/compressed_file.csv.gz file:///path/to/your/desktop/compressed_file.csv.gz;
```



Insert Overwrite

```
-- Insert rows
```

```
INSERT INTO employees (id, name, department) VALUES  
(1, 'John Doe', 'Engineering'),  
(2, 'Jane Doe', 'Sales');
```

```
-- This will overwrite the existing data in the employees table with the new data from the select statement.
```

```
INSERT INTO employees SELECT id, name, department FROM employees WHERE department = 'Engineering';
```

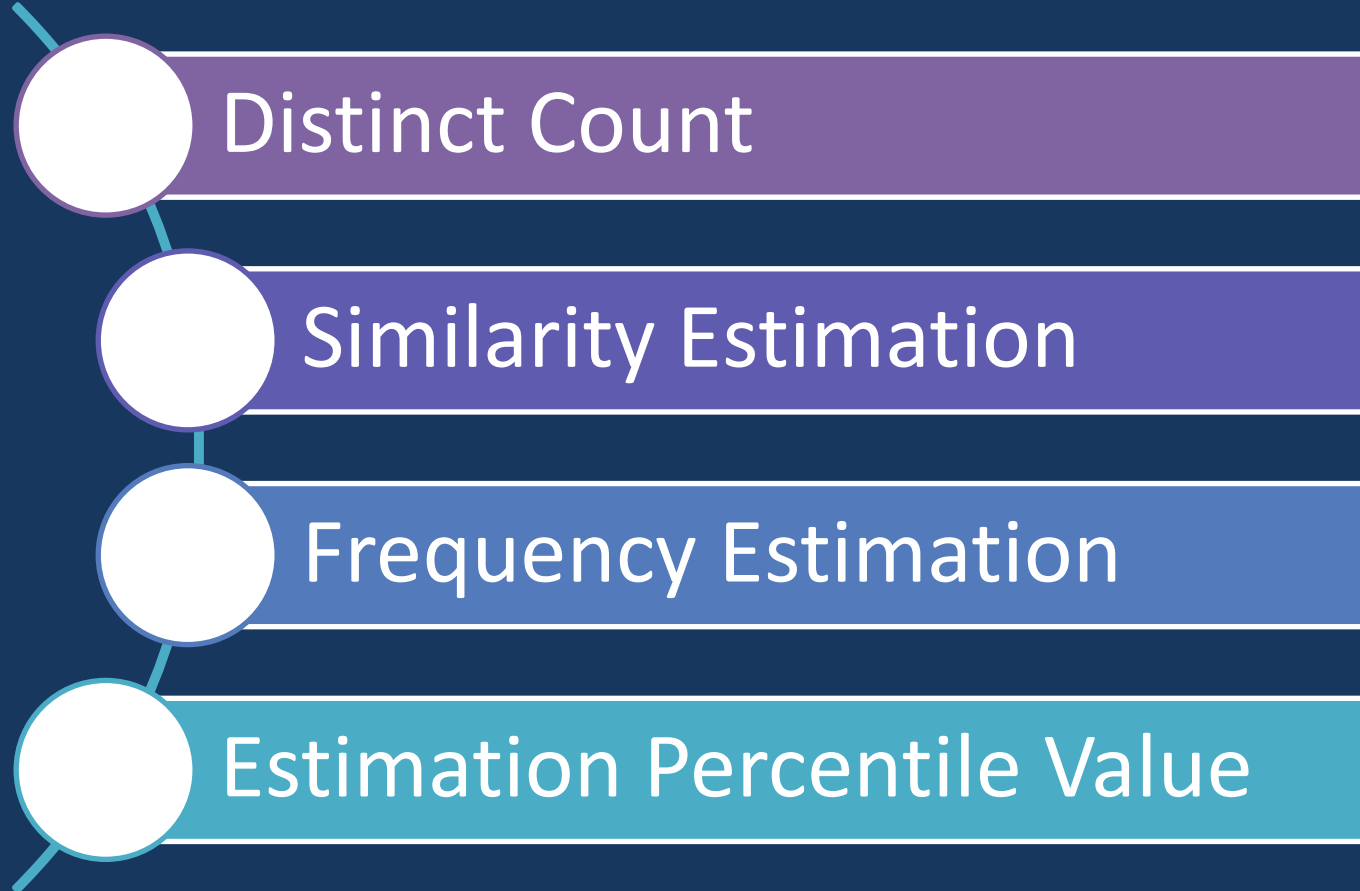
```
SELECT * FROM employees;
```



fx Estimating Functions



fx Estimating Functions



fx Distinct Count

- Idea is to count distinct rows in a table
- Snowflake uses HLL (HyperLogLog) Algo
- HLL error rate is 1.62338%

12
13
14
15

```
SELECT COUNT(DISTINCT WR_ORDER_NUMBER) FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.WEB_RETURNS;
```

Results Chart

	COUNT(DISTINCT WR_ORDER_NUMBER)
1	4,240,802,950

Query Details ...

Query duration 6m 25s

Rows 1

16
17

```
SELECT APPROX_COUNT_DISTINCT (WR_ORDER_NUMBER) FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.WEB_RETURNS;
```

Results Chart

	APPROX_COUNT_DISTINCT (WR_ORDER_NUMBER)
1	4,201,561,184

Query Details ...

Query duration 37s



*fx*Similarity Estimation

- Find Similarity between 2 or more datasets
- Uses Minhash function
- Output ranges from 0 to 1,
- 1 : Same table, 0 - Completely distinct



fxSimilarity Estimation

	ID	NAME	CITY	STATE
1	1	John Doe	San Francisco	CA
2	2	Jane Doe	New York	NY
3	3	Peter Smith	Chicago	IL
4	4	Susan Jones	Los Angeles	CA
5	5	David Williams	Seattle	WA

	ID	NAME	CITY	STATE
1	1	John Doe	San Francisco	CA
2	2	Jane Doe	New York	NY
3	3	Peter Smith	Chicago	IL
4	4	Susan Jones	Los Angeles	CA
5	5	David Williams	Seattle	WA
6	6	Mary Johnson	Dallas	TX
7	7	Michael Brown	Miami	FL

```
68 select approximate_similarity(mh)
69 from (
70     select minhash(100, *) as mh from sample_table1
71     union all
72     select minhash(100, *) as mh from sample_table2
73 );
74
```

→ Results

~ Chart

APPROXIMATE_SIMILARITY(MH)

0.71

- Max K value is 1024, recommended is 100



*fx*Frequency Estimation

- To approximate frequency of value in the dataset
- Snowflake uses Space-saving Algo
- We use function like : APPROX_TOP_K,
APPROX_TOP_K_ACCUMULATE/COMBINE/ESTIMATE



fxFrequency Estimation

```
79
80 SELECT TOP 5 HD_BUY_POTENTIAL, COUNT(HD_BUY_POTENTIAL) as Num FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.HOUSEHOLD_DEMOGRAPHICS
81 GROUP BY HD_BUY_POTENTIAL
82 ORDER BY Num DESC;
83
```

Results Chart

	HD_BUY_POTENTIAL	NUM
1	1001-5000	1200
2	501-1000	1200
3	0-500	1200
4	Unknown	1200
5	5001-10000	1200

```
83
84 SELECT APPROX_TOP_K (HD_BUY_POTENTIAL, 5, 10000)
85 FROM SNOWFLAKE_SAMPLE_DATA.TPCDS_SF100TCL.HOUSEHOLD_DEMOGRAPHICS;
86
87
```

Results Chart

	APPROX_TOP_K (HD_BUY_POTENTIAL,5,10000)
1	[["0-500", 1200], ["Unknown", 1200], [">10000", 1200], ["5001-10000", 1200], ["1001-5000", 1200]]

```
[ ] APPROX_TOP_K (HD_BUY_POTENTIAL,5,10000)
[
  [
    "0-500",
    1200
  ],
  [
    "Unknown",
    1200
  ],
  [
    ">10000",
    1200
  ],
  [
    "5001-10000",
    1200
  ],
  [
    "1001-5000",
    1200
  ]
]
```



*fx*Percentile Estimation

- To approximate percentile values in the dataset
- Snowflake uses t-Digest Algo
- We use function like : APPROX_PERCENTILE,
APPROX_PERCENTILE_ACCUMULATE/COMBINE/ESTIMATE
- Approximate values
- Works only on Numeric Value
- May return data which is not part of the dataset



*fx*Percentile Estimation

C1
0
1
2
3
4
5
6
7
8
9
10

108 | SELECT APPROX_PERCENTILE(c1, 0.99) FROM testtable;

109

↩ Results

~ Chart

	APPROX_PERCENTILE(C1, 0.99)	
1	9.9	





SAMPLING





SAMPLING

FIXED-SIZE

Fixed Numbers of rows are returned

FRACTION BASED (*Faster*)

BERNOULLI | ROW : $(p/100)*n$, Number of rows vary, row wise

SYSTEM | BLOCK : Faster, Might be biased, block wise, often faster than Bernoulli/Row for small dataset

Sampling on Join is possible if, BERNOULLI/ROW BASED, No Seed Used.

SAMPLE and TABLESAMPLE are synonymous
SEED and REPATABLE are synonymous





SAMPLING

FIXED-SIZE

Fixed Numbers of rows are returned

```
40  
41 | SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER SAMPLE(100 ROWS);
```

↪ Results ~ Chart

	COUNT(*)
1	100





SAMPLING

FRACTION BASED

BERNOULLI | ROW : $(p/100)*n$, Number of rows vary, row wise

```
34  
35 | SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER TABLESAMPLE BERNOULLI (20.3) SEED(10);  
36
```

Results Chart

	COUNT(*)
1	30514

```
37 | SELECT count(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER SAMPLE ROW (10);  
38  
39 -- Fixed Size Row Sampling  
40
```

Results Chart

	COUNT(*)
1	15078



SAMPLING

FRACTION BASED

BERNOULLI | ROW : $(p/100)*n$, Number of rows vary, row wise

SYSTEM | BLOCK : Faster, Might be biased, block wise

27		SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER TABLESAMPLE SYSTEM (1);
28		SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER TABLESAMPLE BLOCK (1);
Results Chart		
		COUNT(*)
1		1504

29		SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER TABLESAMPLE BLOCK (1);
30		
31		
32		
Results Chart		
		COUNT(*)
1		1500



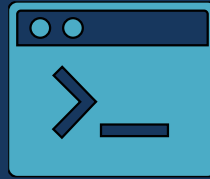
fx Supported-Function Type



fx Supported-Function Type



Scalar



Window



Table



Aggregate



System

Snowflake also supports , UDF and External Function



fx Supported-Function Type



Scalar

Returns 1 value per invocation

Like : File Function, Numeric Function etc

Example : SELECT CEIL(135.135);

Returns >> 136



fx Supported-Function Type



Aggregate

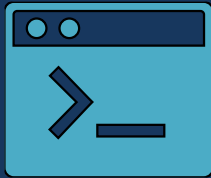
Returns 1 value per group of rows

Like : SUM, AVG etc

*Example : SELECT **AVG**(salary) from <table>;*



fx Supported-Function Type



Window

Aggregate functions that can operate on a subset of rows within the set of input rows

Like : ANY_VALUE, RATIO_TO_REPORT etc

*Example : SELECT province, store_ID, profit,
100 * RATIO_TO_REPORT(profit) OVER (PARTITION BY province) AS percent_profit
FROM store_profit
ORDER BY province, store_ID;*



fx Supported-Function Type



System

Control Functions that allows you to execute action in the system.

*Like : **SYSTEM\$**.... etc*

SELECT SYSTEM\$ABORT_SESSION(1065153868222);



fx Supported-Function Type



Table

A table function returns a set of rows for each input row. The returned set can contain zero, one, or more rows. Each row can contain one or more columns.

*Like : **VALIDATE....** etc*



Table Function



Table Function

Table functions return a set of rows for each input row, which can contain zero, one, or more rows and columns.

Typically used in FROM clause
It is wrapped TABLE()

- **1-to-N**

Functions that return multiple rows for each input, such as record high temperatures for a specified date.

- **M-to-N**

Functions that depend on multiple input rows to generate each output row, like moving averages of stock prices.



Table Function

- System Defined Table Function

Snowflake provides hundreds of built-in table functions, eg. *FLATTEN*, *VALIDATE*

- User Defined Table Function

Users can create their own table functions. Like *UDTF*



Table Function

```
select *  
from table(information_schema.login_history_by_user())  
order by event_timestamp;
```

Results

~ Chart

EVENT_TIMESTAMP	EVENT_ID	EVENT_TYPE	USER_NAME	CLIENT_IP	REPORTED_CLIENT_T
2024-11-24 03:11:36.761 -0800	28796650505	LOGIN	LEARN	115.98.235.19	SNOWFLAKE_UI
2024-11-24 03:11:43.245 -0800	28796651013	LOGIN	LEARN	0.0.0.0	SNOWFLAKE_UI





External Function





External Function

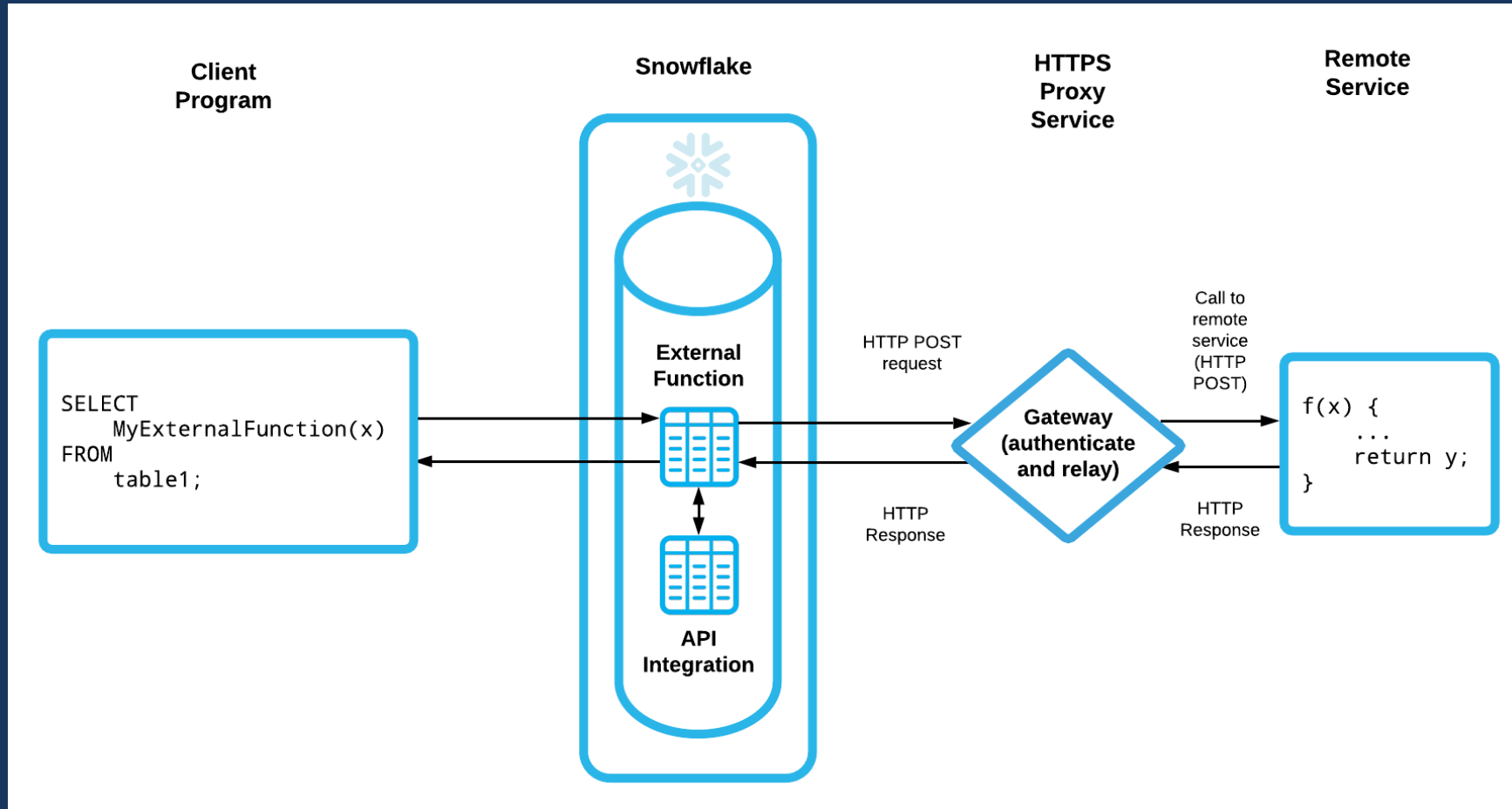
External functions in Snowflake are user-defined functions (UDFs) that call executable code stored and executed outside of Snowflake.

- An external function calls code that is executed outside Snowflake
- The remotely executed code is known as a **remote service**
- **Remote Service**, accepts JSON and return JSON
- Information sent to a remote service is usually relayed through a **proxy service**
- Snowflake stores security-related external function information in an **API integration**





External Function



```
select my_database.my_schema.my_external_function(col1) from table1;
```

Source : Snowflake





External Function

Usage and Limitation

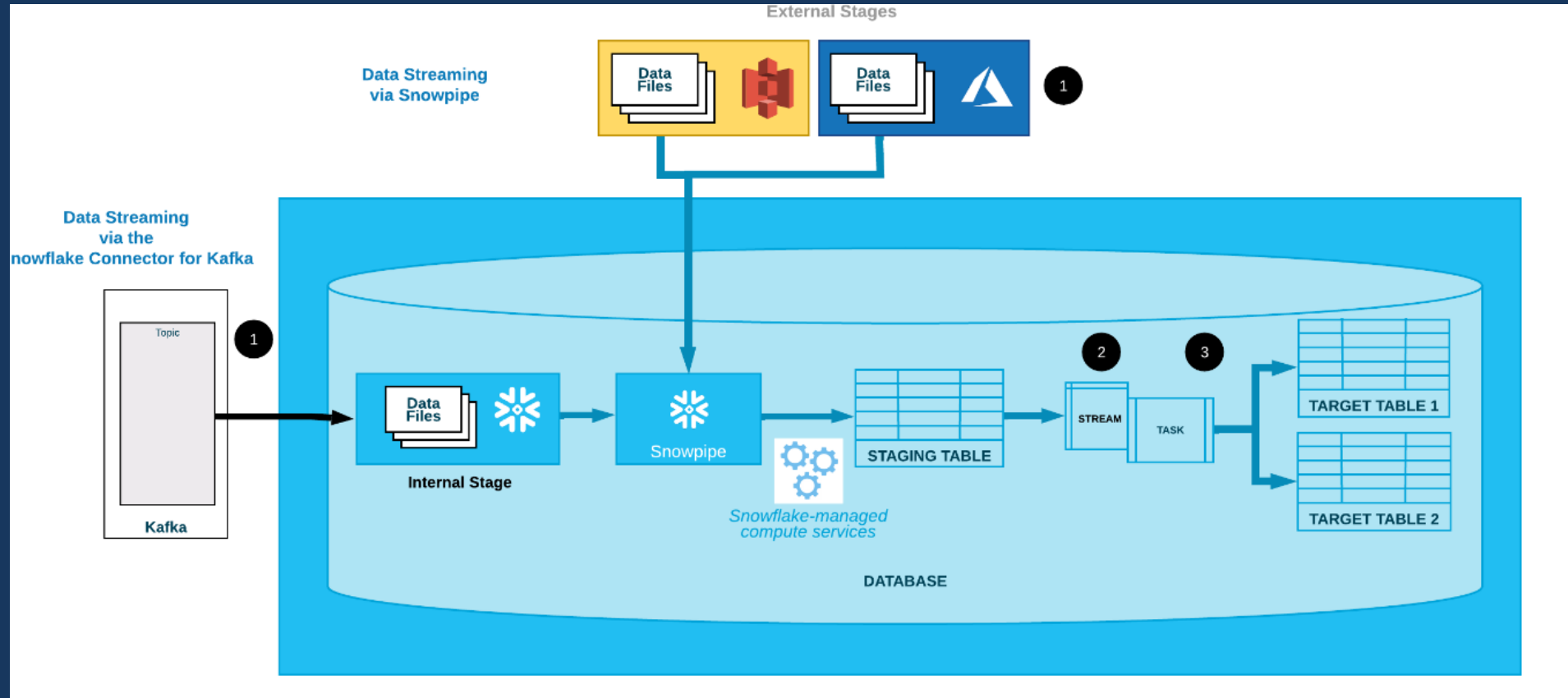
- Behaves like any other UDF
- Support Scalar functions, returning one row per input row
- Usage incurs for WH and Data Transfer and potential third-party charges, such as AWS PrivateLink fees
- Supporting Cloud Network : AWS, GCP, Azure



Stream



Continuous Loading



Source : Snowflake

Continuous Loading

Stream



A *stream* is a Snowflake object type that provides change data capture (CDC) capabilities to track the delta of changes in a table, including inserts and data manipulation language (DML) changes, so action can be taken using the changed data.



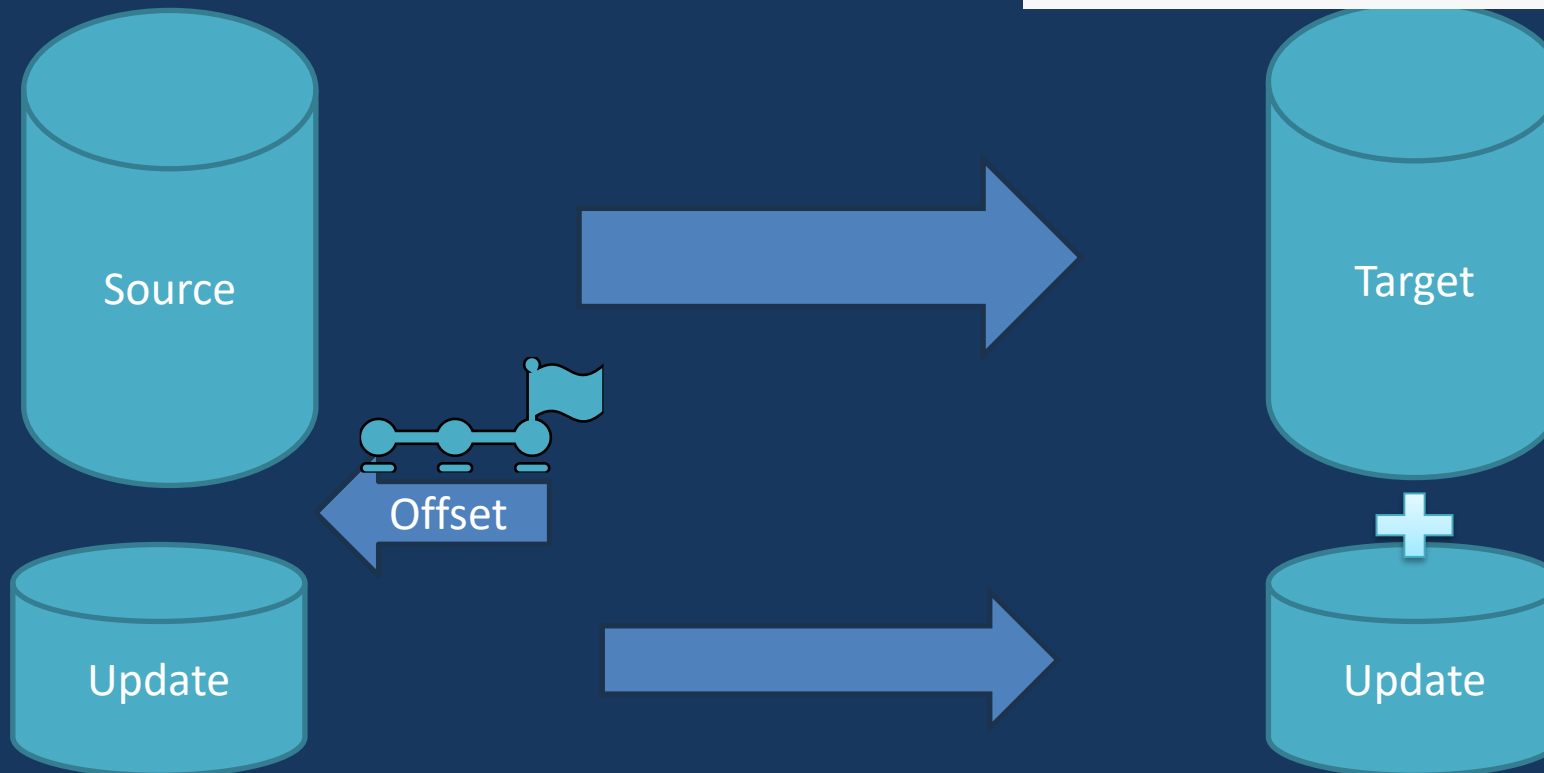
Stream



Continuous Loading

```
SHOW TABLES LIKE '%raw%';
```

Table Name	comment	cluster_by	rows	bytes	owner	retention_time	automatic_clustering	change_tracking
TABLE			5	2560	ACCOUNTADMIN	10	OFF	ON

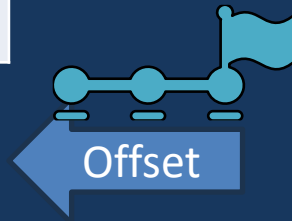


Stream



Continuous Loading

ID	NAME	AGE
1	John	25
2	Mary	30
3	Peter	40
4	Jane	35



ID	NAME	AGE	Change
5	Mike	45	New
1	John	27	update
4	Jane	35	delete



ID	NAME	AGE
1	John	25
2	Mary	30
3	Peter	40
4	Jane	35



ID	NAME	AGE
1	John	25 27
2	Mary	30
3	Peter	40
4	Jane	35
5	Mike	45



Stream



Continuous Loading

ID	NAME	AGE	Change
5	Mike	45	New
1	John	27	update
4	Jane	35	delete

- METADATA\$ACTION
- METADATA\$ISUPDATE
- METADATA\$ROW_ID

ID	NAME	AGE	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
5	Mike	45	INSERT	FALSE	6bbc69d84969b31a64c8f2ba2a8a61d8ae47c234
1	John	27	INSERT	TRUE	f98efdd8f203e090bb96a1bbcac1ffb5f3b1fd39
1	John	25	DELETE	TRUE	f98efdd8f203e090bb96a1bbcac1ffb5f3b1fd39
4	Jane	35	DELETE	FALSE	651a6fc5dc4bb247197e98aeaf876f7388d21b5d



Stream



Continuous Loading

Supported object types

- Standard tables
- Directory tables
- Dynamic tables
- Iceberg tables (with some limitation)
- External tables
- Views, Secure View

Types

- Standard
- Append-only
- Insert-only (External Table)



Stream



Continuous Loading

- Streams goes stale when source tables goes out of retention period
- Stale stream doesn't track changes
- Shall consume the stream within retention period

DATA_RETENTION_TIME_IN_DAYS	MAX_DATA_EXTENSION_TIME_IN_DAYS	Consume Streams in X Days
14	0	14
1	14	14
0	90	90

Check Stream's stale date in column STALE_AFTER



Stream



Continuous Loading

```
71 | SHOW PARAMETERS IN DATABASE;  
72  
73
```

Results				
	key	value	default	level
1	CATALOG			
2	CATALOG_SYNC			
3	DATA_RETENTION_TIME_IN_DAYS	1	1	ACCOUNT
4	DEFAULT_DDL_COLLATION			
5	ENABLE_CONSOLE_OUTPUT	false	false	
6	EVENT_TABLE	snowflake.telemetry.e	snowflake.telemetry.e	
7	EXTERNAL_VOLUME			
8	LOG_LEVEL	OFF	OFF	
9	MAX_DATA_EXTENSION_TIME_IN_DAYS	14	14	

```
SHOW STREAMS;  
  
-- Show specific streams
```

Results									
schema_name	owner	comment	table_name	source_type	base_tables	type	stale	mode	stale_after
LA_SCHEMA	ACCOUNTADMIN		LA_DB.LA_SCHEMA.RAW_DATA	Table	LA_DB.LA_SCHEMA.RAW_DATA	DELTA	false	DEFAULT	2024-11-15 06:12:07.601 -080



Stream



Continuous Loading

```
-- Create Stream on source table
```

```
CREATE OR REPLACE STREAM raw_data_stream ON TABLE raw_data;
```

```
--Append only Stream
```

```
CREATE OR REPLACE STREAM raw_data_stream2 ON TABLE raw_data append_only=true; -- For external table insert_only = true
```



Tasks



Continuous Loading

A task can execute any one of the following types of SQL code:

- Single SQL statement
- Call to a stored procedure
- Procedural logic using Snowflake Scripting





Continuous Loading

User Managed Tasks

```
CREATE TASK TASK_DEMO                                     ©Learn Anywhere
  WAREHOUSE = COMPUTE_WH
  SCHEDULE = '5 MINUTE'
AS
INSERT INTO mytable(ts) VALUES(CURRENT_TIMESTAMP);
```

Serverless Tasks

```
CREATE TASK TASK_DEMO                                     ©Learn Anywhere
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
  SCHEDULE = '5 MINUTE'
AS
INSERT INTO mytable(ts) VALUES(CURRENT_TIMESTAMP);
```





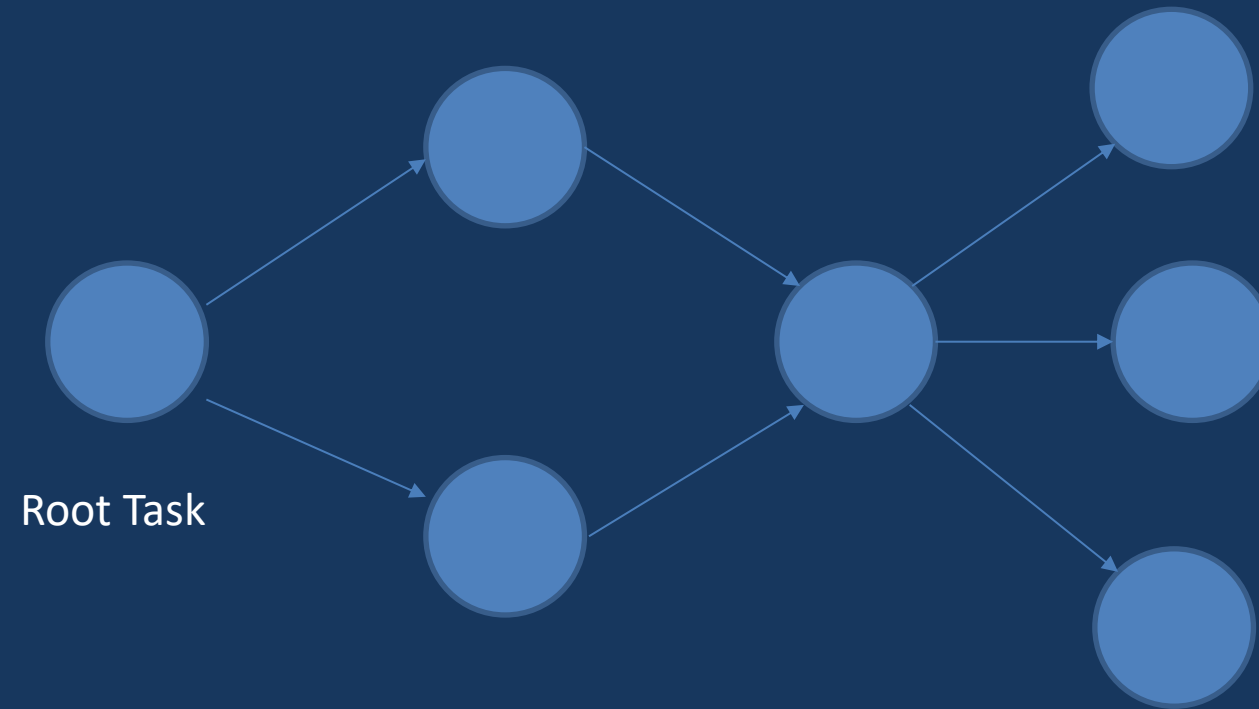
Continuous Loading

Category	Serverless Task	User-Managed Task
Utilization	When you cant fully utilize a WH	When you can fully utilize the WH
Workload	Predictable load	May have Spike
Scheduling	When Schedule is important	When Schedule is less important
Limitation	Max WH size is XXLARGE Java/Python UDF not supported Scala Stored Procedure not supported	





Continuous Loading

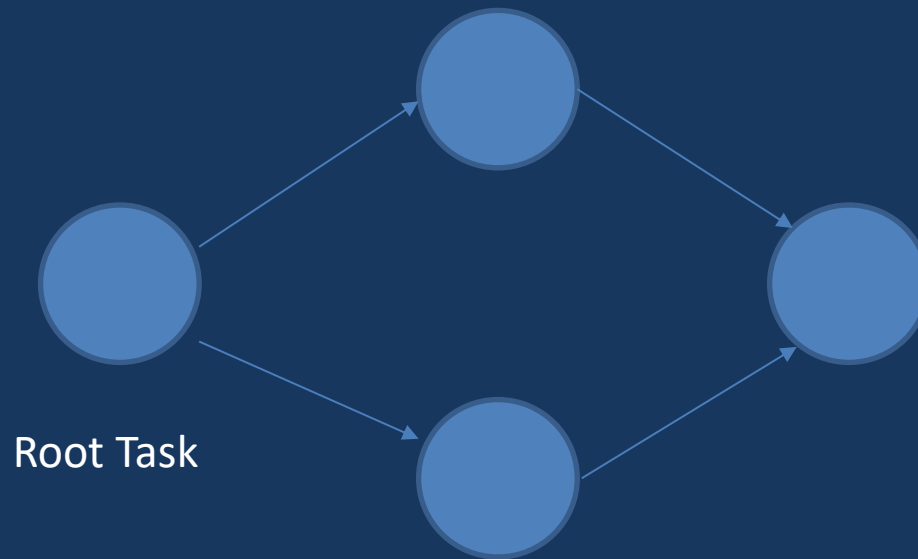


Directed Acyclic Graph (DAG)





Continuous Loading



Root Task

Directed Acyclic Graph (DAG)

A Directed Acyclic Graph (DAG) is a series of tasks composed of a single root task and additional tasks, organized by their dependencies

- One directional
- Can have multiple predecessor/Successor
- No looping allowed
- Max 1000 task
- Task can have Max 100 Predecessor tasks
- Task can have Max 100 Child tasks





Continuous Loading

CRON OPTION

```
CREATE TASK t1                                     ©Learn Anywhere
  SCHEDULE = 'USING CRON 0 9 * * * SUN America/Los_Angeles'
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
  AS <INSERT Command>;
```

Option	Description	Example
Minute	The minute of the hour.	0-59
Hour	The hour of the day.	0-23
Day of Month	The day of the month.	1-31
Month	The month of the year.	1-12
Day of Week	The day of the week.	0-7 (0=Sunday, 1=Monday, etc.)





Continuous Loading

```
-- Create TASK with User Managed Warehouse

CREATE OR REPLACE TASK TASK_DEMO_U
  WAREHOUSE = COMPUTE_WH
  SCHEDULE = '1 MINUTE'
AS
INSERT INTO my_table (id, name, email) VALUES
  (100, 'User Managed', 'john@example.com');

-- Create TASK for Serverless

CREATE OR REPLACE TASK TASK_DEMO_S
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL' --By Default Medium
  SCHEDULE = '1 MINUTE'
AS
INSERT INTO my_table (id, name, email) VALUES
  (100, 'Serverless', 'john@example.com');
```

```
-- Need to Resume the tasks

ALTER TASK TASK_DEMO_U RESUME;
ALTER TASK TASK_DEMO_S RESUME;
```





Continuous Loading

```
--Check history of all tasks
```

```
SELECT * FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY());
```

```
--One time Execution
```

```
EXECUTE TASK TASK_DEMO_S;
```

```
-- Task and Stream
```

```
CREATE OR REPLACE TASK TASK_STREAM
```

```
  USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
```

```
  SCHEDULE = '1 MINUTE'
```

```
  WHEN SYSTEM$STREAM_HAS_DATA('raw_data_stream')
```

```
  AS INSERT INTO processed_data(id,name,age) SELECT id, name, age FROM raw_data_stream;
```





Semi- Structured Data





Semi- Structured Data

“Semi-structured data is data that does not conform to the standards of traditional structured data, but contains tags (labels) or other types of mark-up that identify individual, distinct entities within the data.”

- Doesn't have fixed schema
- Contains N-level hierarchies of nested information

```
{
  "customer": [
    {
      "address": "San Francisco, CA",
      "name": "Joyce Ridgely",
      "phone": "16504378889"
    }
  ],
  "date": "2017-04-28",
  "dealership": "Valley View Auto Sales",
  "salesperson": {
    "id": "55",
    "name": "Frank Beasley"
  }
}
```




Semi- Structured Data

- Snowflake provides native data types (ARRAY, OBJECT, and VARIANT)
- Snowflake supports querying the data
- Supported Data formats (For Load)
 - JSON
 - Avro
 - ORC
 - Parquet
 - XML
- Supported Data formats (For Unload)
 - JSON
 - Parquet





Semi- Structured Data

Data Type	Description	Example
ARRAY (Max 16MB)	A data type used to store an ordered list of values of the same data type.	['apple', 'banana', 'orange']





Semi- Structured Data

Data Type	Description	Example
ARRAY (Max 16MB)	A data type used to store an ordered list of values of the same data type.	['apple', 'banana', 'orange']
OBJECT(Max 16MB)	A data type used to store an unordered collection of key-value pairs, where each key is a string and each value can be a different data type.	{'name': 'John', 'age': 35, 'isMarried': true}





Semi- Structured Data

Data Type	Description	Example
ARRAY (Max 16MB)	A data type used to store an ordered list of values of the same data type.	['apple', 'banana', 'orange']
OBJECT(Max 16MB)	A data type used to store an unordered collection of key-value pairs, where each key is a string and each value can be a different data type.	{'name': 'John', 'age': 35, 'isMarried': true}
VARIANT(Max 16MB)	A data type used to store semi-structured data, including JSON documents, XML documents, and other complex data types. VARIANT can store any type of data and allows you to query and manipulate it as needed.	{ 'name': 'John', 'age': 35, 'isMarried': true, 'hobbies': ['reading', 'hiking', 'cooking'], 'address': { 'street': '123 Main Street', 'city': 'Anytown', 'state': 'CA', 'zip': '12345'} }





Semi- Structured Data

```
{
  "customer": [
    {
      "address": "San Francisco, CA",
      "name": "Joyce Ridgely",
      "phone": "16504378889"
    }
  ],
  "date": "2017-04-28",
  "dealership": "Valley View Auto Sales",
  "salesperson": {
    "id": "55",
    "name": "Frank Beasley"
  }
}
```

```
54 SELECT src
55 FROM raw_source;
56
57
```

Results Chart

	SRC
1	{ "customer": [{ "address": "San Francisco, CA", "name": "Joyce Ridgely", "phone": "1

```
[ ] SRC
{
  "customer": [
    {
      "address": "San Francisco, CA",
      "name": "Joyce Ridgely",
      "phone": "16504378889"
    }
  ],
  "date": "2017-04-28",
  "dealership": "Valley View Auto Sales",
  "salesperson": {
    "id": "55",
    "name": "Frank Beasley"
  }
}
```

```
-- Create table with Variant option
```

```
CREATE OR REPLACE TABLE raw_source (SRC VARIANT);
```





Semi- Structured Data

```
insert into raw_source
select
PARSE_JSON ('{
  "customer": [
    {
      "address": "New York, NY",
      "name": "Bradley Greenbloom",
      "phone": "12127593751"
    }
  ],
  "date": "2017-04-28",
  "dealership": "Tindel Toyota",
  "salesperson": {
    "id": "274",
    "name": "Greg Northrup"
  }
}')
```

```
SELECT src
FROM raw_source;
```

Results 

SRC

```
{ "customer": [ { "address": "San Francisco, CA", "name": "Joyce Ridgely", "phone": "16504
{ "customer": [ { "address": "New York, NY", "name": "Bradley Greenbloom", "phone": "121
```

[] SRC

```
{
  "customer": [
    {
      "address": "New York, NY",
      "name": "Bradley Greenbloom",
      "phone": "12127593751"
    }
  ],
  "date": "2017-04-28",
  "dealership": "Tindel Toyota",
  "salesperson": {
    "id": "274",
    "name": "Greg Northrup"
  }
}
```





Semi- Structured Data

```
{
  "customer": [
    {
      "address": "San Francisco, CA",
      "name": "Joyce Ridgely",
      "phone": "16504378889"
    }
  ],
  "date": "2017-04-28",
  "dealership": "Valley View Auto Sales",
  "salesperson": {
    "id": "55",
    "name": "Frank Beasley"
  }
}
```

```
38 --Bracket Notation
39
40 SELECT src ['salesperson'] ['name']
41 FROM raw_source;
42
43 -- Dot notation
44
45 SELECT SRC:salesperson.name
46 FROM raw_source;
47
```

→ Results ↗ Chart

SRC ['SALESPERSON'] ['NAME']

"Frank Beasley"





Semi- Structured Data

No Database selected Settings

```
1 SELECT * FROM TABLE(FLATTEN(input => parse_json('[1, ,2,3]'))) f;  
2  
3
```

Results Chart

	SEQ	KEY	PATH	INDEX	VALUE	THIS
1	1	null	[0]	0	1	[1, undefined, 2, 3]
2	1	null	[2]	2	2	[1, undefined, 2, 3]
3	1	null	[3]	3	3	[1, undefined, 2, 3]

For simple/static cases we use FLATTEN, for Dynamic/Complex structure we use LATERAL FLATTEN





Semi- Structured Data

```
[
  {
    "Department": "Food and Beverage",
    "Zone": [
      {
        "SubZone": "North",
        "Division": [
          {
            "Store": "Food",
            "Orders": [
              {
                "SR": "SR 1",
                "Sales": 5700
              },
              {
                "SR": "SR 2",
                "Sales": 2000
              }
            ]
          }
        ]
      },
      {
        "Store": "Beverages",
        "Orders": [
          {
            "SR": "SR 3",
            "Sales": 1500
          },
          {
            "SR": "SR 4",
            "Sales": 2345
          }
        ]
      }
    ]
  },
  {
    "Department": "Clothing and Accessories",
    "Zone": [
      {
        "SubZone": "South",
        "Division": [
          {
            "Store": "Clothing",
            "Orders": [
              {
                "SR": "SR 1",
                "Sales": 2100
              },
              {
                "SR": "SR 2",
                "Sales": 867
              }
            ]
          },
          {
            "Store": "Accessories",
            "Orders": [
              {
                "SR": "SR 1",
                "Sales": 3200
              },
              {
                "SR": "SR 2",
                "Sales": 8560
              }
            ]
          }
        ]
      }
    ]
  }
]
```

Department	Zone		
Food and Beverage	SubZone	Division	
	North	Store	Orders
		Food	SR Sales
			SR 1 5700
		Beverages	SR 2 2000
			SR Sales
Clothing and Accessories	South	Clothing	SR Sales
			SR 1 2100
		Accessories	SR 2 867
			SR Sales
			SR 1 3200
			SR 2 8560



Semi- Structured Data

Department	Zone			
Food and Beverage	SubZone	Division		
	North	Store	Orders	
		Food	SR	Sales
			SR 1	5700
		SR 2	2000	
		Beverages	SR	Sales
			SR 3	1500
SR 4	2345			
Clothing and Accessories	SubZone	Division		
	South	Store	Orders	
		Clothing	SR	Sales
			SR 1	2100
		SR 2	867	
		Accessories	SR	Sales
			SR 1	3200
SR 2	8560			

```
58
59 SELECT src:Department::string,
60       Zone.*
61 FROM   raw_source,
62        LATERAL FLATTEN(INPUT => SRC:Zone) as Zone;
63
```

Results Chart

	SRC:DEPARTMENT::STRING	SEQ	KEY	PATH	INDEX	VALUE	THIS
1	Food and Beverage	1	null	[0]	0	{ "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 5700 }, { "SR": "SR 2", "Sales": 2000 }] }, { "Division": [{ "Orders": [{ "SR": "SR 3", "Sales": 1500 }, { "SR": "SR 4", "Sales": 2345 }] }, { "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 2100 }, { "SR": "SR 2", "Sales": 867 }] }, { "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 3200 }, { "SR": "SR 2", "Sales": 8560 }] }] }	[{ "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 5700 }, { "SR": "SR 2", "Sales": 2000 }] }, { "Division": [{ "Orders": [{ "SR": "SR 3", "Sales": 1500 }, { "SR": "SR 4", "Sales": 2345 }] }, { "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 2100 }, { "SR": "SR 2", "Sales": 867 }] }, { "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 3200 }, { "SR": "SR 2", "Sales": 8560 }] }] }
2	Clothing and Accessories	2	null	[0]	0	{ "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 2100 }, { "SR": "SR 2", "Sales": 867 }] }, { "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 3200 }, { "SR": "SR 2", "Sales": 8560 }] }] }	[{ "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 2100 }, { "SR": "SR 2", "Sales": 867 }] }, { "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 3200 }, { "SR": "SR 2", "Sales": 8560 }] }] }

SEQ A unique sequence number associated with the input record; the sequence is not guaranteed to be gap-free or ordered in any particular way.

KEY For maps or objects, this column contains the key to the exploded value.

PATH The path to the element within a data structure which needs to be flattened.

INDEX The index of the element, if it is an array; otherwise NULL.

VALUE The value of the element of the flattened array/object.

THIS The element being flattened (useful in recursive flattening).





Semi- Structured Data

Department	Zone			
Food and Beverage	SubZone	Division		
	North	Store	Orders	
		Food	SR	Sales
			SR 1	5700
		SR 2 2000		
		Beverages	SR	Sales
Clothing and Accessories	South	Clothing	SR 3	1500
			SR 4	2345
		Accessories	SR	Sales
			SR 1	2100
		SR 2 867		
		Accessories	SR 1	3200
			SR 2	8560

```
63
64
65 create or replace table dump_json as
66 SELECT src:Department::string as dep,
67        Zone.value:SubZone :: string as subzone_col,
68        div.value:Store :: string as store_col,
69        ord.value:SR :: string as sr_col,
70        ord.value:Sales :: string as sales_col
71
72 FROM raw_source,
73      LATERAL FLATTEN(INPUT => SRC:Zone) as Zone,
74      LATERAL FLATTEN(INPUT => zone.value:Division) as div,
75      LATERAL FLATTEN(INPUT => div.value:Orders) as ord;
76
```

Results Chart

DEP	SUBZONE_COL	STORE_COL	SR_COL	SALES_COL
Food and Beverage	North	Food	SR 1	5700
Food and Beverage	North	Food	SR 2	2000
Food and Beverage	North	Beverages	SR 3	1500
Food and Beverage	North	Beverages	SR 4	2345
Clothing and Accessories	South	Clothing	SR 1	2100
Clothing and Accessories	South	Clothing	SR 2	867
Clothing and Accessories	South	Accessories	SR 1	3200





Semi- Structured Data

Department	Zone			
Food and Beverage	SubZone	Division		
	North	Store	Orders	
		Food	SR	Sales
			SR 1	5700
		Beverages	SR 2	2000
			SR 3	1500
Clothing and Accessories	South		SR	Sales
			SR 4	2345
		Store	Orders	
		Clothing	SR	Sales
			SR 1	2100
		Accessories	SR 2	867
			SR 1	3200
			SR 2	8560

```
55
56 SELECT src, src:Zone [0].Division [0].Store
57 FROM raw_source;
58
59
```

Results Chart

	SRC	SRC:ZONE [0].DIVISION [0].STORE
1	{ "Department": "Food and Beverage", "Zone": [{ "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 5700	"Food"
2	{ "Department": "Clothing and Accessories", "Zone": [{ "Division": [{ "Orders": [{ "SR": "SR 1", "Sales": 2100	"Clothing"





Unstructured Data

“Unstructured data is information that does not fit into a predefined data model or schema”.



VCF (genomics), KDF (semiconductors), or HDF5 (aeronautics)





Unstructured Data

- Securely access data files located in cloud storage
- Share file access URLs with collaborators and partners
- Load file access URLs and other file metadata into Snowflake tables
- Process Unstructured data





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL .	User with access.	Anyone with URL





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL .	User with access.	Anyone with URL
Expiration	query results cache = 24 hours	Permanent	Can be configured with <i>expiration_time</i>





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL .	User with access.	Anyone with URL
Expiration	query results cache = 24 hours	Permanent	Can be configured with <i>expiration_time</i>
How-to-Fun	BUILD_SCOPED_FILE_URL	BUILD_STAGE_FILE_URL	GET_PREIGNED_URL





Types of URLs to Access File

	Scoped URL	File URL	Pre-signed URL
Definition	Encoded URL that permits temporary access to a staged file without granting privileges to the stage	URL that identifies the database, schema, stage, and file path to a set of files. A role that has sufficient privileges on the stage can access the files.	Simple HTTPS URL used to access a file via a web browser. A file is temporarily accessible to users via this URL using a pre-signed access token.
Access Control	Only the user who generates a scoped URL .	User with access.	Anyone with URL
Expiration	query results cache = 24 hours	Permanent	Can be configured with <i>expiration_time</i>
How-to-Fun	BUILD_SCOPED_FILE_URL	BUILD_STAGE_FILE_URL	GET_PREIGNED_URL
Syntax	BUILD_SCOPED_FILE_URL(@<stage_name> , '<relative_file_path>')	BUILD_STAGE_FILE_URL(@<stage_name> , '<relative_file_path>')	GET_PREIGNED_URL(@<stage_name> , '<relative_file_path>' , [<expiration_time>])





SQL Functions

SQL Function	Description
GET_STAGE_LOCATION	Returns the URL for an external or internal named stage using the stage name as the input.
GET_RELATIVE_PATH	Extracts the path of a staged file relative to its location in the stage using the stage name and absolute file path in cloud storage as inputs.
GET_ABSOLUTE_PATH	Returns the absolute path of a staged file using the stage name and path of the file relative to its location in the stage as inputs.
GET_PREIGNED_URL	Generates the pre-signed URL to a staged file using the stage name and relative file path as inputs. Access files in an external stage using the function.
BUILD_SCOPED_FILE_URL	Generates a scoped Snowflake-hosted URL to a staged file using the stage name and relative file path as inputs.
BUILD_STAGE_FILE_URL	Generates a Snowflake-hosted file URL to a staged file using the stage name and relative file path as inputs.

`GET_PREIGNED_URL` and `BUILD_SCOPED_FILE_URL` are non-deterministic functions; the others are deterministic.

Source : Snowflake







Directory

- Stores file level meta data about files in stage
- Implicit object layered on stage
- Can be enabled for external/internal stage
- Stores URL by BUILD_STAGE_FILE_URL
- Can be enabled during stage creation or later (by alter)
- Data can be refreshed manually or automatically (only for cloud i.e external stages)
- No explicit privilege required to access





Directory

```
-- Create a stage with encryption

CREATE
OR REPLACE STAGE <Stage_name>
  ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE') --Server side encryption
  DIRECTORY=(ENABLE=TRUE);

-- Alter to enable Directory in existing

ALTER STAGE internal_UN_stage
SET DIRECTORY=(ENABLE=TRUE);

-- Check for data in DIRECTORY

SELECT * FROM DIRECTORY(@internal_UN_stage);

-- Manual Refresh

ALTER STAGE internal_UN_stage REFRESH;

-- For External Stage Auto refresh

directoryTableParams (for Amazon S3) ::=
  [ DIRECTORY = ( ENABLE = { TRUE | FALSE }
                  [ REFRESH_ON_CREATE = { TRUE | FALSE } ]
                  [ AUTO_REFRESH = { TRUE | FALSE } ] ) ]
```



Continuous Data Protection Lifecycle



Perform All Standard Operations

AT|BEFORE

TIMESTAMP => <timestamp>

OFFSET => <time_difference>

STATEMENT => <id>

UNDROP

No user operations allowed.
(Recoverable by Snowflake Staff)

 Permanent : 7 Days

Period Retention is configurable at account level with

MIN_DATA_RETENTION_TIME_IN_DAYS



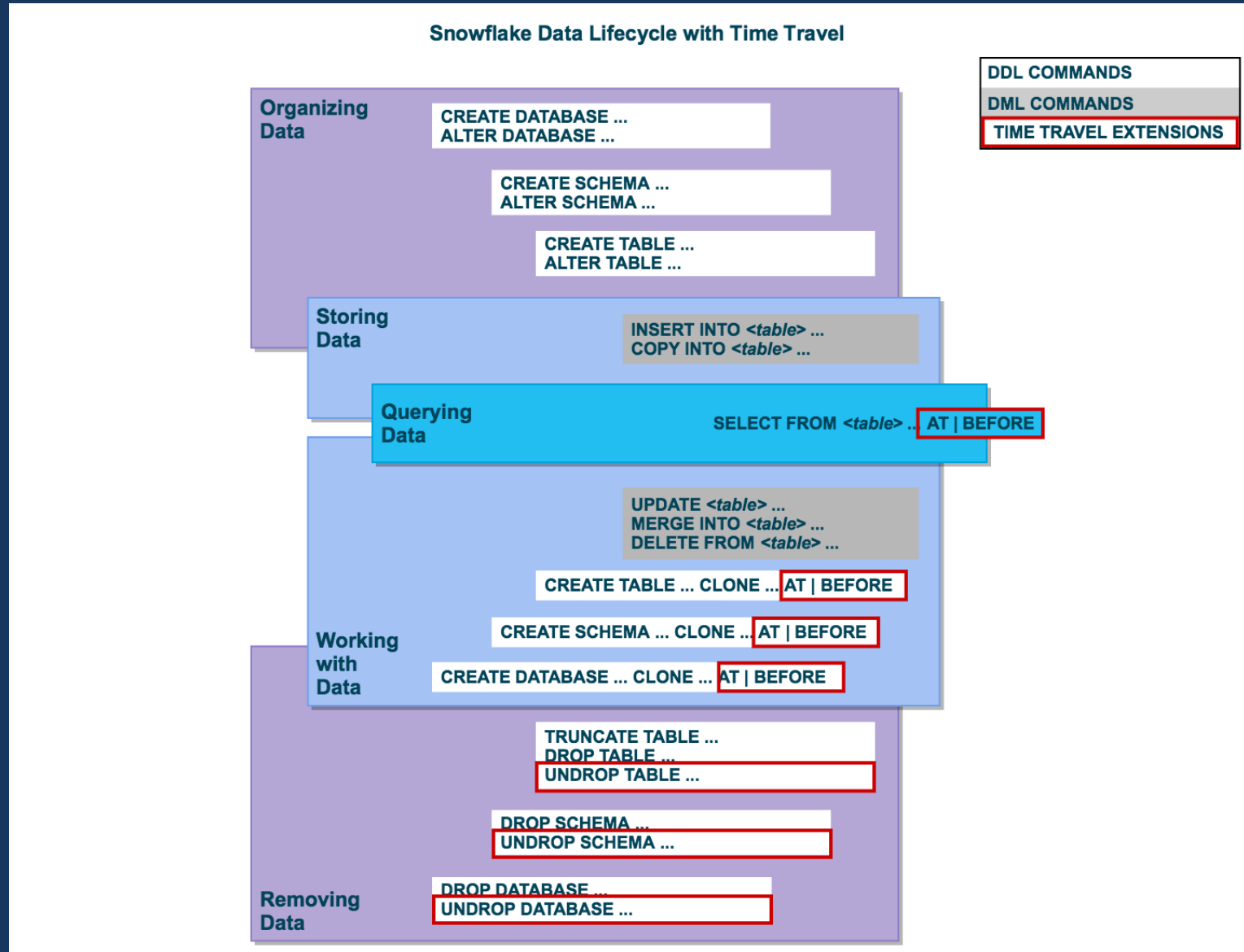
0-1 Standard Edition (Default is 1)



0-90 Enterprise+ (Permanent Objects)



Snowflake Data Lifecycle with Time Travel



Source : Snowflake

Snowflake Data Lifecycle with Time Travel

```
-- View the state of the CUSTOMER table as it existed before below Query executed  
SELECT * FROM CUSTOMER BEFORE(STATEMENT => '01ad0729-3200-c333-0000-0004ade6a125');
```

```
-- This command selects all rows from the CUSTOMER table as it existed 150 seconds ago  
SELECT * FROM CUSTOMER BEFORE(OFFSET => -150);
```

```
-- This command selects all rows from the CUSTOMER table that were valid AT a specific timestamp  
SELECT * FROM CUSTOMER AT(TIMESTAMP => '2023-06-17 12:56:43.446 +0000'::timestamp);
```

```
-- This command drops the CUSTOMER table, if it exists  
DROP TABLE CUSTOMER;
```

```
-- This command undrops the CUSTOMER table, if it was previously dropped  
UNDROP TABLE CUSTOMER;
```



Snowflake Data Lifecycle with Time Travel

```
--Alter the account data retention with ACCOUNTADMIN role, can be overridden  
  
ALTER ACCOUNT SET MIN_DATA_RETENTION_TIME_IN_DAYS = 10;  
  
-- You can also use DATA_RETENTION_TIME_IN_DAYS, Final days will be MAX(DATA_RETENTION_TIME_IN_DAYS, MIN_DATA_RETENTION_TIME_IN_DAYS)  
  
ALTER ACCOUNT SET DATA_RETENTION_TIME_IN_DAYS = 1;
```



Snowflake Data Lifecycle with Time Travel

--You can also do time travel for COPY, Create Schema, Create database

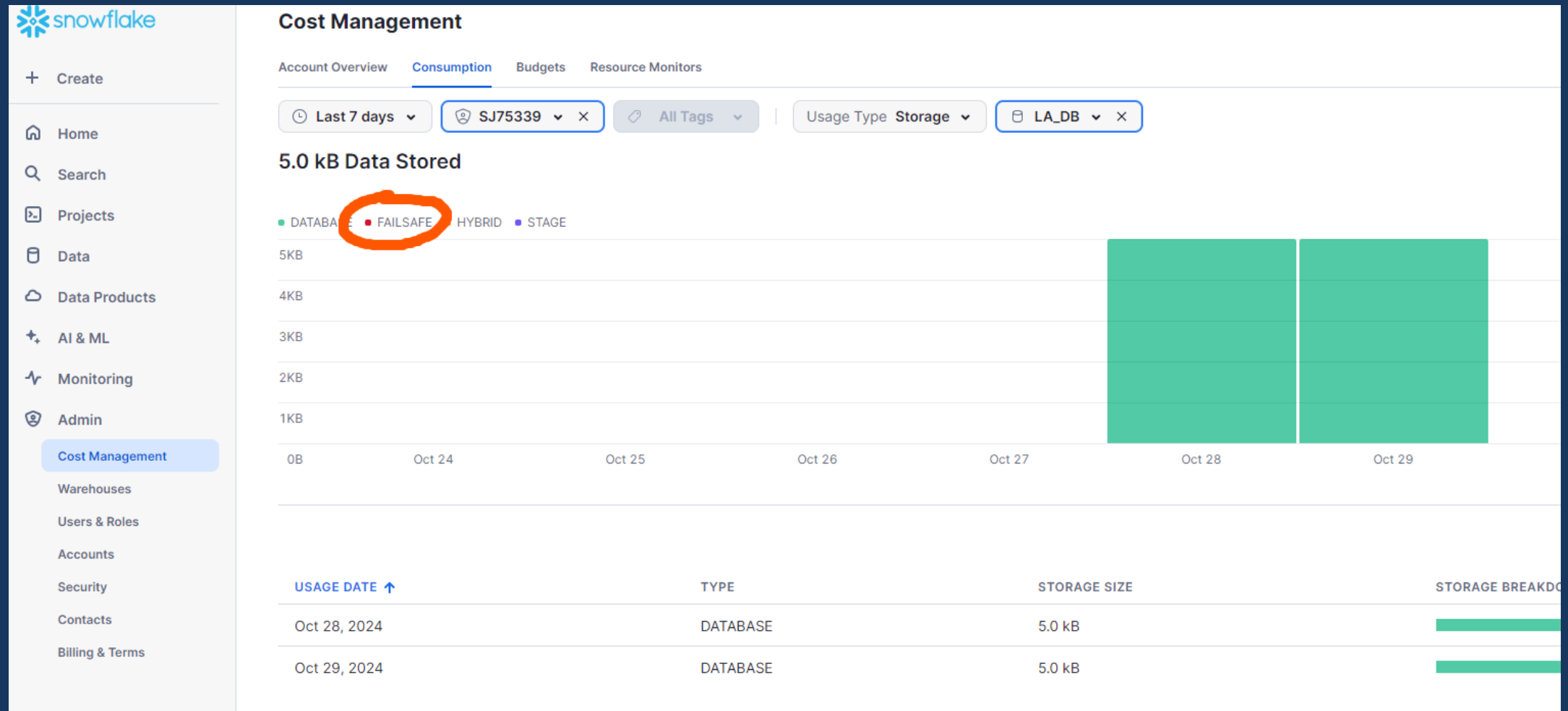
```
CREATE TABLE copy_cust CLONE CUSTOMER
  AT(TIMESTAMP => '2023-06-17 12:56:43.446 +0000'::timestamp);

CREATE SCHEMA restored_schema CLONE LA_SCHEMA AT(OFFSET => -3600);

CREATE DATABASE restored_db CLONE LA_DB
  BEFORE(STATEMENT => '8e5d0ca9-005e-44e6-b858-a8f5b37c5726');
```



Snowflake Data Lifecycle with Time Travel





Data Encryption





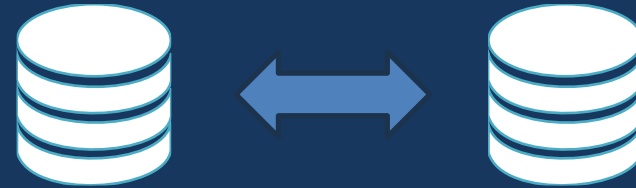
Data Encryption

AT REST



Encrypted with AES 256 Bit

AT TRANSIT



TLS 1.2

- Available by default
- Available for all editions





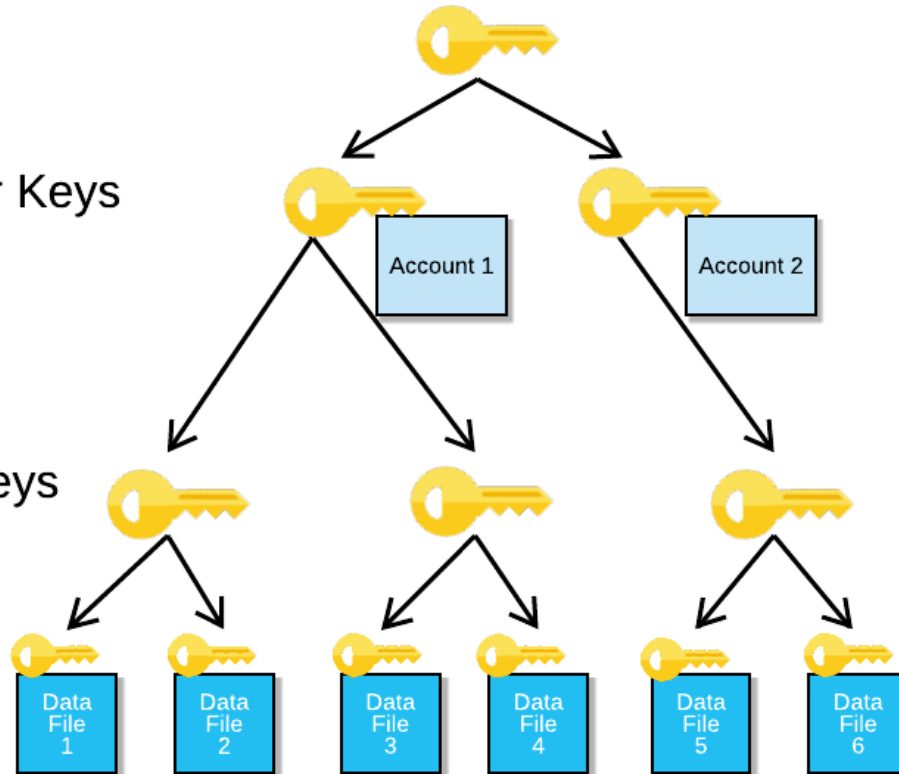
Data Encryption

Root Key

Account Master Keys

Table Master Keys

File Keys



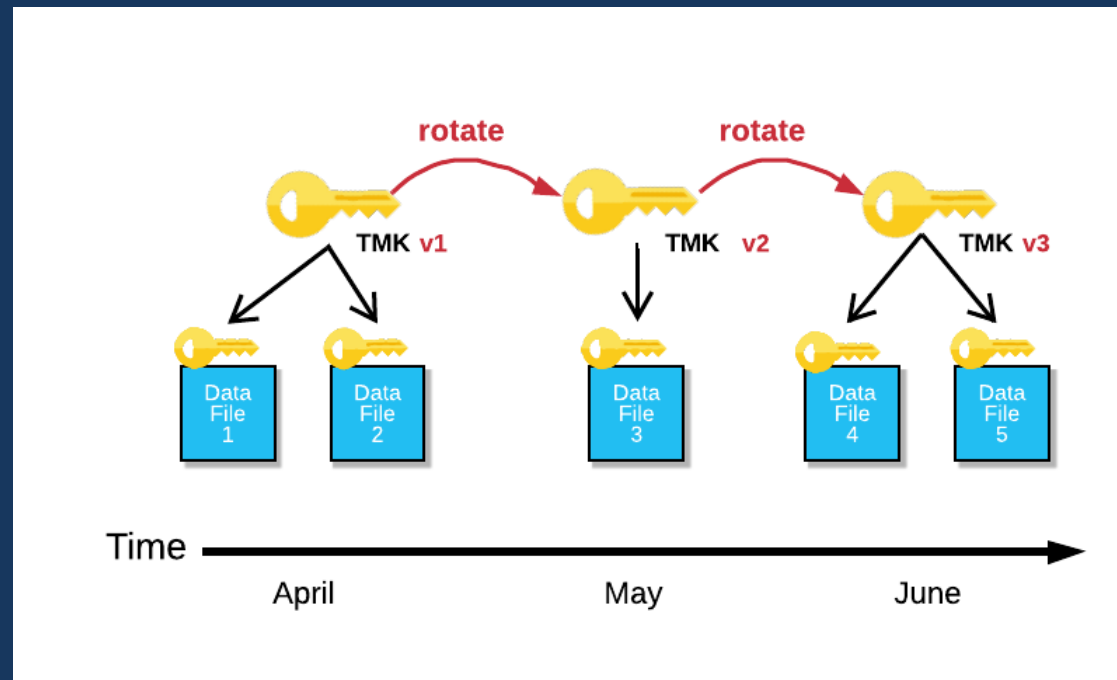
Source: Snowflake



Data Encryption

Periodic Key Rotation

- Snowflake auto-rotates the key every 30 days
- Post 30 days, Active Keys get retired



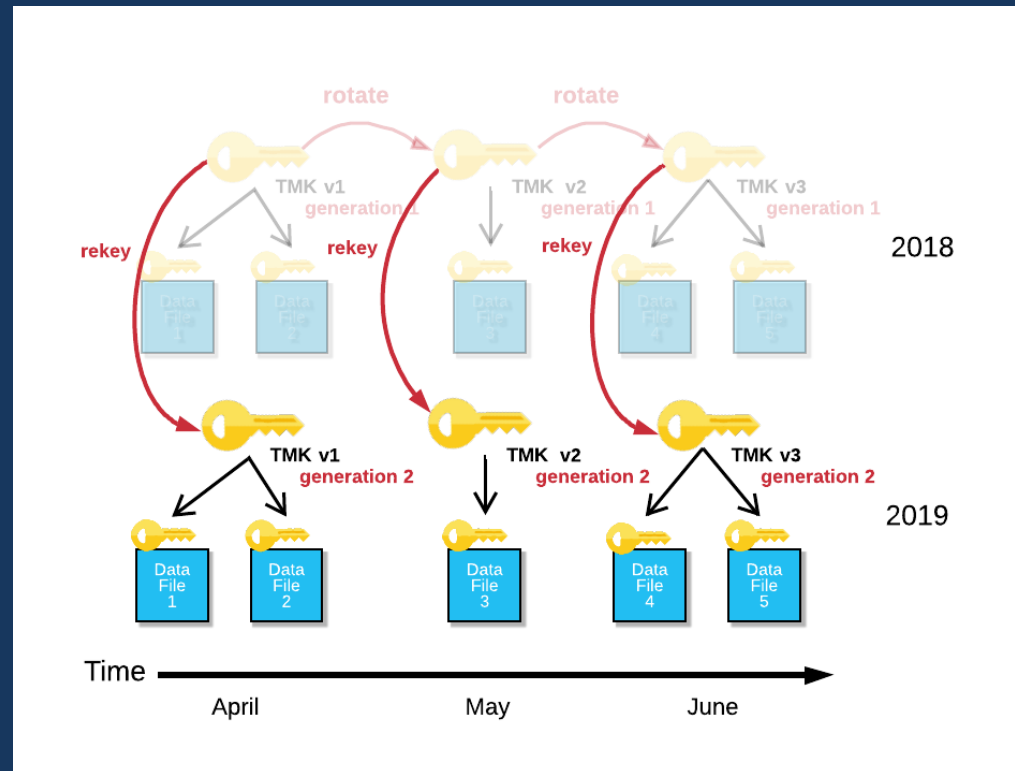
Source: Snowflake



Data Encryption

Periodic Rekeying (Enterprise Edition on Higher)

- Manual Enablement
- Destroys retired key post one year
- Ensures that data is encrypted with latest tech



Source: Snowflake





Data Encryption

Tri-Secret Secure (Business Critical Edition on Higher)

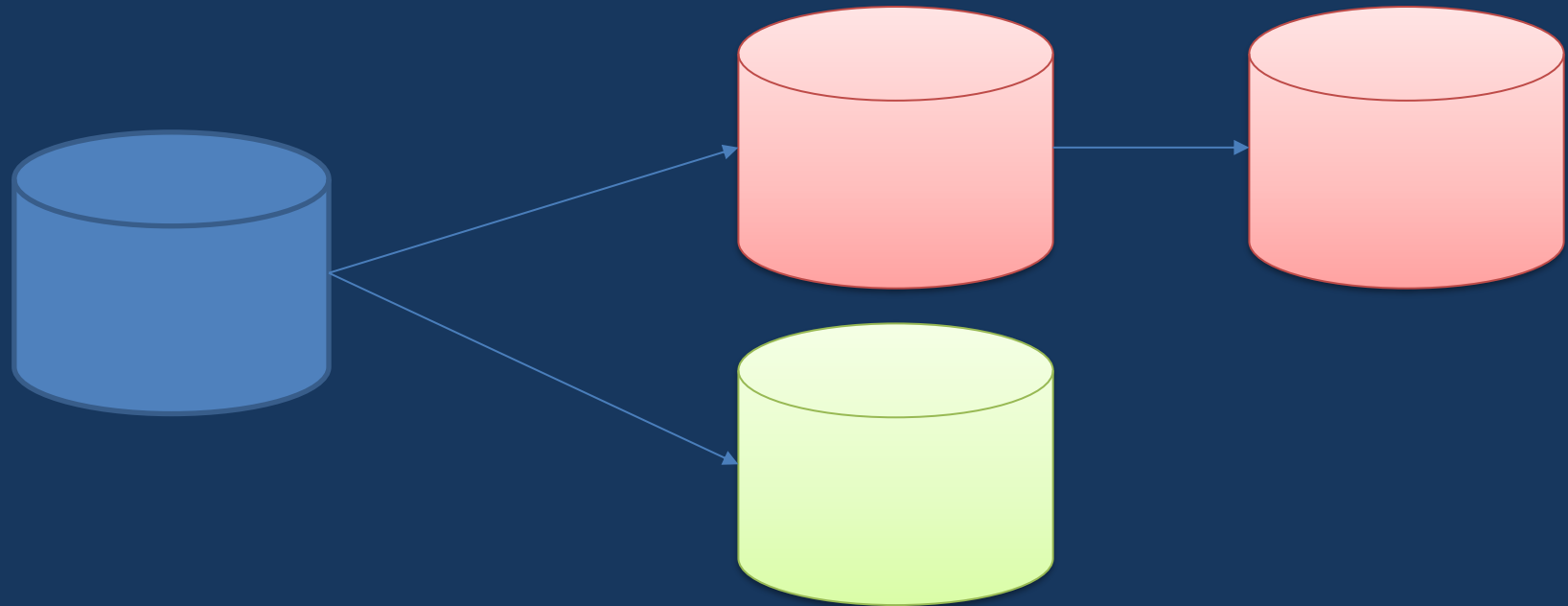
- Combination of CMK and SMK
- Composite Master Key is used
- If CMK is revoked, You cant decrypt data in Snowflake
- Provides 3 levels of data protection: encryption, authentication, and control.





Cloning

Cloning in Snowflake refers to the process of creating an exact copy of a database, schema, or table. This feature can be used to create a backup of existing data, test new data structures, or create a new environment for development or testing purposes.





Cloning

- Database
- Schema
 - External table
 - Internal named Stage
- Tables
- File Format
- Sequence
- Tags
- Clustering Key (Suspended)
- Streams (Unconsumed records remain)
- Steams (as part of Schema/DB, Emptied)
- External Stage
- Table Stage (empty)
- Tasks (By default suspended)
- Pipe (referring External stage)
- View (Only with DB/Schema)

Users

Roles

Grants

Virtual Warehouses

Resource monitors

Storage integrations

Policy

View (Individually)

Internal named staged

Temporary Table





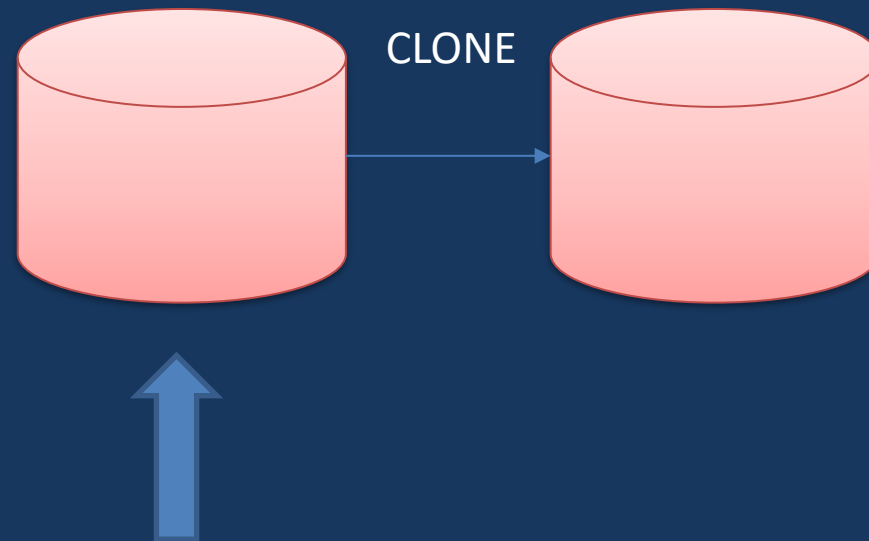
Cloning

Object	Privilege
Tables	SELECT
Pipes, Streams, Tasks	OWNERSHIP
Other objects	USAGE





Cloning



- Takes the start time of job for cloning
- In-between updates wont be cloned
- Maintain at least
`DATA_RETENTION_TIME_IN_DAYS = 1`
- Job may fail if Data Retention is Zero

Update Records





Cloning (with Time Travel)

Only available for Database, Schema, Table & Streams

Cloning fails if

- Source in past doesn't exist
- Pipe object with `AUTO_INGEST = TRUE` recreated

* By default current timestamp is taken





Cloning

```
-- Create a clone  
  
CREATE OR REPLACE TABLE clone_data CLONE sample_data;  
  
-- Create clone from clone  
  
CREATE OR REPLACE TABLE clone_data2 CLONE clone_data;
```

```
CREATE OR REPLACE TABLE orders_clone_restore CLONE sample_data BEFORE (TIMESTAMP => '2024-07-29 05:30:44.611 +0000'::timestamp);  
  
CREATE OR REPLACE TABLE orders_clone_restore CLONE sample_data BEFORE (STATEMENT => '01adebc5-3200-d22f-0005-3db60003005e');  
  
-- This command selects all rows from the CUSTOMER table as it existed 3 hours ago  
CREATE OR REPLACE TABLE orders_clone_restore CLONE sample_data AT(OFFSET => -60*3);
```

```
-- Clone the Schema  
  
CREATE OR REPLACE SCHEMA LA_SCHEMA2 CLONE LA_SCHEMA;
```



Replication



AWS : US West (Oregon)



Replication

PRIMARY DB



AWS : US West (Oregon)



SECONDARY DB



AZURE : UAE North (Dubai)

External tables
Event/Hybrid tables
Apache Iceberg tables
Stages
Pipes
DBs created from SHARE



Replication

PRIMARY DB



AWS : US West (Oregon)



SECONDARY DB



AZURE : UAE North (Dubai)



SECONDARY DB



GCP : EU WEST2 (London)



Replication

- Only for DB objects
- Privileges granted on DB are not replicated
- Error when replicating to lower edition (Can overcome)
- Can replicate to multiple accounts
- Account should be under same Org
- Data transfer and Data Compute cost



Replication

Feature	Standard	Enterprise	Business Critical	VPS
Database replication	✓	✓	✓	✓
Share replication	✓	✓	✓	✓
Replication Group	✓	✓	✓	✓
Account object (other than database and share) replication			✓	✓
Failover Group			✓	✓
Data protected with Tri-Secret Secure			✓	✓

Source: Snowflake



Secure Data Sharing





Secure Data Sharing



Data Provider



Data Consumer

- Database
- Tables
- External Tables
- Dynamic Tables
- Apache Iceberg Table
- Secure Views
- Secure Materialized View
- Secure UDFs
- Models without code (eg. CORTEX_FINETUNED)





Secure Data Sharing



Data Provider



Data Consumer

- Create a SHARE
- GRANT Access to the SHARE
- Share with Consumer Account
- Can add new object to share
- Can be revoked anytime
- Pay of the Storage

- Pays for the compute
- A read only DB is created
- Treated just like any object





Secure Data Sharing



Data Provider



Readers Account

- Create a Share
- Add Object to share
- GRANT Access to the DB
- Share with Consumer Account
- Can add new object to share
- Can be revoked anytime
- Pays all cost

- Can Run Queries
- No other access
- Doesn't pay





Secure Data Sharing

Considerations

- Shared databases are read-only. Users in a consumer account can view/query data, but cannot insert or update data, or create any objects in the database
- No limit on Number of Shares or Number of account it is shared with
- Can not create a clone of a shared database or any schemas/tables in the database
- Not supported, Time Travel for a shared database or any schemas/tables in the database
- Not supported, Editing the comments for a shared database.
- Shared databases and all the objects in the database cannot be re-shared with other accounts
- Shared databases cannot be replicated
- Can consume as many as shares you want but can create only db per share
- Cant share from Business critical to non-Business critical account, can override settings
- Sharing happens between same cloud provider and same region
- Sharing TO and FROM VPS is not supported





Secure Data Sharing

```
-- Create Share Container  
  
CREATE OR REPLACE SHARE my_share;
```

```
USE ROLE ACCOUNTADMIN;  
  
-- Create a reader account  
CREATE MANAGED ACCOUNT reader_acc  
ADMIN_NAME = reader_admin  
ADMIN_PASSWORD = 'Snow@12345'  
TYPE = READER;  
  
{"accountName": "CD82713", "loginUrl": "https://cd82713.ap-southeast-1.snowflakecomputing.com"}
```

```
ALTER SHARE my_share  
ADD ACCOUNT = CD82713  
SHARE_RESTRICTIONS = FALSE;
```

```
--Readers Account Code  
  
SHOW SHARES;  
  
CREATE DATABASE LA_DB2 FROM SHARE PMVXJTN.DX34919.MY_SHARE;  
  
SELECT * FROM LA_DB2.LA_SCHEMA.MY_SECURE_VIEW;
```





Secure Data Sharing

snowflake

+ Create

Home

Search

Projects

Data

Data Products

Marketplace

Apps

Private Sharing

Provider Studio

Partner Connect

AI & ML

Monitoring

Admin

Shared With YouShared By Your AccountReader Accounts

+ Reader Account

Search

1 Reader Account ⓘ

NAME ↑	CLOUD	REGION	LOCATOR	LOCATOR URL	CREATED	
READER_ACC	Azure	Central India (Pune)	EA66813	🔗	37 minut...	⋮





Secure Data Sharing

New Reader Account

Creating as ACCOUNTADMIN

Region

Azure - Central India (Pune) • Business Critical Edition

Account Name

Comment (optional)

Create a user name and password for the intended user. This user will have the ACCOUNTADMIN role and the ability to fully configure the account.

User Name

Password

Confirm password

Cancel

Create Account





Secure Data Sharing



+ Create

Home

Search

Projects

Data

Data Products

Marketplace

Apps

Private Sharing

Provider Studio

Partner Connect

AI & ML

Monitoring

Admin

Shared With You Shared By Your Account Reader Accounts

Search

Direct Shares



SFSALESSHARED.SFC_SAMPLES
_AZCENTRALINDIA

SAMPLE_DATA

SNOWFLAKE_SAMPLE_DATA

Shared 2 years ago



SNOWFLAKE

ACCOUNT_USAGE

SNOWFLAKE

Shared 2 years ago

Share

So

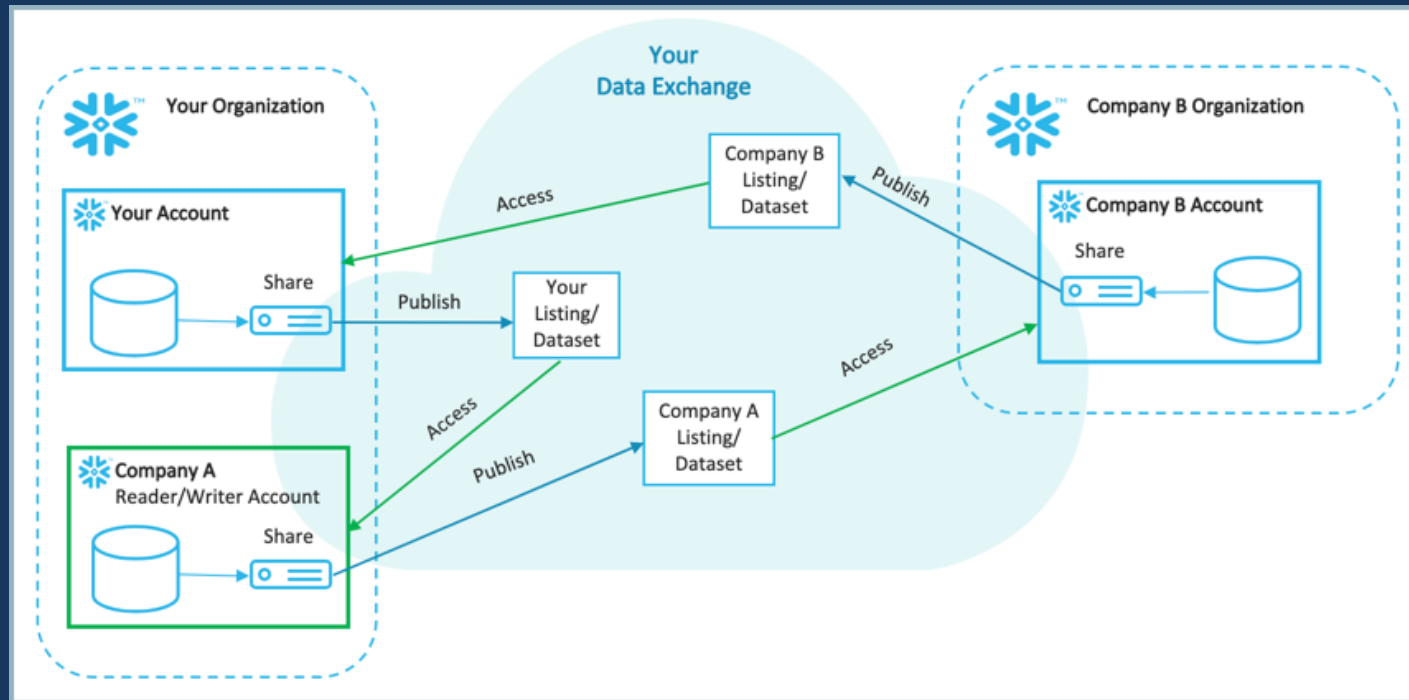
Publish to Specified Consumers
Publish to Marketplace
Create a Direct Share



Data Exchange



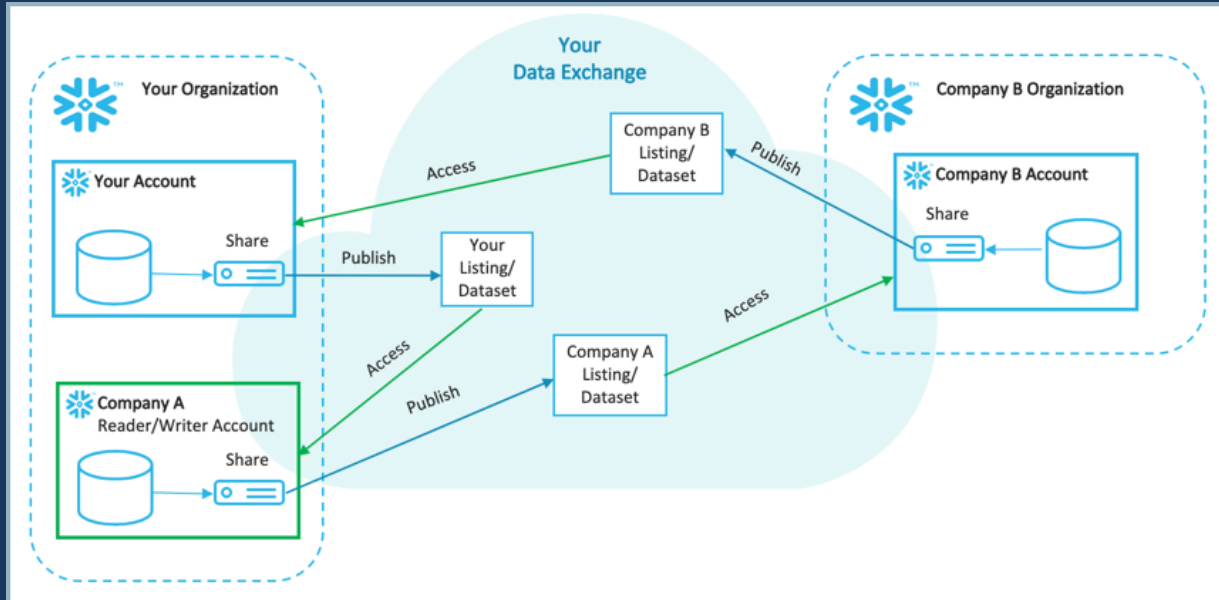
↔ Data Exchange



Source : Snowflake



↔ Data Exchange



- Data shared between invited group of entities
- Contact Support to enable it
- Can share data with partners
- Can share data within the org
- Manage the control
- No available for VPS

Source : Snowflake







Marketplace

- Provides wide variety of data/apps/services
- Discover data/apps as per you need
- Quick and hassle free access to data/apps
- Native to Snowflake
- Lots of free and freemium offering
- Not available for VPS





Marketplace

- Provides wide variety of data/apps
- Discover data/apps as per you need
- Quick and hassle free access to data/apps
- Native to Snowflake
- Lots of free and freemium offering

The screenshot shows the Snowflake Marketplace user interface. At the top, there is a search bar with the placeholder text "Search providers and data products". To the right of the search bar are links for "Browse Data Products", "Providers", and "My Requests". A small "Learn Anywhere" logo is visible in the top right corner. Below the search bar, there is a horizontal row of filter buttons: "Installed", "Instantly Accessible", "Free Data", "Weather Data", "Financial Data", "360-Degree Customer View", "Demand Forecasting", and "Japan Data". Below the filters, there is a promotional banner for "Join us at Snowflake Industry Day" with a megaphone icon. The banner text says "Learn how the below data and apps from Snowflake Marketplace solve key industry challenges." Below the banner, there are four product cards arranged in a 2x2 grid. Each card features a logo, the product name, and the provider name.

Search providers and data products

Browse Data Products Providers My Requests

Installed Instantly Accessible Free Data Weather Data Financial Data 360-Degree Customer View Demand Forecasting Japan Data

Join us at Snowflake Industry Day
Learn how the below data and apps from Snowflake Marketplace solve key industry challenges.

- Acxiom InfoBase® Sample Audience (Synthetic Data)**
Acxiom
- Braze Engagement Benchmarks**
Braze
- CARTO Analytics Toolbox (Core)**
CARTO
- Cybersyn eCommerce Benchmarks**
Cybersyn, Inc

